

Reference Guide

Macro Scripting

**Scripting using the Macro Script
Command Language**



© 2007–2023 InnovMetric Software Inc. All rights reserved. PolyWorks® is a registered trademark of InnovMetric Software Inc. InnovMetric, PolyWorks|Inspector, PolyWorks|Modeler, PolyWorks|Talisman, PolyWorks|Reviewer, IMAlign, IMMerge, PolyWorks|DataLoop, PolyWorks|PMI+Loop, PolyWorks|AR, PolyWorks|ReportLoop, "The Universal 3D Metrology Software Platform", and "The Smart 3D Metrology Digital Ecosystem" are trademarks of InnovMetric Software Inc.

SmartGD&T is a trademark of Multi Metrics Inc. All other trademarks are the property of their respective owners.

This manual, as well as the software described in it, is furnished under license and may be used or copied only in accordance with the terms of such license. The content of this document is furnished for informational use only, and is subject to change without notice. InnovMetric Software Inc. assumes no responsibility or liability for any errors or inaccuracies that may appear in this document.

Except as permitted by such license, reproduction in whole or in part in any way without written permission from InnovMetric Software is strictly prohibited.

Contents

7	1. Introduction
8	1.1 Introduction
8	1.2 Chapter contents
9	1.3 Related documentation
9	1.4 Technical support
11	2. Overview
12	2.1 Introduction
12	2.1.1 The PolyWorks approach to scripting
13	2.1.2 Task automation
13	2.2 Tools
15	2.3 Commands
17	2.4 Resources
17	2.4.1 Macro Script Reference guide
17	2.4.2 Command Reference Guide
17	2.4.2.1 Organization
19	2.4.2.2 Command syntax
21	2.4.3 Obsolete command update
21	2.4.4 Other resources
22	3. Macro Script Editor pane
23	3.1 Introduction
23	3.2 Elements of the Macro Script Editor pane
24	3.3 Menus
26	3.3.1 File menu
26	3.3.1.1 Opening macro scripts
28	3.3.1.2 Saving macro scripts
29	3.3.2 Edit menu
30	3.3.3 View menu
31	3.3.4 Debug menu
31	3.3.4.1 Introduction
32	3.3.4.2 Menu items
34	3.3.4.3 Other ways to run macro scripts
34	The MACRO EXEC command
34	Automatically launching macro scripts after operations
35	3.3.5 Tools menu
35	3.3.5.1 Options
36	General options
38	Text display options

40	3.3.6	Window menu
41	3.3.7	Help menu
41	3.4	Toolbars
43	3.5	Macro script window features
43	3.5.1	Macro script display and functions
43	3.5.2	Help when writing commands
43	3.5.2.1	Command Completion feature
46	3.5.2.2	Command Help feature
47	3.6	Shortcut keys
50		4. Command History Pane
51	4.1	Introduction
51	4.2	Elements of the Command History pane
51	4.2.1	Toolbar
52	4.2.2	Command history area
53	4.2.3	Command-line area
54	4.3	Using the pane to find application commands
56		5. Creating a Basic Macro Script
57	5.1	Introduction
57	5.2	The task to automate
58	5.3	The steps to create a macro script
59	5.4	Creating a new macro script
59	5.5	Recording actions and pasting them to the macro script
59	5.5.1	Recording the task
60	5.5.2	Pasting recorded commands
61	5.6	Saving the macro script
61	5.7	Checking for syntax errors
61	5.8	Running the macro script
61	5.9	Editing the macro script
62	5.9.1	Basic rules when writing macro scripts
63	5.10	Compatibility between versions
64		6. Macro Script Control Language
65	6.1	Introduction
66	6.2	General information
66	6.2.1	Rules used in writing macro scripts
67	6.2.2	Comments regarding sample scripts
67	6.3	MSCL commands
67	6.3.1	Command format
68	6.3.2	List of MSCL commands
69	6.4	Defining and using variables
69	6.4.1	Variable types
70	6.4.2	Creating variables
71	6.4.3	Assigning values to existing variables
72	6.4.4	Using the value stored in variables

73		6.4.4.1	Using braces to delimit variable names
73	6.4.5		Using variables as input and output arguments for commands
75	6.4.6		Array variables
76		6.4.6.1	Assigning values to an array
77		6.4.6.2	Using array indices
78		6.4.6.3	Using implicit indices
78		6.4.6.4	Finding the size of an array
80	6.4.7		Reserved variables
81	6.4.8		String variable modifiers
82	6.4.9		Incrementing and decrementing variable values
83	6.4.10		The scope of variables
83	6.4.11		Special variables
83		6.4.11.1	Macro script variables
84		6.4.11.2	Global variables
84	6.5		Constructing mathematical expressions
84	6.5.1		Mathematical operators
85	6.5.2		Mathematical functions
86	6.5.3		Mathematical expressions
88	6.6		Constructing logical expressions
88	6.6.1		Comparison operators
89	6.6.2		Logical operators
90	6.6.3		Logical expressions
92	6.6.4		Evaluating logical expressions
93	6.7		Controlling the flow of execution
93	6.7.1		Constructing conditions
93	6.7.2		Conditionalizing the execution of statements using the IF statement
93		6.7.2.1	One condition
94		6.7.2.2	Multiple conditions
95	6.7.3		Repeating the execution of statements using the WHILE statement
97	6.7.4		Other examples
97	6.8		Controlling the scope of variables
101	6.9		Converting data types
101	6.9.1		Standard data type conversion
102	6.9.2		Explicit data type conversions
103	6.9.3		Implicit data type conversions
105			7. Getting Started
106	7.1		Introduction
106	7.2		Application commands
106	7.2.1		Specifying objects and object properties
106		7.2.1.1	Specifying objects
107		7.2.1.2	Specifying object properties
107	7.2.2		Getting information about objects in the tree view
109	7.2.3		Working with text boxes and check boxes
111	7.2.4		Using QR codes
112	7.2.5		Replacing obsolete application commands

113	7.2.5.1	Automatically upgrading scripts containing obsolete commands
115	7.3	Utility commands
115	7.3.1	Refreshing the screen and echoing commands
116	7.3.2	Querying the user
126	7.3.3	Pausing macro scripts
127	7.3.4	Reading parameters from a text file
130	7.3.5	Working with string values
134	7.3.6	Using shared variables
137		8. Basic Programming Concepts
138	8.1	Introduction
138	8.2	Modular programming
138	8.2.1	An example of modular programming
140	8.2.2	Introducing macro script variables
143	8.2.3	Using output arguments
144	8.3	Error handling
147		9. Sample Macro Scripts
148	9.1	Introduction
148	9.2	IMAlign macro script
148	9.3	PolyWorks Modeler macro script
150	9.4	PolyWorks Inspector macro scripts
150	9.4.1	Macro script with an alignment and a data color map
151	9.4.2	Macro script with an array variable
153	9.5	A multiapplication script
154	9.5.1	The tasks to accomplish in each module
155	9.5.2	The macro scripts to create
157	9.5.3	Running the script
158		Glossary
159		Index

Introduction



1

The IMAlign, the PolyWorks|Modeler, and the PolyWorks|Inspector modules, as well as the Workspace Manager, all offer a Macro Script Editor pane. It is a tool that allows recording user actions performed to accomplish a task, and saves the actions as commands to a macro script file. The recorded commands can be edited if desired. The final goal is to load the macro script in the module's Macro Script Editor and run it to reperform the recorded task.

1.1 Introduction

The Macro Scripting Reference Guide is intended for users of the PolyWorks® Metrology Suite. It explains how to build powerful macros that allow automating tasks in the IMAlign™, the PolyWorks|Modeler™, and the PolyWorks|Inspector™ modules. Macros can also be built in the Workspace Manager that execute local scripts in, and send commands to, one or more of those modules.

Commands that are useful for most macro scripts are presented and basic but important programming concepts are presented. Finally, examples of macro scripts for the IMAlign, the PolyWorks|Modeler, and the PolyWorks|Inspector modules are presented.

This guide provides concepts that are illustrated using examples of the scripting language. As users work through the examples, they should gain understanding and confidence that will serve in building their own scripts. The only real way to learn is to start simple and build toward personal objectives. Eventually, gains in time, repeatability, and productivity will be the rewards.

1.2 Chapter contents

This document contains the following chapters:

1. Introduction

This chapter introduces macro scripting in the PolyWorks Metrology Suite.

2. Overview

This chapter presents the PolyWorks approach to scripting and then introduces the tools, commands, and resources used to create macro scripts in the PolyWorks Metrology Suite.

3. Macro Script Editor pane

This chapter describes the Macro Script Editor tool.

4. Command History Pane

This chapter presents the Command History pane and explains its usefulness in macro scripting.

5. Creating a Basic Macro Script

This chapter demonstrates the workflow used to create a macro script by taking a simple task and realizing the corresponding macro script.

6. Macro Script Control Language

This chapter presents the Macro Script Control Language, a simple but powerful programming language offered in the PolyWorks Metrology Suite. It allows adding standard flow control commands to macro scripts (such as the IF and WHILE command structures), managing variables, evaluating mathematical expressions, performing error handling, and more.

7. Application commands

This chapter presents applications commands that are useful to get started writing macro scripts.

8. Utility Commands to Get Started

This chapter presents utility commands that are useful to get started writing macro scripts.

8. Basic Programming Concepts

This chapter presents programming concepts to help users write more efficient and more robust macro scripts.

9. Sample Macro Scripts

This chapter presents samples of macro scripts that perform tasks in the IMAlign, the PolyWorks|Modeler, and the PolyWorks|Inspector modules.

1.3 Related documentation

The following documents offered in PDF format, available from the *Help > Reference Guides* menu of the Workspace Manager, offer related information:

- The *PolyWorks® Reference Guide* presents the PolyWorks Metrology Suite, including the Workspace Manager, and explains how to customize the user interface by means of visual layouts. It also explains how to open modules from the Workspace Manager. Finally, it describes the installation procedure for node-locked, computer-locked, and floating license key files.

All reference documentation can be accessed from the Workspace Manager. See its *Help > Reference Guides submenu*.

1.4 Technical support

Report any problems, or send your suggestions, directly to InnovMetric Software at www.innovmetric.com. The InnovMetric Software technical support team can also be contacted by e-mail at support@innovmetric.com.

Overview

A decorative graphic on the right side of the page. It features a large, dark blue circle containing the white number '2'. Below this circle, a white circuit board trace extends downwards, then turns left and continues horizontally across the page. The trace has several circular nodes or vias. The background is a light gray gradient.

2

This chapter provides an overview of the environment for macro script writing in the PolyWorks Metrology Suite.

2.1 Introduction

This chapter presents the PolyWorks approach to scripting. It then presents the tools and types of commands that are used to write scripts and describes the resources that are available for each.

2.1.1 The PolyWorks approach to scripting

The key choices made in designing the scripting language were to use a command-based approach and to keep it simple. PolyWorks users are familiar with using menu items to perform actions. So, it was only natural to make available commands that mimic the menu items. For example, the *Edit > Delete > Objects* menu item has the following equivalent *application* command in a script:

```
EDIT OBJECT DELETE (string)
```

where string is an optional argument (i.e., the name of the object to delete); if no name is provided, the current object selection in the project is considered.

Care was taken to incorporate help items for the user as well:

- There are command completion and command help tools directly available in the Macro Script Editor.
- The Macro Script Editor also offers the Command Reference guide in HTML format that provides the syntax for all commands.
- Each time the user performs an action using the GUI (e.g., using menu items, toolbars, shortcut keys), the resulting application command appears in the Command History pane.

Application commands can be run in a macro script to do the same actions in the application (e.g., PolyWorks|Inspector).

There are several key advantages to this approach:

- Scripting is available to all users immediately – the learning curve is shorter than the one associated with object-oriented scripting languages.
- The scripting language itself has been kept simple – only a limited number of commands are offered, so again the learning curve is kept to a minimum.
- Help is made available as the user constructs scripts.

Scripting a complete workflow in PolyWorks

The Workspace Manager has its own Macro Script Editor tool, and as the Workspace Manager sits above the executing modules and knows what is going on, it is the natural site for the development of scripts for an entire workflow in the PolyWorks Metrology Suite.

Basic commands are available that allow opening the IMAlign, the PolyWorks|Modeler™, and the PolyWorks|Inspector™ modules, and PolyWorks|Reviewer™, executing remote commands in the modules, executing scripts written for each module, and then closing each module. Another command allows making the necessary specifications to run the IMMerge module. As a result, the tools exist to script a complete PolyWorks process.

2.1.2 Task automation

A task can be simple but useful, such as a combination of keystrokes used repeatedly, or more complex such as changing the value of a parameter for all circles in a project. Scripting allows creating a personal set of tools to save time and perform tasks in a predefined, controlled manner. And scripts can be run on the shop floor by employees who don't need to understand the PolyWorks Metrology Suite to produce exact results.

User actions in a module can be automatically recorded and saved to a macro script as application commands to be played back. Macro scripts are Unicode files that can be edited in the Macro Script Editor, or by using any text editor that supports Unicode. Module-specific commands will be referred to as *application commands*. Other commands, called *utility commands*, are also offered that are not module specific; they are available in all modules. For example, a utility command that allows querying the user for a numeric value by way of a dialog box.

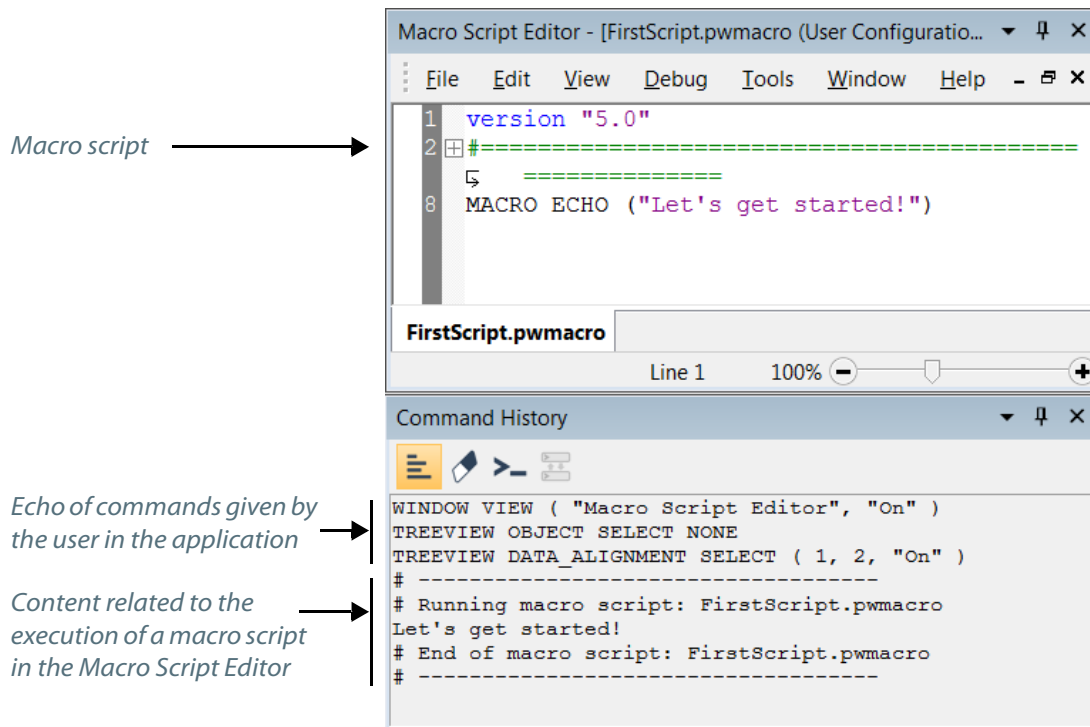
Real-world tasks require that certain commands in a macro script be used repetitively, or conditionally, depending on run-time conditions. For this reason, PolyWorks offers control commands in the form of the Macro Script Control Language that allows adding basic control structures (e.g., IF, WHILE) to macro scripts. The Macro Script Control Language is a simple but powerful programming language supported by the Macro Script Editor.

2.2 Tools

Writing, testing and executing macro scripts requires using the Macro Script Editor pane and its companion, the Command History pane.

The Macro Script Editor is the tool to use to write and save scripts, check their syntax, and run them. It also offers useful display options, breakpoints to control macro script execution, as well as command help and completion features to increase efficiency. This subject is presented in [Chapter 3 Macro Script Editor pane](#).

Figure 2.1 The Macro Script Editor pane and the Command History pane. The FirstScript.pwmacro macro script has been run in the Macro Script Editor and the output can be seen in the Command History pane.



The Command History pane displays the commands given by the user in the application using menu items, toolbars, and mouse clicks used to select objects and control object visibility. These application commands can be directly copied to a macro script and executed in the Macro Script Editor to repeat the same operations in the application. The Command History pane can also be used to give individual commands to see what they do and to see error messages if they are incorrectly written. This subject is presented in [Chapter 4 Command History Pane](#).

In [Figure 2.1](#), both panes are shown using the PolyWorks|Inspector module. The macro script open in the Macro Script Editor has been executed, sending a message to the Command History pane. The Command History pane shows commands given in the application by the user as well as information concerning the execution of the macro script.

2.3 Commands

Three types of commands can be used in a macro script:

- Application commands

These commands carry out operations in the application. They are the commands to use to import objects, extract measured components, generate report tables, and more. Note that where a dialog box is used in the application to perform an operation, for example to best-fit align Data objects to Reference objects in PolyWorks|Inspector, several application commands may have to be given in a script to perform the same operation.

Application commands can be given in the Command History pane to test command behavior and syntax.

- Macro Script Control Language (MSCL) commands

The MSCL is a simple but powerful programming language that adds control to script execution in the form of loops or command execution based on conditions. In addition, it offers mathematical, comparison, and logical operators and expressions. MSCL commands, combined with utility commands (see next bullet) let you create powerful and flexible scripts. For example, a script could query the user for a tolerance value, loop through all circle objects in the project, and for unmeasured circles apply the tolerance to a specific dimensional control. This subject is presented in [Chapter 6 Macro Script Control Language](#).

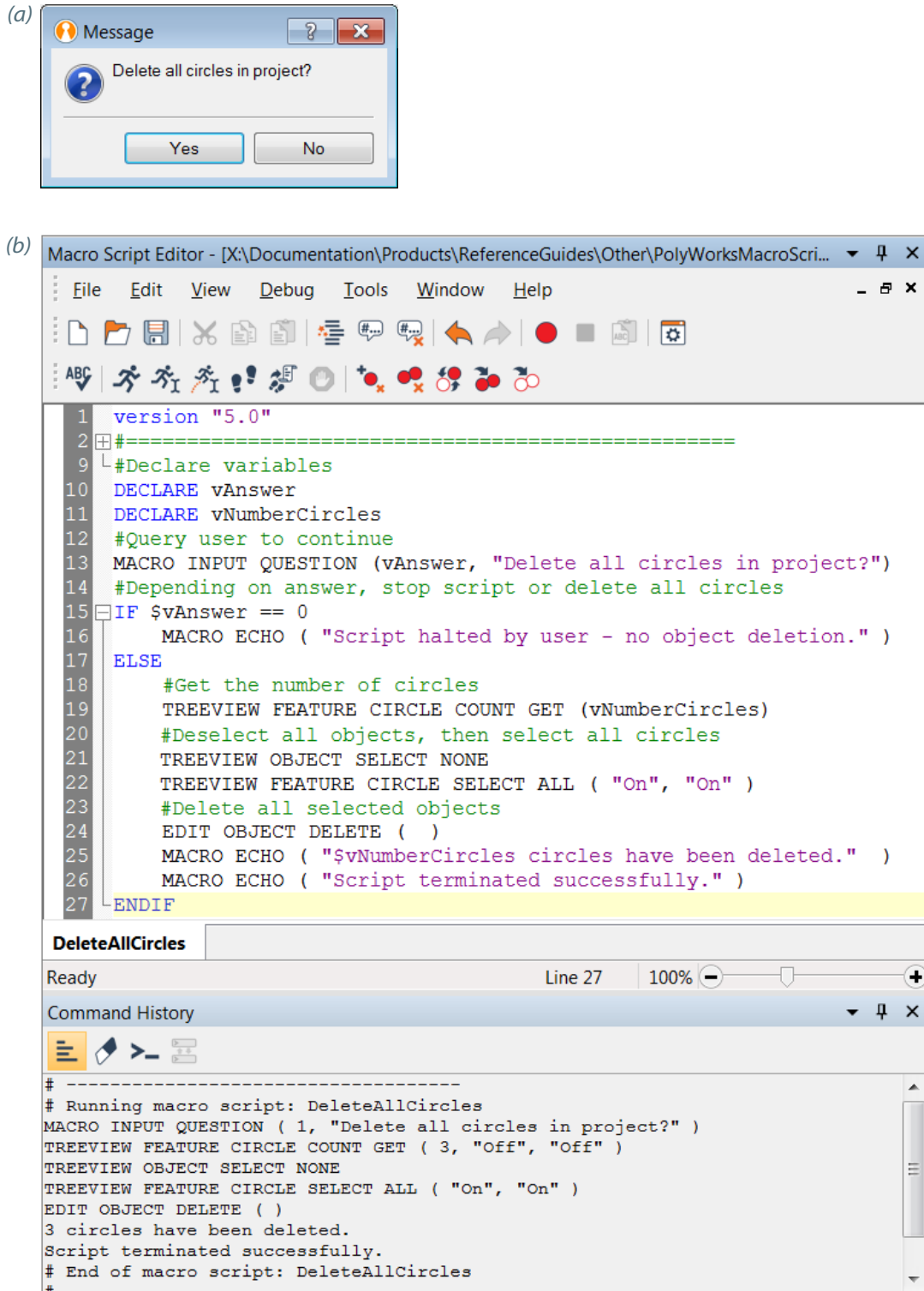
- Utility commands

Utility commands let you get input from the user, send output to the Command History pane, handle errors during macro script execution, run other macro scripts, work with text files and strings, and more. Utility commands are available in all modules that have a Macro Script Editor. Most utility commands can be given in the Command History pane to test command behavior and syntax.

For demonstration purposes, a simple macro script that selects and deletes all circle objects in the project is shown in [Figure 2.2](#). It contains all three types of commands. The user is queried to confirm doing the operation – the answer is stored in a variable. Depending on the answer, the operation is carried out or not. A message is sent to the Command History pane indicating the result of the operation and, if carried out, the number of circles deleted. Note that comments have been added to the macro script to describe the actions that commands perform.

The workflow for macro script creation is presented in [Chapter 5 Creating a Basic Macro Script](#).

Figure 2.2 A macro script that deletes all circles in the open project if the user presses the Yes button. In (a), the dialog box displayed by the script to query the user to continue. In (b), the macro script – MSL commands are in blue and comments are in green. The Command History pane contains the output of the MACRO ECHO utility commands.



2.4 Resources

Several resources are available to help users learn how to successfully write scripts in PolyWorks. They are presented in the subsections that follow.

2.4.1 Macro Script Reference guide

The present reference guide presents the entire scripting environment, explains how to create macro scripts, presents in detail the Macro Script Control Language (MSCL), and provides useful examples that users can paste into a macro script and try out in order to understand the commands.

In addition, additional commands are presented to help users get started with scripting, and then basic programming concepts are presented to users help create more robust macro scripts more efficiently. These subjects are presented in [Chapter 7 Application commands](#), [Chapter 8 Utility Commands to Get Started](#), and [Chapter 8 Basic Programming Concepts](#).

Finally complete sample macro scripts are provided that perform a described task for the IMAAlign, the PolyWorks|Modeler, and the PolyWorks|Inspector modules, as well as a multiapplication macro script. For more information, see [Chapter 9 Sample Macro Scripts](#).

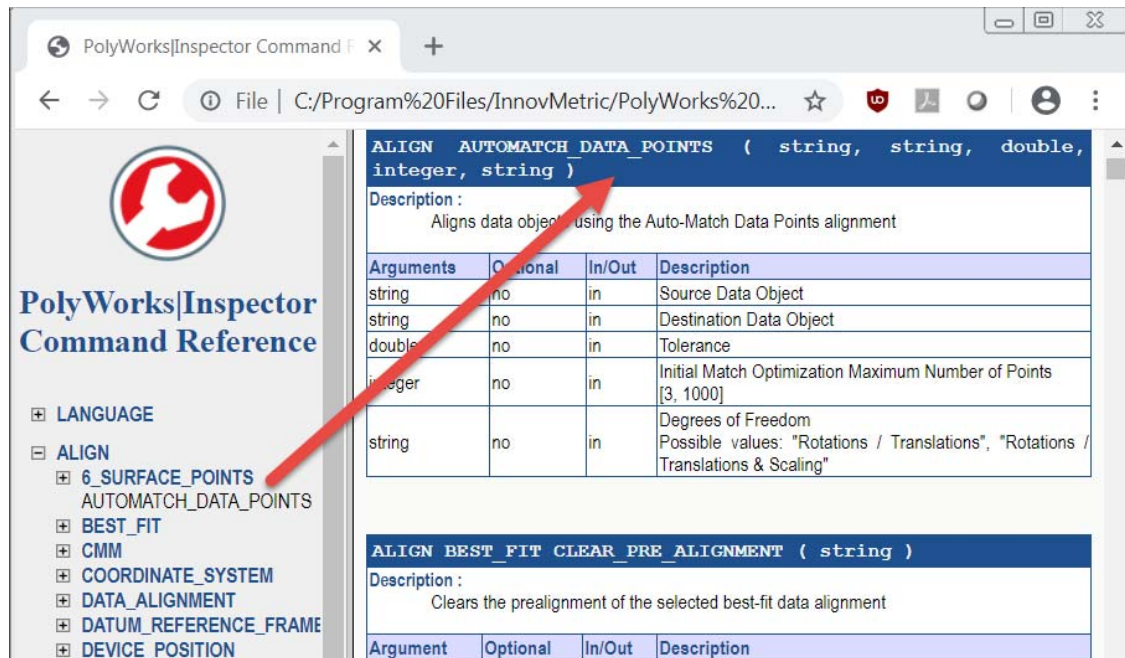
2.4.2 Command Reference Guide

The *Help > Commands* menu item of the Macro Script Editor displays the user's web browser with HTML help on MSCL and utility commands as well as application commands for the module. An example of the guide is shown in [Figure 2.3](#).

2.4.2.1 Organization

Most top-level items are related to application commands that are specific to the current application (e.g., PolyWorks|Inspector). Clicking an item opens the command branch, displaying other subbranches or terminal items. Information is provided for terminal items, as they form a complete command, such as a command description, the description of command arguments, and special notes when required.

Figure 2.3 The Command Reference guide provides a description and the syntax of application commands for the current module, utility commands, and the elements of the Macro Script Control Language.



Other top-level items provide information on MSCL commands and utility commands. The Language branch contains the elements of the MSCL and the DATA FILE and MACRO branches contain the utility commands.

Figure 2.4 displays the following command branches:

Branch	Description	Reference
APPLICATION COMMANDS		
Most branches, except those named below.	Commands that instruct the application to perform operations that correspond to menu items, toolbar items, and actions performed using the mouse that affect object selection and visibility status.	Examples given throughout this guide as well as Chapter 7 Application commands and Chapter 9 Sample Macro Scripts .
MSCL COMMANDS		

Branch	Description	Reference
LANGUAGE	The elements of the Macro Script Command Language: commands, reserved variables, and mathematical functions.	Chapter 6 Macro Script Control Language
UTILITY COMMANDS		
DATA FILE	Commands that manipulate text files: create files, append lines, and read file contents line by line and field by field.	Section 7.3.4 Reading parameters from a text file
MACRO	Commands that send messages to the Command History pane, control the execution of macro scripts, provide error-handling mechanisms, and more.	Section 7.3.3 Pausing macro scripts , Section 8.2 Modular programming , and Section 8.3 Error handling .
MACRO INPUT (subbranch)	A group of commands that can be used to query information from a user using dialog boxes at macro script run time.	Section 7.3.2 Querying the user ; also used in many examples.

MSCL commands can only be given in the Macro Script Editor while application commands and utility commands can be given in both the Macro Script Editor and the Command History pane.

Note that the OBSOLETE branch contains commands that no longer exist in the current version of the PolyWorks Metrology Suite but that are supported for compatibility reasons. Those commands should no longer be used; for more information, see [Section 2.4.3 Obsolete command update](#).

2.4.2.2 Command syntax

The Command Reference guide provides the complete syntax for each command, as shown in the example below.

MACRO INPUT STRING (string, string, array of strings, string)			
Description :			
Pops up an interface to query the user for a string value			
Arguments	Optional	In/Out	Description
string	no	out	String value
string	yes	in	Window title
array of strings	yes	in	Window prompt
string	yes	in	Suggested value

Figure 2.4 Command Reference branches that are not application specific.

*Elements of the Macro Script
Control Language*

[-] LANGUAGE
++
--
BREAK
CONTINUE
DECLARE
ELSE
ENDIF
ENDWHILE
EXPR
EXPR_I
IF
[+] MATHEMATICAL FUNCTIONS
[-] RESERVED VARIABLES
_APP_NB_BIT
_DOLLAR
_INSTALL_PATH
_NEWLINE
_PI
_PWK_FILES_PATH
_QUOTES
_TAB
_TEMP_PATH
_USERCONFIG_PATH
SET
SIZE
STRING_DECIMAL
VERSION
WHILE

Examples of utility commands

[-] DATA_FILE
[+] APPEND
CREATE
[-] PROPERTIES
[+] NB_FIELDS_IN_LINE
[+] NB_LINES
[-] READ
COLUMNS
LINE
LINE_FIELD
LINE_FIELDS
WAIT_UNTIL_MODIFIED
[-] MACRO
[+] ARRAY
[+] CHRONOMETER
[+] COMMAND
ECHO
END
END_ON_ERROR
[+] EXEC
[+] FILE
GET_ERROR_STATUS
GUIDING_INSTRUCTIONS
[-] INPUT
DOUBLE
FILE_PATH
FOLDER_PATH
INTEGER
MULTIPLE_PARAMETERS
PASSWORD
PWK_OBJECTS
QUESTION
STRING
[+] INTERACTIVE_MODE
INTERACTIVITY
OUTPUT_ARGUMENT
PAUSE
[+] SHARED_VARIABLE
SLEEP
[+] STRING

The command shown in the figure can take four arguments – they are located after the command, between parentheses and separated by commas. A short description of the command is provided, and a note if required. In addition, the following information is provided for each argument:

- **Arguments:** The argument's type (e.g., string, integer, double, array of strings).
- **Optional:** Whether the argument is optional or not.
- **In/Out:** Whether the argument is an In argument (i.e., provides information to the command) or an Out argument (i.e., receives information from the command).

- **Description:** Its description or purpose.

Note that some commands do not have arguments. If a command in a macro script does not have the correct syntax, it will fail and the macro script will stop. The Macro Script Editor offers the *Check Syntax* functionality to make sure that commands have the correct syntax before running a script.

2.4.3 Obsolete command update

As the PolyWorks Metrology Suite evolves, certain application commands become obsolete and are replaced by new commands that represent the current state of the software. For example, the SELECT OBJECT_TREE ALL command is now the TREEVIEW OBJECT SELECT ALL command.

An obsolete command may be replaced in the application by a new command that is identical (i.e., parameters, function) except for the name, or may be superseded by a different command (i.e., name, parameters, function). Obsolete commands continue to be valid, when possible, to maintain backward compatibility with existing macro scripts.

The *Upgrade* command of the Macro Script Editor allows identifying and resolving issues related to obsolete commands. This subject is presented in [Section 7.2.5 Replacing obsolete application commands](#).

2.4.4 Other resources

Training and complete macro scripts are available to script writers:

- Scripting may be taught in certain advanced PolyWorks training courses.
- Macro scripts can be found on the InnovMetric Web site, in the Macro Zone area of the Technical Support Zone. Certain scripts have been provided by PolyWorks users while other scripts have been created by specialists at InnovMetric Software Inc.

Consulting and integration services are available to users who desire to have macro scripts developed for them:

- InnovMetric Software Inc. offers integration services that include the creation of automated solutions through macro-script programming. For more information, go to the InnovMetric website at www.innovmetric.com.
- The PolyWorks user community includes users that have become very proficient in scripting tasks and who make themselves available as consultants to other users.

Macro Script Editor pane

A decorative graphic on the right side of the page. It features a large, dark blue circle containing the white number '3'. Below this circle, a white line resembling a circuit board trace extends downwards, then turns left and continues horizontally across the page. Several small white circles, representing solder points or vias, are placed along this circuit-like path.

3

This chapter presents the Macro Script Editor that is used to create, save, test, and run macro scripts.

3.1 Introduction

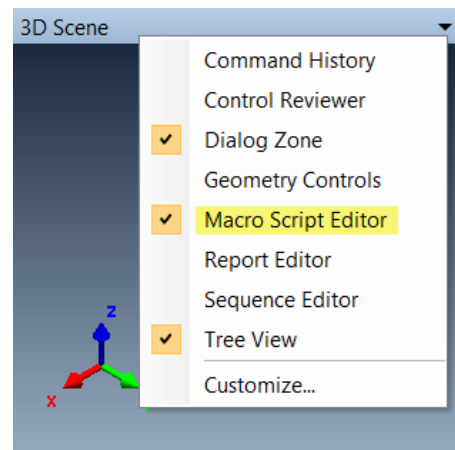
The Macro Script Editor is an integrated environment that allows creating, testing, and running macro scripts that perform actions in your PolyWorks Metrology Suite application (e.g., PolyWorks|Inspector). The Macro Script Editor resides within its own dedicated pane.

3.2 Elements of the Macro Script Editor pane

To access the Macro Script Editor, click the title bar of any pane and click **Macro Script Editor** (shown to the right), or choose the *Tools > Macro Scripts > Macro Script Editor* menu item. The Macro Script Editor pane, shown in [Figure 3.9](#), is displayed.

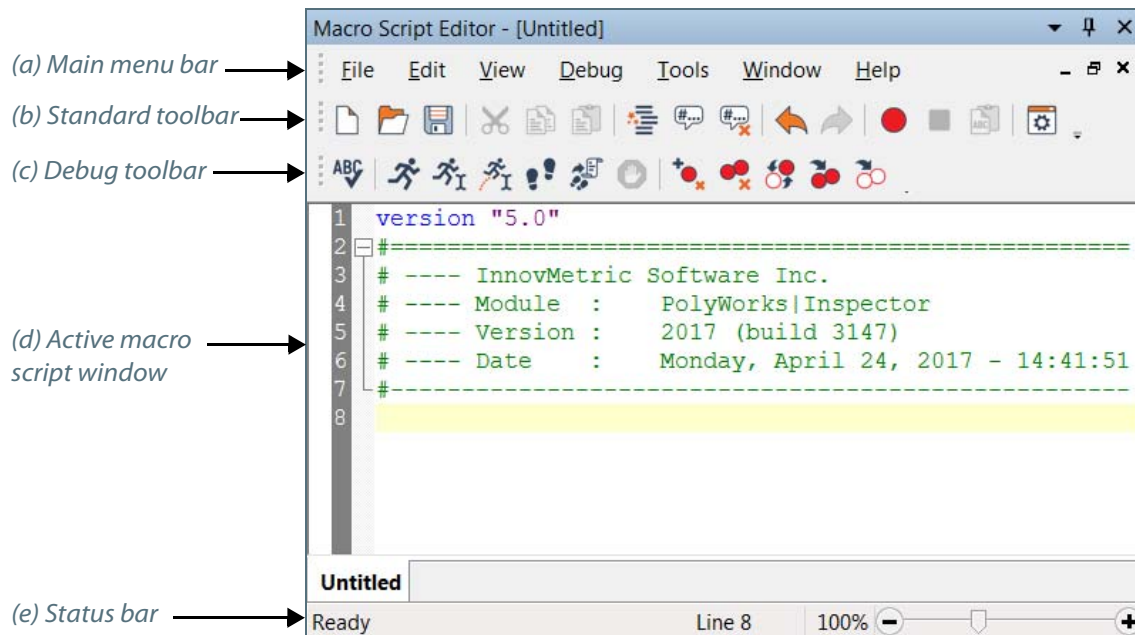
The pane features the following elements which are identified in [Figure 3.9](#):

- Menu bar – Provides all the functionalities offered by the editor.
- Toolbars – The Standard toolbar provides quick access to the main functionalities of the editor, while the Debug toolbar offers quick access to most of the functionalities offered on the *Debug* menu.
- Script window – An area where macro scripts can be written, saved, tested for syntax errors, run, and edited. The window offers tools to help users create macro scripts easily and quickly. Note that shortcut keys are offered to work efficiently within the commands in the script.
- Status bar – An area where information concerning the status of operations is displayed. In addition, it offers a scroll bar to change the zoom level of the active text window.



The sections that follow describe the Macro Script Editor. First, the menus and the options offered by the Macro Script Editor are explained. Then, an example of a macro script is given to illustrate different options and functionalities that are built into the script window to make it easier to work with, test, debug macro scripts. In addition, tools that make it easier to write commands are presented. Finally, the shortcut keys to be used in the script window are described.

Figure 3.1 The Macro Script Editor features (a) a main menu bar, (b) and (c) toolbars, (d) a text window for each open script, and (e) a status bar.

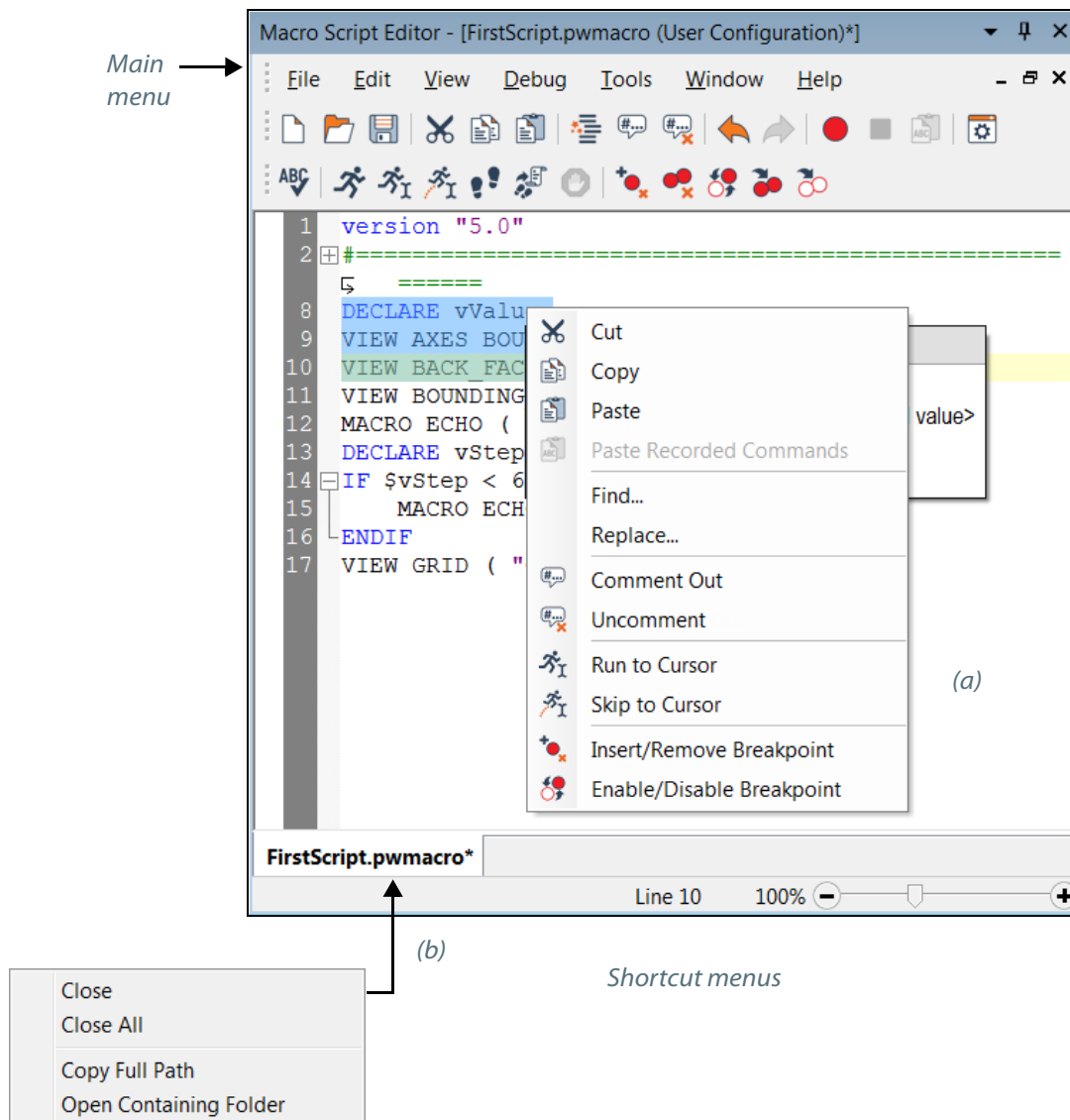


3.3 Menus

This section provides presents each menu item offered on the Macro Script Editor's menu bar. For each menu item a description is provided as well its shortcut key and toolbar button, if available. Note that most of the menu items are only available when a script window is open.

Certain menu items are also offered on the shortcut menu that is available on right-clicking a document tab or a command in a script window. Examples of these menus are shown in [Figure 3.2](#).

Figure 3.2 Right-clicking (a) a command or (b) a document tab displays two shortcut menus. In (a), the right-click was performed on a selection of lines.



3.3.1 File menu

The *File* menu offers the following items:

Menu item + shortcut	Description	Toolbar button
<i>New</i> <i>Ctrl+N</i>	Creates a new macro script in a new window.	
<i>Open</i> <i>Ctrl+O</i>	Opens a macro script in a new script window. This operation is presented in Section 3.3.1.1 Opening macro scripts .	
<i>Close</i>	Closes the active macro script. If the script has been modified, a message window opens asking to save the changes. To save the changes, press the Yes button, otherwise press the No button. To cancel the file close operation, press the Cancel button.	
<i>Close All</i>	Closes all macro scripts.	
<i>Save</i> <i>Ctrl+S</i>	Saves the contents of the active macro script under the current file name. This operation is presented in Section 3.3.1.2 Saving macro scripts .	
<i>Save As</i>	Saves the contents of the active macro script to a file. This operation is presented in Section 3.3.1.2 Saving macro scripts .	
<i>Recent Scripts</i>	Presents a list of recent scripts. Clicking a list item opens the macro script.	

3.3.1.1 Opening macro scripts

To open a macro script file, choose the *File > Open* menu item. The macro script file browser, shown in [Figure 3.3](#), is displayed. It offers the items that follow:

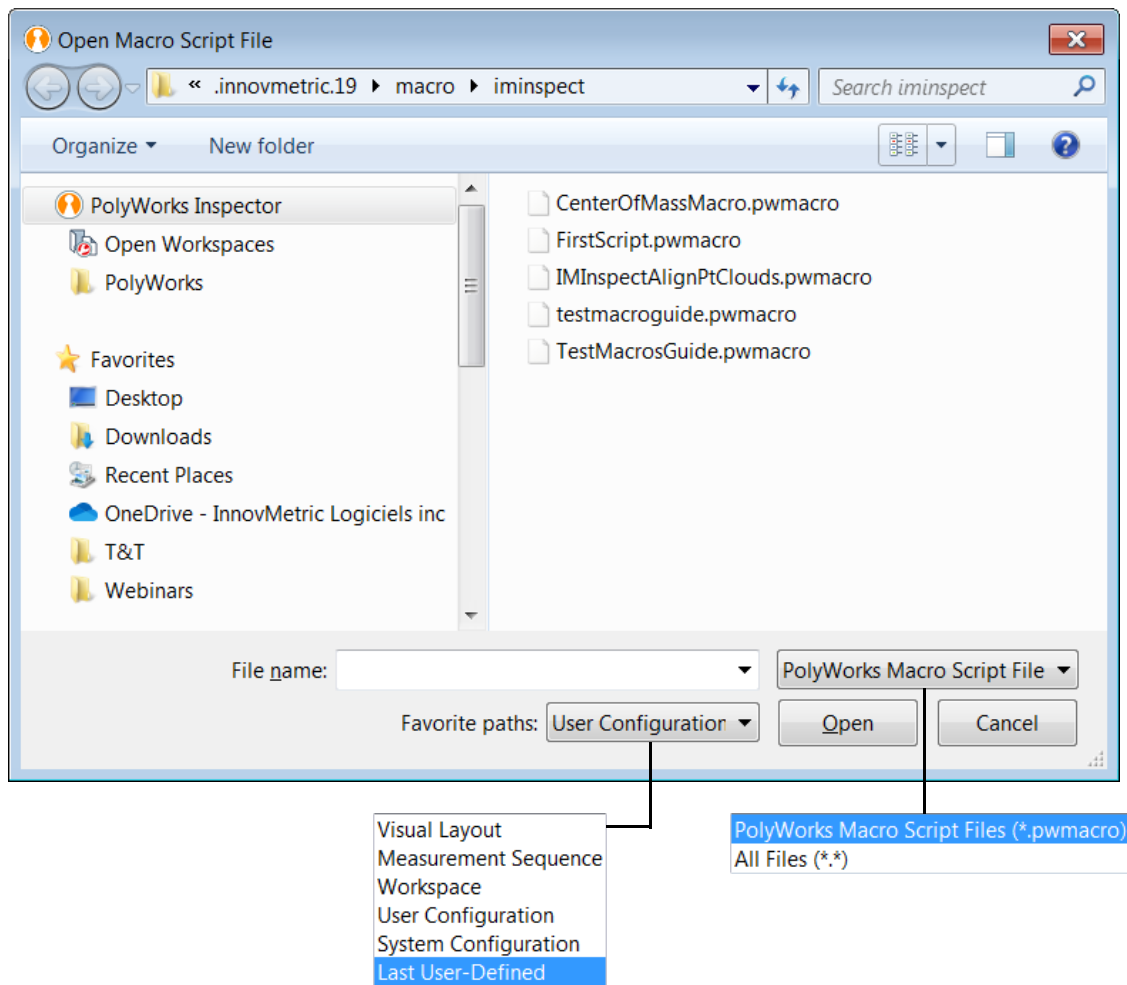
File name

A text box that allows specifying the name of the file to open.

(file filter)

The file filter list box is located after **File name**. It allows specifying the file types to find. Choose from **PolyWorks Macro Script File (*.pwmacro)** (only show files with the .pwmacro file extension) and **All Files (*.*)**. By default, the filter is set to PolyWorks Macro Script File.

Figure 3.3 The file browser used to open macro scripts.



Favorite paths

A list box, located at the bottom left of the browser, that allows specifying a preferred location for the file browser. Choose from the values presented in the table:

Path	Description
Visual Layout	The application's (i.e., module or the Workspace Manager) current visual layout.
Measurement Sequence	The measurement sequences of the current piece. This path is used to identify macro scripts located in the measurement sequences of a piece in order to open them in the Macro Script Editor pane.

Path	Description
Workspace	The module's active workspace.
User Configuration	The user's personal configuration folder.
System Configuration	The PolyWorks Metrology Suite global folder; only offered when the folder contains macro scripts for the module that the macro script browser is associated with (e.g., PolyWorks Modeler).
Last User-Defined	The path where the last macro script was opened from or saved to.
Project	Points to a special temporary folder when it is not empty. This choice is offered when on opening a project where the customizable toolbar links to a project macro script with the same name but not the same definition as an existing macro script on disk, the macro script is copied to a special temporary folder. In this way, access to the macros that have the same name but a different content is possible. This choice is not offered in the macro script browser of the Workspace Manager.

Press the **Open** button to open the file in the Macro Script Editor, or the **Cancel** button to cancel the operation.

It is also possible to open a script file by dragging it from an application, such as a file browser, onto the Macro Script Editor pane. If multiple files are dragged, they are all opened in individual cascading windows.

3.3.1.2 Saving macro scripts

To save a macro script file, choose the *File > Save* menu item to save the script in the active macro script window. This command displays a macro script file browser, similar to the one shown in [Figure 3.3](#), that offers the items that follow:

File name

A text box that allows specifying a file name. The name cannot contain the special characters that must be excluded in file names (e.g., *, ?, /, \), and the name is not case sensitive.

Save as type

A list box that allows specifying the type of file. Choose from **PolyWorks Macro Script Files (*.pwmacro)** and **All Files (*.*)**. The default value is PolyWorks Macro Script Files. When **All Files** is specified, files may be saved with any or no file extension.

Favorite paths

A list box, located at the bottom of the file browser, that allows specifying a preferred location for saving macro script files. Choose from the values presented in the table:

Path	Description
Visual Layout	The current visual layout of the application (i.e., instance of module) or of the Workspace Manager.
Workspace	The module's active workspace.
User Configuration	The user's personal configuration folder.
Last User-Defined	The path where the last macro script was opened from or saved to.

Press the **Save** button to save the file, or the **Cancel** button to cancel the operation.

3.3.2 Edit menu

The *Edit* menu offers the following items that apply to the active script window:

Menu item + shortcut	Description	Toolbar button
<i>Undo</i> <i>Ctrl-Z</i>	Undoes the last operation. Available when an operation has been performed.	
<i>Redo</i> <i>Ctrl-Y</i>	Reperforms the last undone operation. Available when an operation has been undone.	
<i>Cut</i> <i>Ctrl-X</i>	Cuts the selected text and moves it to the clipboard. Available when text is selected.	
<i>Copy</i> <i>Ctrl-C</i>	Copies the selection to the clipboard. Available when text is selected.	
<i>Paste</i> <i>Ctrl-V</i>	Inserts the clipboard contents at the cursor position. Available when the clipboard contains text.	
<i>Delete</i> <i>Del</i>	Deletes the selection. Available when text is selected.	
<i>Select All</i> <i>Ctrl-A</i>	Selects the entire document.	

Menu item + shortcut	Description	Toolbar button
<i>Find</i> <i>Ctrl-F</i>	Displays the dialog box show in Figure 3.4 (a) to find the specified text.	
<i>Replace</i> <i>Ctrl-H</i>	Displays the dialog box show in Figure 3.4 (b) to find and replace the specified text.	
<i>Auto Indent All</i> <i>Ctrl-I</i>	Automatically indents the contents of the active macro script.	
<i>Comment out</i> <i>Ctrl-T</i>	Transforms the block of text to comment format.	
<i>Uncomment</i> <i>Ctrl-U</i>	Removes the comment format from the block of text.	
<i>Upgrade</i> <i>Ctrl-G</i>	Automatically processes a script to resolve issues related to obsolete application commands; the original file is saved with a time stamp added to its name. This operation is documented in Section 7.2.5 Replacing obsolete application commands .	

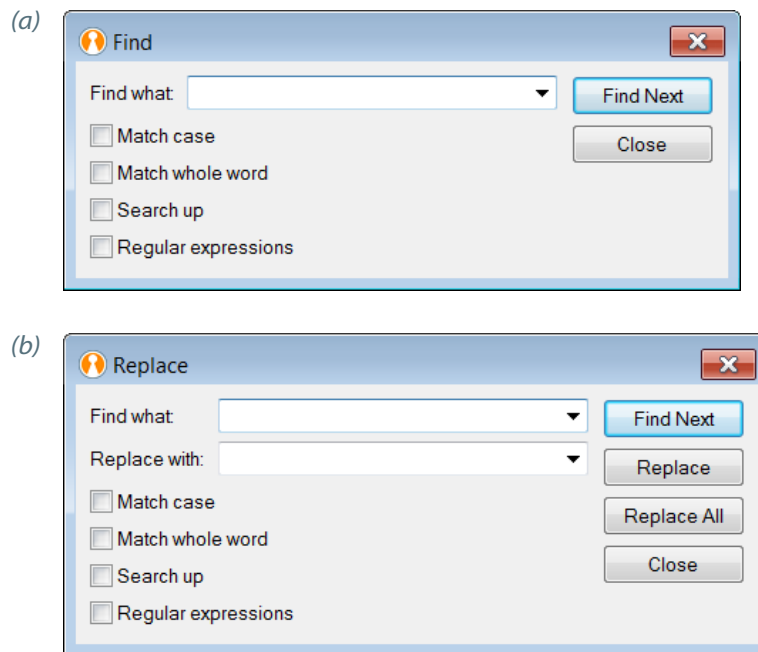
3.3.3 View menu

The *View* menu offers the following items:

Menu item + shortcut	Description	Toolbar button
<i>Zoom In</i> <i>Ctrl+Page Up</i>	Increases the magnification of the active macro script window.	
<i>Zoom Out</i> <i>Ctrl+Page Down</i>	Decreases the magnification of the active macro script window.	
<i>Zoom 100%</i> <i>Ctrl+Shift+Page Up</i>	Sets the magnification of the active macro script window to 100%.	

Zooming the active script window can also be performed using the slider located on the right of the status bar, or when the cursor is placed in a script window, by using the mouse wheel (i.e., scroll forward to increase the zoom, and scroll backward to decrease the zoom).

Figure 3.4 In (a), the Find dialog box, and in (b), the Replace dialog box.



3.3.4 Debug menu

The *Debug* menu offers items that allow testing and running, or executing, the active macro script.

This section introduces the running of macro scripts, presents the *Debug* menu, and presents other ways of executing macro scripts.

3.3.4.1 Introduction

When a command is chosen that launches the execution or the syntax checking of a macro script, the contents of all modified macro scripts must be saved; the exact behavior is determined by the **Before debugging** option (see [Section 3.5.1 Macro script display and functions](#)).

Menu items on the *Debug* menu allow adding breakpoints to scripts and enabling/disabling them. Breakpoints can be added to a script to stop the execution at that line. Breakpoints allows running the script in a controlled manner (i.e., one group of commands at a time), and allows verifying the current value of variables, if the **Information tooltips** option is enabled. When a script being executed stops at a breakpoint, that line is referred to as the *current execution line*; if the execution is

continued, it continues from the current execution line. If a breakpoint is disabled, it has no effect on script execution.

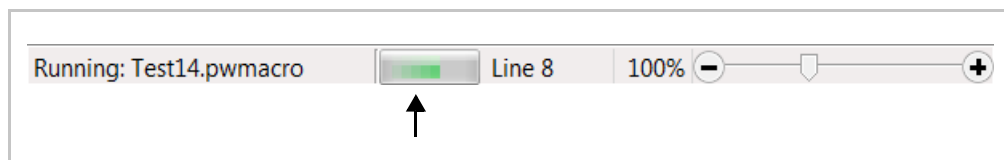
3.3.4.2 Menu items

The *Debug* menu offers the following items:

Menu item + shortcut	Description	Toolbar button
<i>Check Syntax</i> <i>F7</i>	Checks the contents of the active macro script for syntax errors. Available when there is at least one script window.	
<i>Run</i> <i>F5</i>	Executes the contents of the active macro script. Available when there is at least one script window. Once the command is invoked, the script is executed from the current line of execution (by default the first line) until a breakpoint is encountered, in which case the execution stops at that line without executing it, or until the end of the script is reached.	
<i>Run File</i>	Displays a file browser to select a macro script. Specify the file name and location, and press the Open button. The macro script is executed.	
<i>Run to Cursor</i> <i>Ctrl+F10</i>	Executes the active macro script from the current execution line up to the line containing the cursor, or until the end of the script is reached. If execution stops at the cursor position, that line is not executed, and it becomes the new current execution line. Available when there is at least one script window.	
<i>Skip to Cursor</i> <i>Ctrl+Shift+F10</i>	Skips all the commands from the current execution line up to the line containing the cursor, or until the end of the script. Available when there is at least one script window.	
<i>Step</i> <i>F10</i>	Executes the current line of the active macro script, which is useful for macro script diagnostics. Press the ESC key to interrupt the mode. If a MACRO EXEC command is encountered that calls another macro script, it is executed normally (i.e., the associated macro script is not opened and executed one line at a time).	

Menu item + shortcut	Description	Toolbar button
<i>Step Into</i> <i>F11</i>	Very similar to the <i>Step</i> menu item. The only difference is that if a MACRO EXEC command is encountered that calls another macro script, the associated macro script is opened and executed one line at a time, starting at the beginning of the script.	
<i>Stop</i> <i>Esc</i>	Cancels the execution of all macro scripts. Available when a script is being executed.	
<i>Insert/Remove Breakpoint</i> <i>F9</i>	Inserts or removes a breakpoint at the current line. Breakpoints can also be added or removed by clicking before a line (or before its line number if it is displayed). Available when there is at least one script window.	
<i>Remove All Breakpoints</i> <i>Ctrl+Shift+F9</i>	Removes all breakpoints. Available when there is at least one script window.	
<i>Enable/Disable Breakpoint</i> <i>Ctrl+F9</i>	Enables or disables the breakpoint at the current line. Breakpoints can also be enabled or disabled by clicking them with the Shift key held down. Available when there is at least one script window.	
<i>Enable All Breakpoints</i>	Enables all breakpoints. Enabled breakpoints stop script execution at that line. The line is not executed, and it becomes the new execution line if the execution of the script is continued. Available when there is at least one script window.	
<i>Disable All Breakpoints</i>	Disables all breakpoints. Disabled breakpoints do not effect script execution. Available when there is at least one script window.	

When a macro is running, a continuous progress bar is displayed on the Macro Script Editor's status bar:



3.3.4.3 Other ways to run macro scripts

3.3.4.3.1 The MACRO EXEC command

It is possible to run a macro script on disk from the Command History pane or from within a macro script. These methods use the MACRO EXEC command and are described in the chapters that follow.

3.3.4.3.2 Automatically launching macro scripts after operations

A simple folder-based mechanism allows creating macro scripts with specific names that are executed on performing specific actions in the application.

The actions are the following: opening and closing a workspace, and ending a scan pass. Each macro is presented in the subsections that follow.

Macros linked to opening and closing a workspace

On opening a workspace, the *Autorun.WorkspaceOpen* script is run if it is found in the *macro\polyWorks* subfolder of the workspace's *_Files* folder.

This may be useful to automatically launch certain PolyWorks Metrology Suite modules or open specific PolyWorks projects. Make sure to load only one workspace at a time to prevent conflicts.

Similarly, on closing a workspace, the *Autorun.WorkspaceClose* script is run if it is found in the same locations. This may be useful, for example, to restore a specific configuration file.

A macro linked to ending a scan pass

When the **Stop Scan** button in a scanning plug-in is pressed, PolyWorks|Inspector automatically looks for a macro called *EndOfPass.mac* in the standard macro locations, in the following order:

- The current visual layout of the instance of PolyWorks|Inspector where the scanning is being performed.
- The workspace associated with the instance of PolyWorks|Inspector where the scanning is being performed.
- The user configuration folder.
- The system configuration folder.

If the macro is found, it is run. This enables additional, automatic processing of scan pass data.

Note that the processing done by this macro is performed after other processing specified in the plug-in dialog box (i.e., first scan line filtering and then other specified processing, such as extracting point normals).

3.3.5 Tools menu

The *Tools* menu offers the following items:

Menu item + shortcut	Description	Toolbar button
<i>Record Commands > Start</i>	Starts the recording of commands invoked by the user. Available unless script recording has begun.	
<i>Record Commands > Stop</i>	Stops the recording of invoked commands. Available once script recording has begun.	
<i>Paste Recorded Commands</i>	Pastes the recorded commands into the active macro script. Available when commands have been recorded.	
<i>Options</i>	Opens the Macro Script Editor Options dialog box, which allows configuring the editor. For complete information, see Section 3.5.1 Macro script display and functions .	

3.3.5.1 Options

To display the Macro Script Editor Options dialog box, choose the *Tools > Options* menu item or press the toolbar button shown to the right. The dialog box shown in [Figure 3.5](#) is displayed. Options are organized by pages. Once changes are made in a page, press the **Apply** button to apply the changes, or the **OK** button to apply the changes and dismiss the dialog box.



When changes are made using the Macro Script Editor of a specific module, the changes must be saved explicitly using the *Tools > Save User Configuration* menu item to be available next time an instance of the module is opened. All the Macro Script Editors share the same configuration file. When options are saved in one module, using the *Save User Configuration* menu item, the configuration file is updated with the changed values for those options. When opening a Macro Script Editor in an instance of the same or a different module, or the Workspace Manager, the editor uses the options in the updated file. Note that changes made to the options of the Macro Script Editor of the Workspace Manager cannot be saved explicitly; rather, they are automatically saved when that application is closed.

3.3.5.1.1 General options

The **General** page of the Macro Script Editor Options dialog box, shown in [Figure 3.5](#), offers the following general items:

Before debugging

A group label that allows specifying the behavior to apply to all unsaved macro scripts when an operation that comprises debugging is launched, such as the *Check Syntax*, *Run*, or *Step* menu items. Two options are offered:

Save all modified macro scripts

An option button that specifies saving all modified macro scripts without any prompting.

Prompt to save all modified macro scripts

An option button, selected by default, that specifies saving all modified macro scripts with prior prompting.

Word wrap

A check box that enables the word wrap function, which is a visual effect. A line that is longer than the width of the script window is continued by wrapping. Note that it is still considered to be the same line (i.e., has the same line number, and is all highlighted in yellow). By default, the check box is selected.

The **Indentation** section offers the following items

Smart Indent

A check box that enables the use of indenting logical blocks, such as IF-ENDIF, when the ENTER key is pressed. By default, the check box is selected. The following item is also offered to configure the feature:

Number of spaces

A list box that allows specifying the number of spaces to use for the smart indent feature. Choose from: **2**, **4**, **6**, and **8**. The default value is 4.

The **Display** section offers several features that control the display in the script window. Many items are illustrated in [Figure 3.9](#), while other items are illustrated in the text that follows, adjacent the item description. The following items are offered:

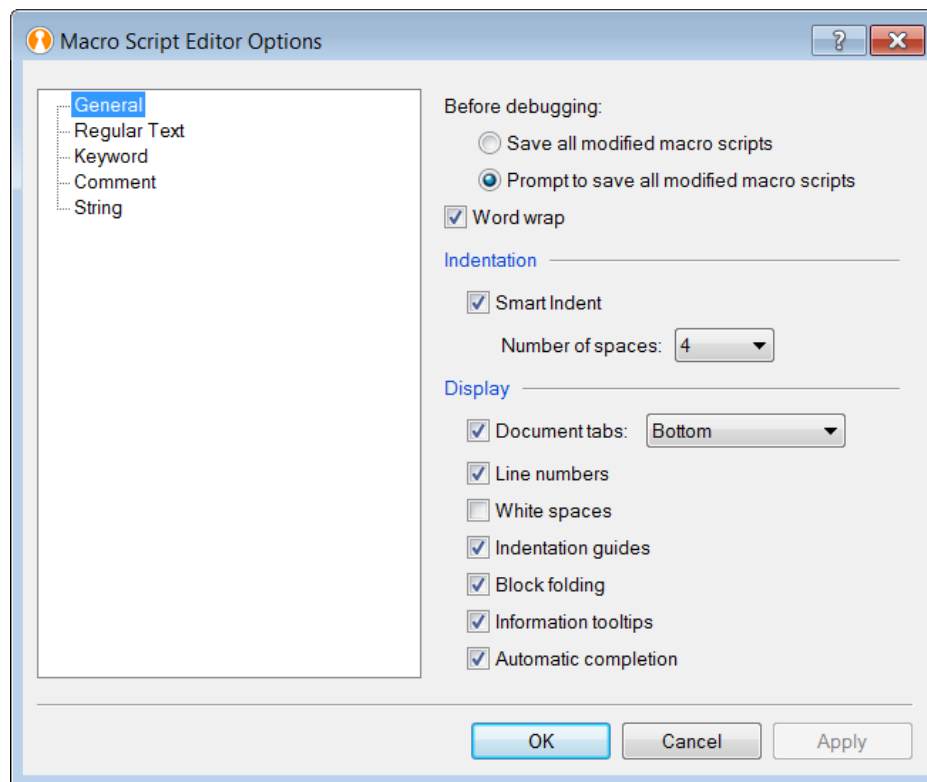
Document tabs

A check box that enables using document tabs which display the name of the open macro scripts. When selected, it makes available an adjacent list box that allows specifying where to position document tabs. Choose from: **Top**, **Bottom**, **Left**, and **Right**. The default value is Bottom.

Line numbers

A check box that enables displaying line numbers in the left margin. By default, the check box is selected.

Figure 3.5 The Macro Script Editor Options dialog box.



White spaces

A check box that enables displaying spaces using a visual representation. By default, the check box is cleared.

Indentation guides

A check box that enables displaying indentation guides. By default, the check box is selected.

Block folding

A check box that enables displaying block folding. Blocks are consecutive commands delimited by two commands, one that begins the block (e.g., IF), and one that ends the block (e.g., ENDIF). By default, the check box is selected. See the example to the right.

```

16 IF $Name == "Bob"
17     MACRO ECHO ("Cool")
18 ENDIF

```

Information tooltips

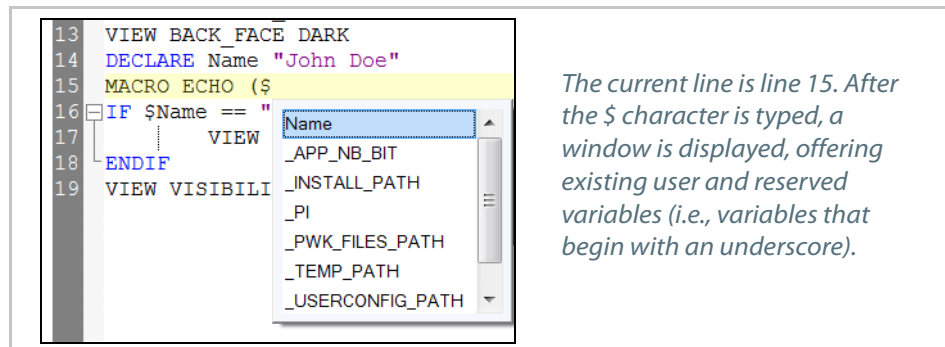
A check box that enables displaying information tooltips for elements of a command by hovering the pointer over the desired element:

- When the pointer is hovered over a command, a command help window is displayed.
- When the pointer is hovered over a variable that has been assigned a value, a window is displayed showing the value.

By default, the check box is selected.

Automatic completion

A check box that enables an automatic completion feature. This feature proposes valid commands based on the first characters typed by the user, or the names of existing or reserved variables once the **\$** and first characters have been typed. By default, the check box is selected.



To transfer the changes to the application, press the **Apply** button.

3.3.5.1.2 Text display options

The **Regular Text**, **Keyword**, **Comment**, and **String** pages of the Macro Script Editor Options dialog box all offer the same items to configure the appearance of those individual text elements. The **Regular Text** page is shown in [Figure 3.6](#). See [Figure 3.7](#) for an illustration of each text element.

The items are explained in the text that follows, and are found on each individual page.

The **Syntax highlighting** section offers the following items:

Font

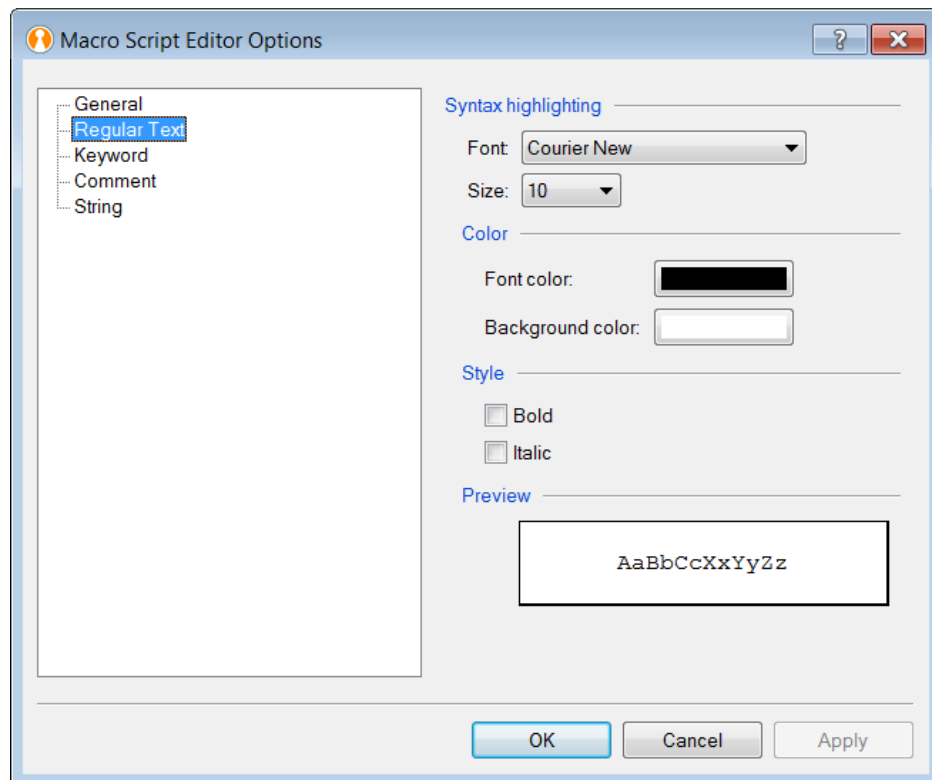
A list box that allows specifying a font for the text. The default value is Courier New.

Size

A list box that allows specifying a size for the text. The default value is 10.

The **Color** subsection offers the following items:

Figure 3.6 The Regular Text page of the Macro Script Editor Options dialog box.



Font color

A color box that allows specifying the color of the font. To edit the color, click the color box and click a new color in the list that is displayed. The default values are black (for regular text), blue (for keywords), green (for comments), and magenta (for string values).

Background color

A color box that allows specifying the color of the background. To edit the color, click the color box and click a new color in the list that is displayed. By default, the background color is white.

The **Style** subsection offers the following items:

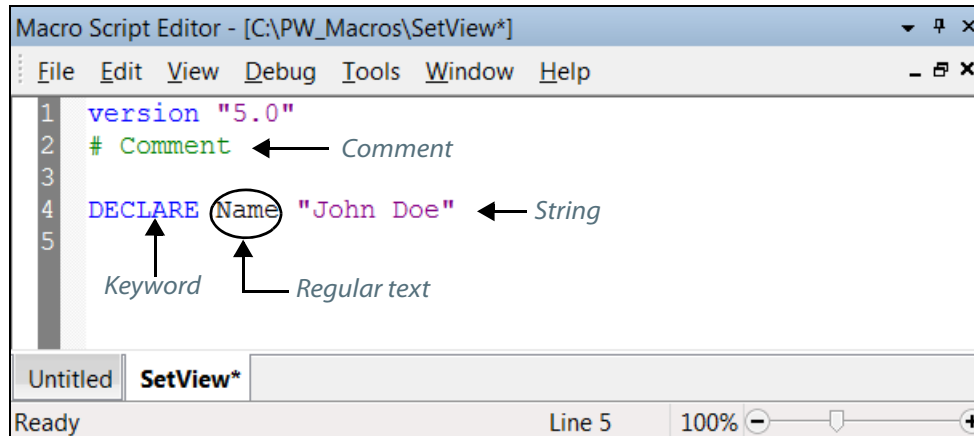
Bold

A check box that enables using the bold style component. By default, the check box is cleared.

Italic

A check box that enables using the italic style component. By default, the check box is cleared.

Figure 3.7 The four text elements for which the display can be configured using the corresponding pages of the Macro Script Editor Options dialog box.



The **Preview** subsection offers a text area that displays sample text configured by the specifications made on the page. The text is updated for each new specification.

3.3.6 Window menu

The *Window* menu offers items that allow organizing multiple script windows:

Menu item + shortcut	Description	Toolbar button
<i>Cascade</i>	Arrange script windows so they overlap.	
<i>Tile Horizontally</i>	Arrange script windows as nonoverlapping tiles organized horizontally.	
<i>Tile Vertically</i>	Arrange script windows as nonoverlapping tiles organized vertically.	

The menu also lists the current script windows. The one preceded by a check mark is the active script window. To make another script the active script, choose it in the list.

3.3.7 Help menu

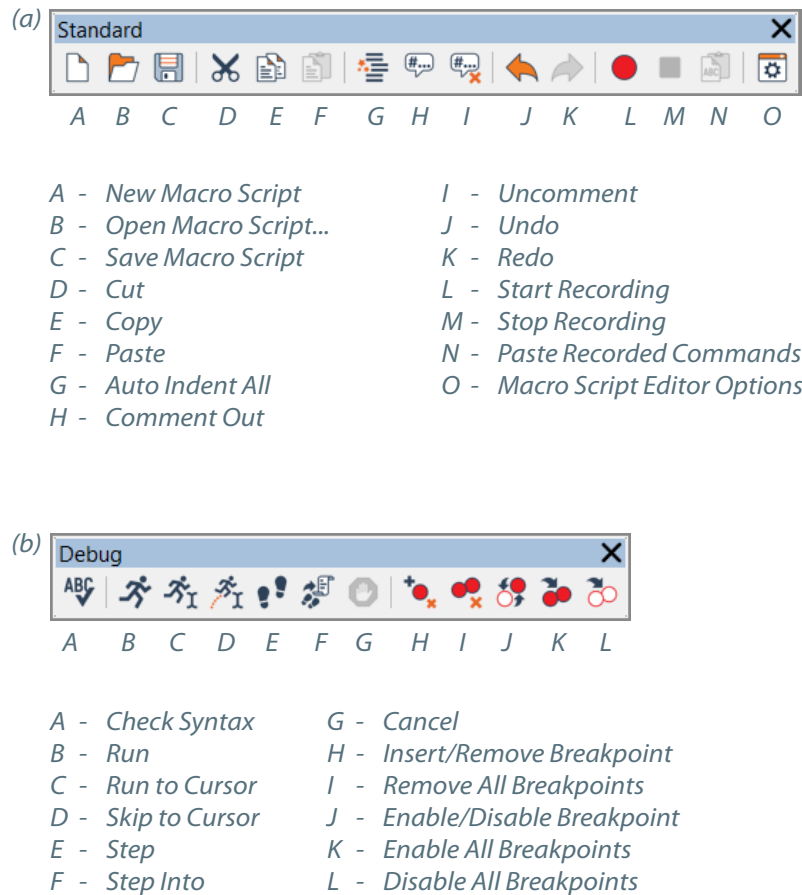
The *Help* menu offers the following items:

Menu item + shortcut	Description	Toolbar button
<i>Commands</i>	Opens your current Web browser with HTML help on available commands for the application and command syntax in general. For more information, see Section 2.4.2 Command Reference Guide .	
<i>Macro Script Reference Guide</i>	Opens this document in PDF format.	

3.4 Toolbars

The Macro Script Editor features the Standard and Debug toolbars. The Standard toolbar offers shortcuts for items located on the *File*, *Edit*, and *Tools* menus. The Debug toolbar offers shortcuts for items located on the *Debug* menu. Both toolbars are shown in [Figure 3.8](#) with the tooltips of the toolbar buttons.

Figure 3.8 In (a), the Standard toolbar and in (b) the Debug toolbar. The tooltip associated with each toolbar button is also presented.



3.5 Macro script window features

Several macro script windows can be open at a time. The one that contains the cursor is referred to as the *active* macro script window. Script windows offer functions and features that provide helpful information and that make it quick and easy to construct complete commands.

3.5.1 Macro script display and functions

An example of a script is shown in [Figure 3.9](#). It illustrates the following features:

- Several macro scripts can be open at a time. The name of each one can be displayed in a document tab.
- Numbered lines.
- Block folding for logical structures which allows closing/opening IF-ENDIF and DO WHILE-END WHILE blocks of commands.
- Indentation guides to identify commands located within logical structures.
- Auto-completion of commands and variable names to accelerate script creation.
- Breakpoints used for script testing and execution.

Many of them are options and can be activated or deactivated. The editor's options are described in [Section 3.5.1 Macro script display and functions](#).

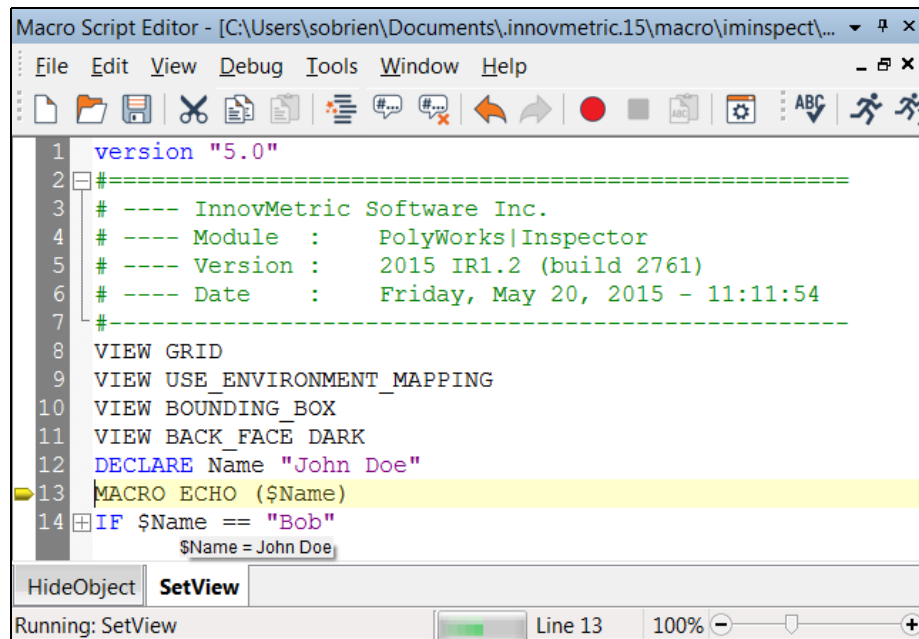
3.5.2 Help when writing commands

When working in a macro script window, two features are available to help users write commands quickly and correctly: Command Completion and Command Help.

3.5.2.1 Command Completion feature

Command completion can be invoked at any time by pressing the TAB key, either in a macro script window or in the command-line area of the Command History pane. Whenever the TAB key is pressed, the Macro Script Editor will try to resolve the keyword or sentence being typed. If a single solution is found, the keyword is completed automatically; if several solutions are possible, they will be listed in a window and one can be selected using the mouse or the arrow keys. Whenever an item is selected in the list, the Macro Script Editor will display *Intermediate command word* if the proposed

Figure 3.9 The Macro Script Editor pane. Two macro scripts are open in separate script windows. *SetView* is the active script. It demonstrates several of the editor's features. *SetView* is being executed in the step-by-step mode (line 13 will be executed next). Line 9 has an enabled breakpoint, and line 11 has a disabled breakpoint. The cursor is placed on the variable *\$Name* on line 13, and the variable's current content is displayed. The IF block encompassing lines 14-16 is folded. The name of the macro script being executed, and the current execution line number, are displayed on the editor's status bar.



solution does not complete a command, or it will display command help if its selection completes a valid command, as shown in [Figure 3.10](#).

When invoked, the *Command Completion* feature lists the available command keywords.

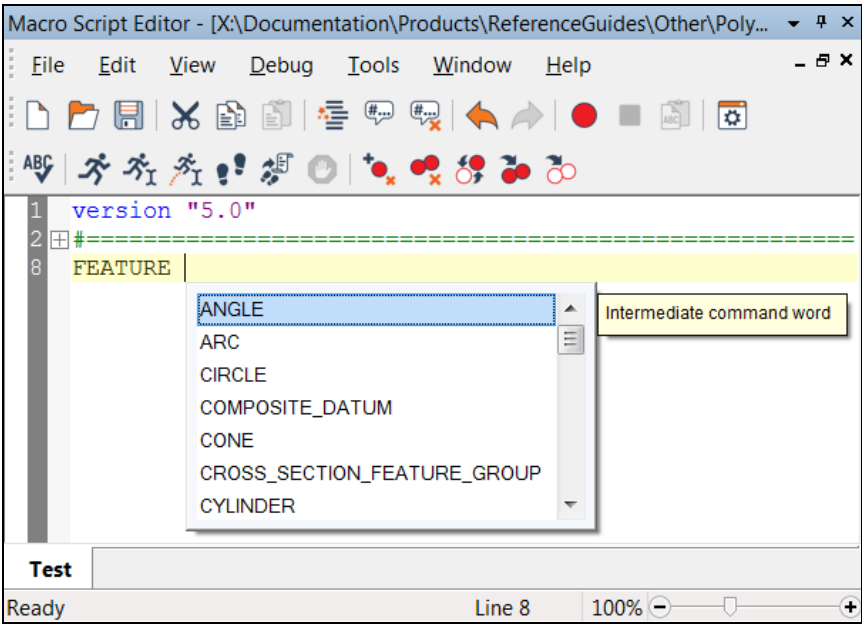
To select a keyword and dismiss the window, double-click the desired item in the list, or use the arrow keys to navigate in the list, and press ENTER. To dismiss the window without choosing a keyword, press the ESC key or click anywhere outside the list.

If a command is complete, pressing the TAB key will display command help that displays a general description of the command and its arguments.

The behavior described previously can be made automatic (i.e., no need to use the TAB key) by selecting the **Information tooltips** and the **Automatic completion** options on the **General** page of the Macro Script Editor Options dialog box. For more information, see [Section 3.3.5.1.1 General options](#).

Figure 3.10 Command completion and command help are available when writing macro scripts.

(a) The TAB key was pressed after typing the FEATURE command. A window is displayed that contains all the other commands that can follow the FEATURE command. In the window, ANGLE is highlighted – the text *Intermediate command word* is displayed to indicate that it does not form a complete command. Since the command is not complete, help information cannot be displayed.



(b) When MAKE COAXIAL is selected, a valid command can be formed: FEATURE MAKE COAXIAL. As a result, a help window is displayed for that command.

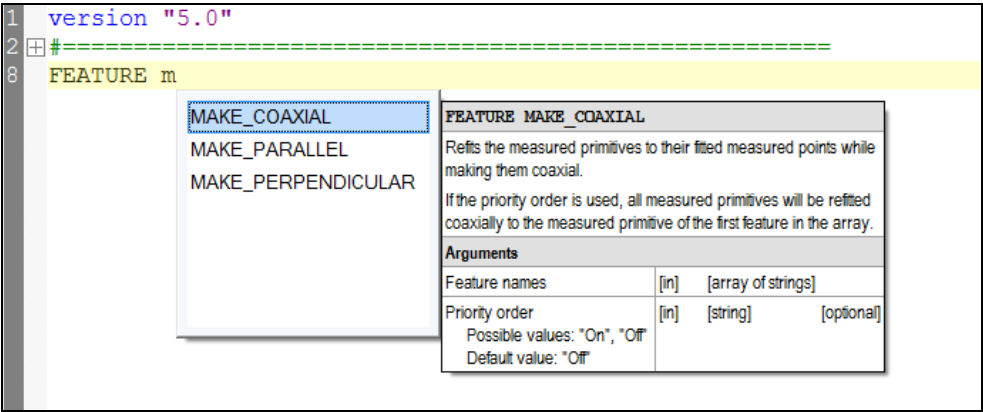
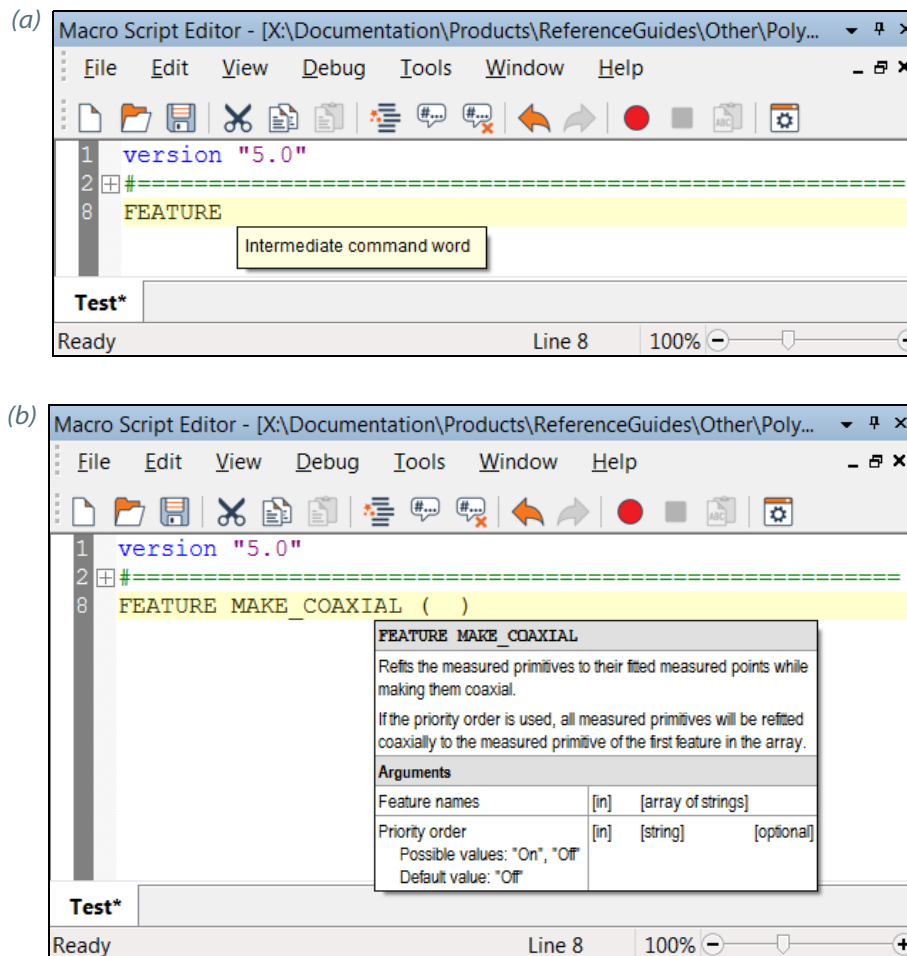


Figure 3.11 Clicking over a command and pressing the F1 key displays the Command Help window. If information tooltips are enabled, the same information appears when the pointer is placed over the command.



3.5.2.2 Command Help feature

Command help can be obtained by clicking within a keyword and then pressing the F1 key, either in the macro script window or in the command-line area of the Command History pane. If the keyword is an intermediate keyword, *Intermediate command word* is displayed; if it completes a command along with the preceding keywords, the command help is displayed instead. See [Figure 3.11](#) for an example.

3.6 Shortcut keys

The Macro Script Editor offers shortcut keys for the following actions in a script window:

Action	Shortcut key	
	Modifier	Key
Cancel		Esc
Character left		Left arrow
Character left extend selection	Shift	Left arrow
Character left rectangular extend selection		Left arrow
Character right	Shift	Right arrow
Character right extend selection	Ctrl	Right arrow
Character right rectangular extend selection	Alt Shift	Right arrow
Copy	Ctrl	Insert
Copy	Ctrl	'C'
Cut	Ctrl	'X'
Cut selection	Shift	Delete
Delete char left		Backspace
Delete char left	Shift	Backspace
Delete char right		Delete
Delete line left	Ctrl Shift	Backspace
Delete line right	Ctrl Shift	Delete
Delete word left	Ctrl	Backspace
Delete word right	Ctrl	Delete
Duplicate selection	Ctrl	'D'
End of display line	Alt	End
End of line		End
End of line extend selection	Shift	End
End of script	Ctrl	End
End of script extend selection	Ctrl Shift	End
End of script extend selection	Alt Shift	End
Indent		Tab
Line copy	Ctrl Shift	'T'

Action	Shortcut key	
	Modifier	Key
Line delete	Ctrl Shift	'L'
Line down		Down arrow
Line down extend rectangular selection	Alt Shift	Down arrow
Line down extend selection	Shift	Down arrow
Line scroll down	Ctrl	Down arrow
Line scroll up	Ctrl	Up arrow
Line up		Up arrow
Line up extend rectangular selection	Alt Shift	Up arrow
Line up extend selection	Shift	Up arrow
New line		Enter
New line	Shift	Enter
Page down		Page Down
Page down extend rectangular selection	Alt Shift	Page Down
Page down extend selection	Shift	Page Down
Page up extend rectangular selection	Alt Shift	Page Up
Page up extend selection	Shift	Page Up
Page up, where a page is defined as the number of lines that can be displayed in the active script window.		Page Up
Paste	Shift	Insert
Paste	Ctrl	'V'
Redo	Ctrl	'Y'
Select all	Ctrl	'A'
Selection lowercase	Ctrl	F3
Selection uppercase	Ctrl Shift	F3
Start of display line	Alt	Home
Start of line		Home
Start of line extend selection	Shift	Home
Start of script	Ctrl	Home
Start of script extend selection	Ctrl Shift	Home

Action	Shortcut key	
	Modifier	Key
Start of script extend selection	Alt Shift	Home
Toggle overwrite		Insert
Undent	Shift	Tab
Undo	Alt	Backspace
Undo	Ctrl	'Z'
Word left	Ctrl	Left arrow
Word left extend selection	Alt Shift	Left arrow
Word part left	Ctrl	'/'
Word part left extend selection	Ctrl Shift	'/'
Word part right	Ctrl	'\'
Word part right extend selection	Ctrl Shift	'\'
Word right	Alt Shift	Right arrow
Word right extend selection	Alt Shift	Right arrow
Zoom 100%	Ctrl	'/'
Zoom in	Ctrl	'+'
Zoom out	Ctrl	'-'

Command History Pane



4

This chapter introduces the Command History pane, which is a complement to the Macro Script Editor pane.

4.1 Introduction

The Command History pane is an important companion tool for users who write scripts.

The Command History pane displays commands given by the user in the application, which can be directly copied to a macro script window.

Application and utility commands can be given in the pane to try them out or to see if they contain syntax errors, which are displayed in the pane.

The pane also displays information related to the execution of macro scripts in the Macro Script Editor, such as error messages or the output of certain commands, including MACRO ECHO.

4.2 Elements of the Command History pane

To display the Command History pane, click the title bar of any pane and on the list of panes that is displayed click *Command History*, or choose the *Tools > Commands > Command History* menu item on the module's main menu.

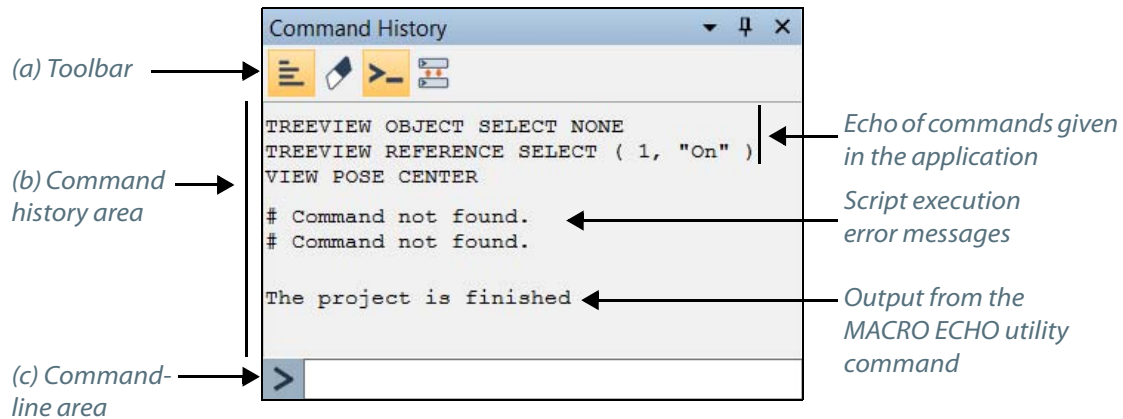
The Command History pane is shown in [Figure 4.1](#). It features a toolbar, a command history area, and a command-line area. These elements are presented in the subsections that follow.

4.2.1 Toolbar

The toolbar offers operations to configure the behavior and the layout of the Command History pane:

Button	Description
	Toggle Command History Enables or disables the display of commands executed in the application (e.g., by way of a menu item, a button).
	Clear Command History Removes the contents from the command history area.
	Toggle View Command Line Enables or disables the display of the command-line area.
	Toggle Command Line Position (Top or Bottom) Switches the position of the command-line area from the top of the command history area to the bottom of the command history area, or vice versa. This button is only available when the command-line area is displayed.

Figure 4.1 The Command History pane. It features (a) a toolbar, (b) a command history area, shown here with content, and (c) a command-line area. The display of application commands in the command history area can be toggled and the command-line area can be hidden.



4.2.2 Command history area

The command history area can display application commands that correspond to actions performed by the user in the application, as well as content related to commands given explicitly by the user either on the command-line area or by executing a macro script.

Application commands

The command history area can display application commands. All the commands given by the user by way of the graphic user interface that are translated into application commands, as shown in [Figure 4.1](#). These commands include those chosen on main or shortcut menus, selections in the tree view or in the 3D scene, and actions performed by pressing toolbar buttons.

The command history area can also display application error messages that are normally displayed if an action was not done correctly. For example, if a plane was not selected for an action requiring a plane, an error message is displayed in the Command History pane.

Application commands can be copied and pasted into a macro script.

Explicit user-given commands

The command history area can display information related to commands given in the command-line or by macro script execution:

- The result of certain commands (e.g., MACRO ECHO), as shown in [Figure 4.1](#).
- Macro script syntax errors found while testing or running a macro script, as shown in [Figure 4.2](#).
- When a macro script is executed its name is displayed. When the execution is over, a corresponding message is displayed.

4.2.3 Command-line area

The command-line area allows entering individual application commands or utility commands and executing them on pressing the ENTER key. This is an easy way to test commands and see syntax error messages, if any, in the pane. Correct commands may then be pasted in a macro script window.

The command-line area supports the command completion and help features, presented in [Section 3.5.2 Help when writing commands](#). Previous commands can be retrieved by navigating using the UP ARROW and the DOWN ARROW keys.

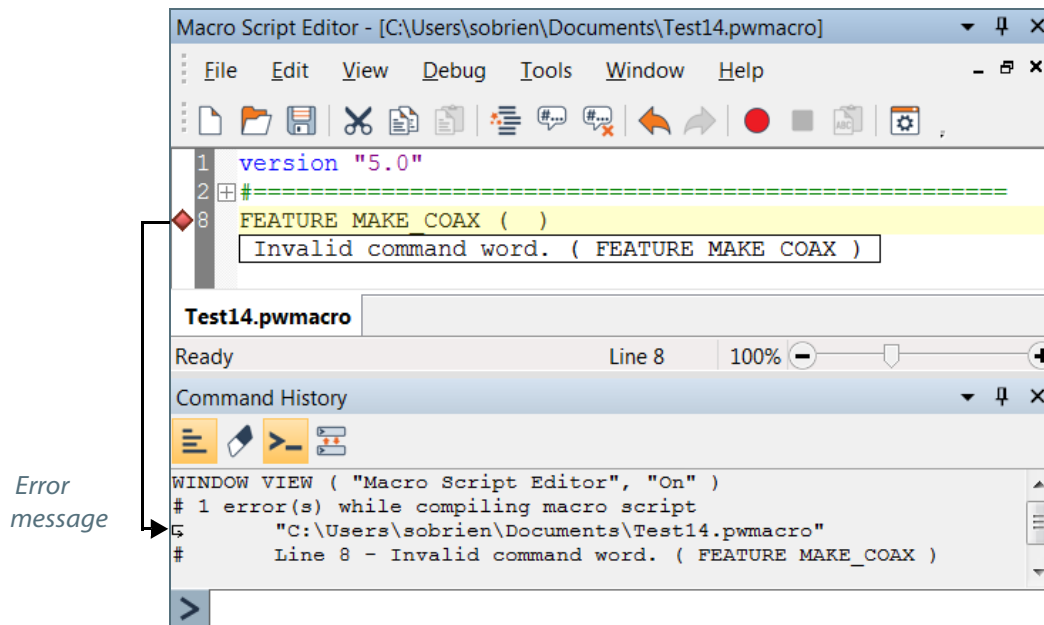
Some commands invoked in this way may differ from the same commands invoked within a macro script. For example, the command

FILE OPEN PROJECT

Displays a project file browser as if it were invoked by way of the application's *File* menu, whereas the same command invoked in a macro script must specify the location and name of the project to open:

FILE OPEN_PROJECT_IN_PWK ("Workspace_name", "Project_name")

Figure 4.2 An error displayed in the Command History pane as a result of the Check Syntax operation.



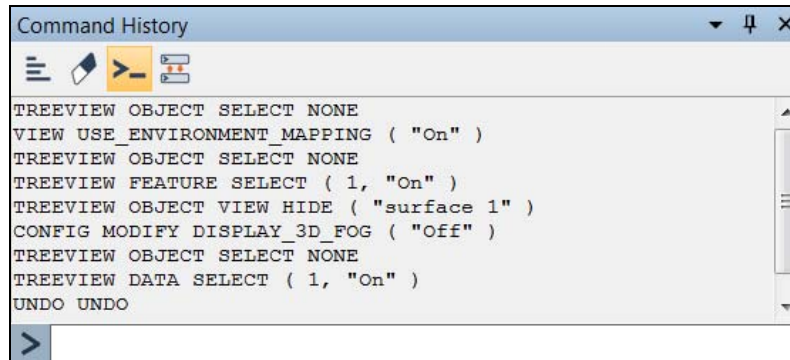
The difference stems from the fact that commands in a macro script, as a general rule, should not display windows that can block the execution of a script.

4.3 Using the pane to find application commands

If you are not sure of the command to use in a given situation, remember that the Command History pane shows the command that corresponds to each user action made through the main or shortcut menus, toolbars, or when affecting the visibility or selection status of objects in the Tree View pane using the mouse.

Finding the command may be as easy as giving the command manually and copying the command that appears in the Command History pane to the active script. Or, the command can be used to initiate a search in the Command Reference offered by the Macro Script Editor's *Help* menu. For an example of the Command History pane containing given commands, see [Figure 4.3](#).

Figure 4.3 The Command History pane echoes the commands given by the user in the application. It is a useful learning tool for writers of macro scripts.



Creating a Basic Macro Script



5

This chapter describes how to record, edit, and run a macro script using the Macro Script Editor. The Macro Script Editor offers an environment that allows testing macro scripts and finding the lines that contain errors.

5.1 Introduction

This chapter introduces the steps required to create and run a macro script, and then explains how to edit the macro script. Remember to always run new and untested or undebugged scripts on test projects and not on real projects.

The approach taken here is to record commands while you work with the menus and toolbars of your application. Then, the recording will be ended, and the recorded commands pasted into a macro script in the Macro Script Editor pane. This may be a first contact with application commands; their names usually clearly identify the action they perform. Once the macro script is saved, it can be played to redo the same steps automatically.

The chapters that follow explain how to use the Macro Script Command Language and utility commands to go beyond the simple recording of user actions and make more powerful macro scripts through the use of simple programming structures.

5.2 The task to automate

The context is the following: The user has a project with an aligned Data object, and has created several features that are unmeasured, which are hidden.

The user often has to do the following tasks:

- Select all the features under the Features branch.
- Make them visible.
- Extract their measured component.
- Generate a report.
- Open the report editor.
- Save the inspection project.

The sections that follow will explain how to create a single script that will execute the above tasks.

5.3 The steps to create a macro script

To record commands given via the application and replay them in a macro script, follow this general procedure:

1. Prepare to record user actions.

Open the desired inspection project and display both the Command History pane and the Macro Script Editor pane.

2. Record the application commands.

- 2.1 Activate the recording mode before starting the task.

- 2.2 Carry out the specific task in the application using menus and toolbar buttons, and by manipulating objects in the tree view (e.g., selection, visibility).

Each action produces an application command in the Command history pane, which is recorded.

- 2.3 Stop the recording of application commands at the end of the task.

3. Add the commands to a macro script.

- 3.1 In the Macro Script Editor, create a new, empty macro script.

- 3.2 Paste the recording in a script window.

- 3.3 Save the macro script – file names do not require a file extension, but the default extension, .pwmacro, is offered.

4. Check the macro script for syntax errors.

This step is important when commands have been modified or new commands have been added to the script.

5. Run the macro script in either the normal mode or in the step-by-step mode. Observe the effects on the inspection project.

The step-by-step mode is useful for command diagnostics as it executes only one command at a time. For more information, see [Section 3.3.4 Debug menu](#).

6. Save the script for execution at a later time.

These steps in this general workflow are covered in detail in the sections that follow.

5.4 Creating a new macro script

To create a macro script, choose the *File > New* menu item to open a new macro script window.

The script version number is automatically inserted at the beginning of the script (e.g., VERSION "6.0"). This line must always appear before any command in the script.

5.5 Recording actions and pasting them to the macro script

Recording a task and pasting the recorded commands in a script window can be done from the module's *Tools > Commands* submenu, or from the *Debug > Record Commands* submenu of the Macro Script Editor.

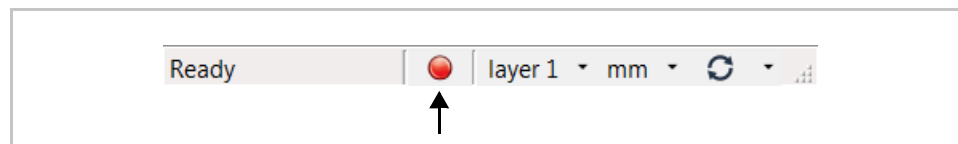
The procedures that follow use items on the menus offered by the Macro Script Editor.

5.5.1 Recording the task

Recording a task is performed as follows:

1. Start the recording of a task by choosing the *Tools > Record Commands > Start* menu item, or click the corresponding button on the Standard toolbar (shown to the right).

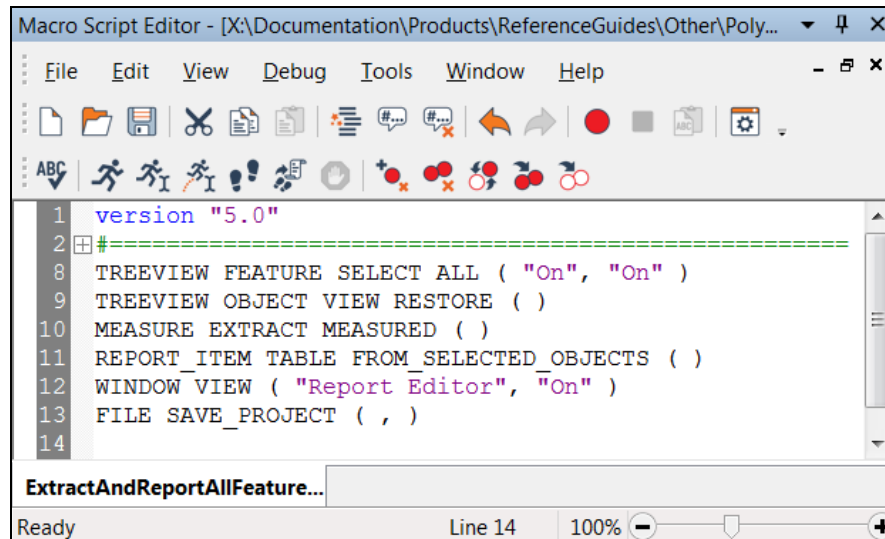
A blinking red button appears to the right of the module's status bar to indicate that a macro script is being recorded.



Note that clicking the red button displays the Macro Script Editor pane if it is currently hidden.

2. Perform the specific task until it is completed. In recording mode, every recordable action that is performed in the PolyWorks Metrology Suite application is recorded in the form of an application command.
3. Stop the recording by choosing the *Tools > Record Commands > Stop* menu item, or clicking the corresponding button on the Standard toolbar (shown to the right). The revolving PolyWorks Metrology Suite logo disappears from the status bar.

Figure 5.1 An example of recorded application commands that correspond to the simple task defined earlier in the chapter. Application commands are loosely based on the module's top level menus (e.g., FILE SAVE PROJECT). Note that the commands on lines 10 and 11 could apply to specific, named objects, but by default they use the list of objects selected by the command at line 8, which selects all feature objects.



5.5.2 Pasting recorded commands

To paste recorded command into the active macro script, proceed as follows:

1. Click at the desired location in the macro script open in the Macro Script Editor.
2. Choose the *Tools > Paste Recorded Commands* menu item to insert the recording, or click the corresponding button on the Standard toolbar (shown to the right).



For an example of the application commands that correspond to the simple task that was described in the introduction, see [Figure 5.1](#). The actual commands follow:

```

TREEVIEW FEATURE SELECT ALL ("On","On")
TREEVIEW OBJECT VIEW RESTORE ( )
MEASURE EXTRACT MEASURED ( )
REPORT_ITEM TABLE FROM_SELECTED_OBJECTS ( )
WINDOW VIEW ("Report Editor","On")
FILE SAVE_PROJECT ( , )

```

5.6 Saving the macro script

To save a macro script file, choose the *File > Save* menu item, or click the corresponding button on the Standard toolbar (shown to the right). For more information, see [Section 3.3.1.2 Saving macro scripts](#).



5.7 Checking for syntax errors

Once macro script commands have been edited, or new commands have been added to a macro script, the macro script should be tested for syntax errors.

To check the syntax of the contents of the active macro script window, choose the *Debug > Check Syntax* menu item, or click the corresponding button on the Debug toolbar (shown to the right).



If errors are found, the execution of the active macro script window is disabled, and a red marker is displayed in the left margin of the line at which the first error was encountered, as shown in [Figure 5.2](#).

5.8 Running the macro script

There are several ways to run the macro script file in the active script window:

- To run the contents of the active macro script window using the step-by-step mode (i.e., execute one command at a time), choose the *Debug > Step* menu item for each command to execute, or click the corresponding button on the Debug toolbar (shown to the right). This is recommended for new or edited macro scripts.
- To run the contents of the active macro script window in normal mode, choose the *Debug > Run* menu item, or click the corresponding button on the Debug toolbar (shown to the right). This is recommended for debugged and tested macro scripts.

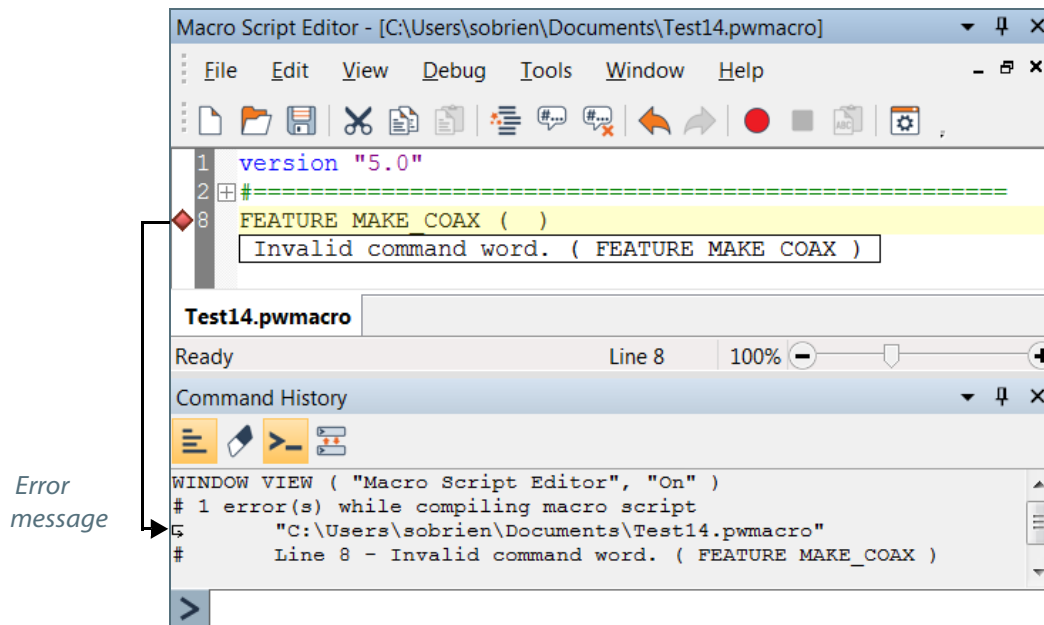


A running macro script can be cancelled at any time by pressing the ESC key when the Macro Script Editor has the input focus.

5.9 Editing the macro script

Editing a macro script involves correcting existing commands and adding additional commands.

Figure 5.2 An error displayed in the Command History pane as a result of the Check Syntax operation.



Error message

Two important tools that help to edit and add commands are *Command Completion* and *Command Help*; for more information, see [Section 3.5.2 Help when writing commands](#).

Always check the syntax before running a script.

A few general notions pertaining to macro scripts are presented below; they are presented in full in the chapter that follows. If you do not intend to add commands unassisted at this time, you may skip this section and continue to [Chapter 6 Macro Script Control Language](#).

5.9.1 Basic rules when writing macro scripts

A macro script consists of a Unicode file containing statements. The following rules apply to macro scripts:

- Macro scripts are case insensitive. The use of capital letters is to make the commands more readable.
- When a space is required, multiple spaces can be used.

- To add comments to a macro script, begin a line with the # character. Only spaces can be placed before the # character. All the characters that follow the # character are ignored.
- The following restriction applies to the first line of each macro script – it must begin with the VERSION keyword, followed by the version number of that particular macro script. For example:

```
VERSION "6.0"
```

5.10 Compatibility between versions

The current version is 6.0. The PolyWorks Metrology Suite can execute correctly macro scripts that begin with an earlier version number command, such as version "5.0", and that contain elements of that version of the Macro Script Control Language (MSCL) or earlier.

To incorporate certain elements of version 6.0 in macro scripts created using versions 1.0, 2.0, 3.0, 4.0, or 5.0, the first command must be replaced with the command Version "6.0".

Macro Script Control Language

6

This chapter explains how to turn simple, recorded scripts into robust scripts that execute real-world tasks. Efficient macro scripts have to be able to repeat commands, or not execute certain commands in a given situation, offer error handling mechanisms, and more. To this end, InnovMetric Software offers the *Macro Script Control Language (MSCL)*, a simple but powerful programming language that allows adding *control* commands to your macro scripts. Now in version 5, the MSCL allows building macro scripts that can save users time and control task execution.

This chapter is the complete reference for the MSCL. It presents the general syntax of each language element. The MSCL consists of keywords that can be combined with other language elements, such as values and mathematical expressions, as well as commands, to form executable *statements*. The basic unit of a macro script is the statement.

This chapter contains many examples. They are an important part of the explanations and contain pertinent comments. Look at them carefully. Many can be copied and pasted directly into an empty macro script, saved, and executed. Your ability to create macro scripts will grow with each new script that you realize.

6.1 Introduction

The Macro Script Control Language (MSCL) is a high-level macro (scripting) language that permits you to add control commands to a macro script. They can be added with any Unicode text editor. The resulting macro script must always be tested and executed in the appropriate module's Macro Script Editor tool. Note that IMAlign, PolyWorks|Modeler, and PolyWorks|Inspector all have application commands specific to their own module, and that are not used by the other modules. The Workspace Manager also has a Macro Script Editor, which lets you execute scripts that can launch processes in several modules.

The MSCL is explained mainly through its keywords. Each keyword is explained and demonstrated using short macro script examples. Concepts associated with high-level programming are touched on as a result. They include:

- Variables, and variable types, when explaining the DECLARE keyword.

Variables are containers used to store and retrieve information. They can be used to provide information to commands, such as an employee number (input variables), as well as to receive information from commands, such as a result computed by a command (output variables).

- Mathematical expressions, functions (e.g., COS()), and operators (e.g., +, -, /).
- Comparison expressions and operators (e.g., >, ==, <).
- Logical expressions and operators (e.g., AND, OR, NOT), which leads to the notion of conditions.
- Conditions, when discussing the IF-ENDIF and the WHILE-ENDWHILE structures.
- Loops and conditions, when discussing the WHILE-ENDWHILE structure.
- The flow of loop execution, when explaining the BREAK and CONTINUE keywords that belong to the WHILE-ENDWHILE structure.

The group of MACRO utility commands also offers functions that are associated with high-level programming:

- Modular programming, which consists in dividing a large task into many smaller ones, and writing a main script that calls, or makes use of, the smaller scripts. The smaller scripts are easier to test/debug, and often they are so specialized that they can be reused by other macro scripts. This is discussed when explaining the MACRO EXEC command (see [Section 8.2 Modular programming](#)).
- Error handling, which involves actions to take when something has not been performed properly (see [Section 8.3 Error handling](#)).

6.2 General information

This section presents rules used in writing macro scripts as well as information concerning the sample macro scripts provided in the chapter.

6.2.1 Rules used in writing macro scripts

A macro script consists of a Unicode file containing statements. The following rules apply to macro scripts:

- Version number

The following restriction applies to the first line of each macro script – it must begin with the VERSION keyword, followed by the version number of that particular macro script. For example:

```
VERSION "6.0"
```

Note that the PolyWorks Metrology Suite 2022 can execute correctly macro scripts that begin with the VERSION "6.0" command and that contain elements of MSCL version 6.0 or earlier. It can also execute correctly macro scripts that begin with an earlier version number command (e.g., VERSION "5.0") and that contain elements of the MSCL pertaining to that version or to an earlier version.

- Case insensitive

Macro scripts are case insensitive (e.g., MSCL commands, variable names, application commands, comments). Small and capital letters are considered to be identical (e.g., WHILE and while, vMyVar and vmyvar). Capital letters are often used to make the commands more readable.

Note that string variables may contain values with uppercase and lowercase letters.

Certain operating systems, such as Unix and Linux, differentiate between small and capital letters with regards to file names.

- Comments

To add comments to a macro script, begin a line with the # character. Only spaces can be placed before the # character. All the characters that follow the # character are ignored.

- White spaces

When a space is required, mostly to separate the words in a statement, multiple spaces can be used. Add spaces using the SPACEBAR, or add tabs using the CTRL + TAB keys.

- One statement per line

There can only be one statement per line. Note that the editor offers the **Word wrap** option which allows wrapping a long statement within a window's current width. This is a visual effect, and the statement is still considered to be on one line.

A statement can occupy more than one line by placing the \ character at the end of each consecutive line, except the last one. The exception to this rule is a comment line, as all the characters that follow the # character are ignored. To obtain consecutive comment lines, begin each one with the # character. There are no start of comment and end of comment characters that allow transforming to comments all the text between the characters.

6.2.2 Comments regarding sample scripts

MSCL keywords and ordinary commands can be written in lowercase or uppercase as macro scripts are case insensitive. Read the examples attentively, as they contain comments that add to the comprehension of the concept being explained.

Most of the application commands included in the macro script examples in this chapter belong to the PolyWorks|Inspector module. They are used to demonstrate concepts that apply equally well to scripting in the IMAlign and the PolyWorks|Modeler modules. A commented macro script for the IMAlign, the PolyWorks|Modeler, and the PolyWorks|Inspector modules are offered in [Chapter 9 Sample Macro Scripts](#), and one for the Workspace Manager can be found in [Section 9.5 A multiapplication script](#).

In most cases, you can copy a sample script directly into a macro script window, save it, and then execute it to see how it works. Note that for the examples that include PolyWorks|Inspector application commands, those scripts should be run in the Macro Script Editor of the PolyWorks|Inspector module.

6.3 MSCL commands

MSCL commands allow you to manage variables, control the execution of statements in a script, and compute numerical expressions. All MSCL commands begin with a keyword. Most, but not all, MSCL commands form executable statements on their own.

6.3.1 Command format

All MSCL commands have the same format:

KEYWORDS <arguments>

where <arguments> are values that may be used by certain keywords to form a statement. Arguments may be optional or required. If you omit required arguments, or if too many arguments are entered for a command, an error message will be displayed on running the macro script in the script window and the Command History pane.

The MSCL commands are described in the following sections. For each command, the following information is provided:

- The definition of the keyword.
- The related syntax which explains how to write a statement using the keyword.
- The list and description of its arguments.
- Helpful examples.

6.3.2 List of MSCL commands

The commands that make up the MSCL include:

Category	Command	Description
Manage variables	DECLARE	Creates variables; can also assign a value on creation.
	SET	Assigns a value to a variable.
	SIZE	Gets the size of an array variable.
	HAS VALUE	Indicates if a variable contains a value (available from version 6.0).
Control the execution of statements	IF ELSE ELSEIF ENDIF	Executes statements contained within the IF/ENDIF structure based on a run-time condition.
	WHILE BREAK CONTINUE ENDWHILE	Executes statements within the WHILE/ENDWHILE structure repeatedly (loop) until a condition is met. BREAK exits the structure immediately and CONTINUE ignores commands and restarts the next loop.
Mathematical expressions	EXPR EXPR_I	Calculates a mathematical expression. EXPR_I returns an integer result.

Category	Command	Description
Mathematical functions	ABS ACOS ASIN ATAN ATAN2 COS LOG10 POW SIN SQRT TAN	Standard mathematical functions used within the mathematical expressions listed above.
Other	STRING_DECIMAL	Converts a double value to a string value using a decimal notation. The number of digits after the decimal symbol must be specified. Allows displaying values in decimal notation instead of scientific notation.
	VERSION	Returns the version of the MSCL language.

6.4 Defining and using variables

Variables are system-managed containers to which values can be assigned and later retrieved for use in other statements.

6.4.1 Variable types

There are three variable types:

Type	Definition
Integer	An integer is a whole number (no decimal point) ranging from INT_MIN to INT_MAX.
Double	A real is a number that allows decimal points; values range from DOUBLE_MIN to DOUBLE_MAX.
String	A string variable is a list of characters (case sensitive). A string can be of any length.

The type of a variable is determined by the content that is stored in the variable. Since the content of the variable may change, the type of a variable may also change.

6.4.2 Creating variables

Use the DECLARE keyword to create variables. When a variable is declared, it is created, assigned a user-specified name, and assigned a user-specified or system default value.

The DECLARE keyword has the following syntax:

```
DECLARE <variable_name> <initial value>
```

where <variable_name> represents the name of the variable being declared. A variable name must respect the following conditions:

- Be composed of any combination of alphabetic (a to z, and A to Z), numeric (0 to 9), and special characters (_). The number of characters is not restricted.
- Begin with a nonnumeric character.
- Not be a MSCL keyword (for a list of the keywords, see [Section 6.3 MSCL commands](#)).

As a convention in this document, user variable names will always start with a small letter *v* (e.g., *vNumberCircles*). This makes it easy to identify variables. Note also that variable names are case insensitive (e.g., *vMyVar* and *vmyvar* are the same variable).

The following examples demonstrate valid and invalid variable names:

```
# declarations containing valid variable names
```

```
DECLARE vFileName
```

```
DECLARE vMax_number
```

```
# declarations containing invalid variable names
```

```
# - name contains an invalid character
```

```
DECLARE vMax_#
```

```
# - name begins with a digit
```

```
DECLARE 4circle
```

<initial value> is an optional element that permits you to assign an initial value to the variable on declaration:

```
# assigning an integer value
```

```
DECLARE vMax_number 4
```

```
# assigning a string of characters
```

```
DECLARE vFilename "c:\temp\example"
```

If no value is specified, the variable is assigned the value integer 0 (version 5.0 and earlier) or the variable is left empty (from version 6.0):

```
DECLARE vMax_number
```

6.4.3 Assigning values to existing variables

Use the SET keyword to assign values to declared variables. It has the following syntax:

```
SET <variable_name> <value>
```

where <variable_name> is replaced by the name of a variable previously declared using the DECLARE keyword, and <value> is replaced by the value to be assigned to the variable. The specified value will replace any previous value stored in the variable, and determines as well the type of the variable. In the following examples, the three types of values are assigned to variables:

```
# declaring the variables without assigning specific values
```

```
DECLARE vMyInteger
```

```
DECLARE vMyDouble
```

```
DECLARE vMyString
```

```
# these three variables have all been assigned the default value integer 0
```

```
# assigning an integer value to a variable
```

```
SET vMyInteger 4
```

```
SET vMyInteger -8
```

```
# assigning a double value to a variable
```

```
SET vMyDouble 1.4
```

```
SET vMyDouble -0.5
```

```
# assigning a double value using scientific notation
```

```
SET vMyDouble 1e2
```

```
SET vMyDouble .4e-2
```

```
# assigning a string value to a variable
```

```
# a string begins with the first " and ends with the following "
```

```
# the " characters that enclose the text string are not included in the string
```

```
# the " character is not allowed in a text string, but you can reproduce it using a
```

```
# reserved variable
```

```
SET vMyString "Hello!!!"
```

```
SET vMyString "8"
```

```
SET vMyString "8.4"
```

```
# assigning the result of a logical expression to a variable
```

```
# if the condition is True, 1 is returned, and if False, 0 is returned
```

```
# Here, 0 is returned (False)
```

```
DECLARE vReturnedValue 5 > 7
```

```
# Here, 1 is returned (True)
```

```
SET vReturnedValue (8 > 7) AND ("abc" == "abc")
```

6.4.4 Using the value stored in variables

To access the value stored in a variable, precede the variable name with the \$ character. The value can be used for different purposes, such as serving as a keyword argument, assigning a value to another variable, or participating in a conditional expression.

In the following example, the value stored in the vGrayLevel variable is used to specify values for the EDIT OBJECT COLOR command:

```
DECLARE vGrayLevel 100
# using the content of the variable as an argument
EDIT OBJECT COLOR ($vGrayLevel, $vGrayLevel, $vGrayLevel)
# the same value for each RGB component results in a level of gray
```

In the following example, the value stored in the vRed variable is used to assign a value to other variables:

```
# using a variable to assign a value to another variable in a statement
DECLARE vRed 100
DECLARE vGreen $vRed
DECLARE vBlue $vRed
EDIT OBJECT COLOR ($vRed, $vGreen, $vBlue)
```

It is also possible to insert the content of a variable in a text string. For example:

```
# inserting the content of a variable in a text string
DECLARE vNb 10
# MACRO ECHO displays information in the Command History pane
MACRO ECHO ("The number is $vNb")
# displays: The number is 10
```

For obvious reasons, the \$ character by itself is not allowed in a text string. To obtain that character, use the `_DOLLAR` reserved variable (see [Section 6.4.7 Reserved variables](#)).

From version 6.0:

If the content of a variable that was left empty is used to assign a value to a second variable, the second variable is left empty.

```
DECLARE vEmpty
DECLARE vCreatedFromEmpty $vEmpty
# vCreatedFromEmpty is left empty
DECLARE vSetFromEmpty
SET vSetFromEmpty $vEmpty
# vSetFromEmpty is still empty
```


If the content of a variable that was left empty is used in another scenario, a warning is generated in the command history.

```
DECLARE vEmpty
IF $vEmpty
    # WARNING: Uninitialized variable used: $vEmpty.
ENDIF
```

6.4.4.1 Using braces to delimit variable names

It is sometimes necessary to enclose a variable name in braces { } in order to separate its last character from the following characters in the string. In the next example, the text string within quotation marks is evaluated in two different ways, depending on whether or not the variable name, `vPartNB`, is enclosed by braces:

```
# declaring and assigning a value to a variable
DECLARE vPartNB 4
FILE OPEN_PROJECT_IN_PWK ( , "piece${vPartNB}Front")
# this command results in the desired evaluation of the text string as:
# "piece4Front"
```

However, without the { } characters, the variable name is interpreted as `vPartNBFront`:

```
FILE OPEN_PROJECT_IN_PWK ( , "piece$vPartNBFront")
```

As no variable of that name has been declared, running the script results in an invalid statement.

Variables declared at the beginning of a script are usually available throughout the script, but variables declared within a logical structure are only available within the structure. This is the notion of variable scope, explained in [Section 6.8 Controlling the scope of variables](#).

6.4.5 Using variables as input and output arguments for commands

Consider the syntax of the `MACRO INPUT STRING` command, shown in [Figure 6.1](#).

This command does the following:

- Displays a window with an optional user-defined title (second argument).
- Displays an optional user-defined prompt (third argument).
- Stores the value entered by the user in a variable (first argument).

The Command Reference help indicates that the first argument is an output argument, while the other two are input arguments:

Figure 6.1 An example of information provided by the Command Reference: The first argument is an “output” argument (i.e., receives a value), and the other arguments are “input” arguments (i.e., provide a value).

MACRO INPUT STRING (string, string, array of strings, string)			
Description :			
Pops up an interface to query the user for a string value			
Arguments	Optional	In/Out	Description
string	no	out	String value
string	yes	in	Window title
array of strings	yes	in	Window prompt
string	yes	in	Suggested value

- Input argument: An argument that passes information to a command is called an *input* argument. It can be specified directly as a value, or as a variable name preceded by the \$ character.
- Output argument: An argument that receives information from a command is called an *output* argument. In this case, the argument must be a variable, and it is not preceded by the \$ character.

The following example uses the MACRO INPUT STRING command. It queries the user for a project name and assigns it to the output argument, the variable vProject_name. The same variable is then used as an input argument for the FILE SAVE_AS command.

```

DECLARE vProject_name
DECLARE vTitle "Project Information"
MACRO INPUT STRING (vProject_name, $vTitle, "Please enter a project name")
# the variable vProject_name is used as an output argument and does not take
# a preceding $ character
# save the project using the value stored in the variable vProject_name
FILE SAVE_PROJECT_AS ($vProject_name)
# the variable vProject_name is used as an input argument, and requires a
# preceding $ character

```

The MACRO INPUT DOUBLE, MACRO INPUT INTEGER, and MACRO INPUT STRING commands also allow querying the user, and assign the value (of type double, integer, and string) entered in the query window to an output argument variable; for more information on these commands, see [Section 7.3.2 Querying the user](#). Other

commands, which end typically with GET, can also be used to communicate a value to an output argument variable.

```
DECLARE vAxis
DECLARE vColor
# gets the destination axis
# to perform a plane, axis, center point alignment
ALIGN PLANE_AXIS_CENTER_POINT DESTINATION_AXIS GET (vAxis)
# gets the color used to draw cross-sections in color map mode
MEASURE CROSS_SECTION OPTIONS DISPLAY VERTEX_COLOR GET \
(vColor)
```

The GET command is especially useful for storing specific configuration values and parameter values in variables. New values can then be assigned to these items and used temporarily. Finally, the items' original values can be restored. In the example that follows, the vMaxAngleBackup variable is first used as an output argument variable and then as an input argument variable (with a preceding \$).

```
DECLARE vMaxAngleBackup
# make a backup before changing the max angle value
FILE OPTIONS IMPORT PLANAR_GRID MAX_ANGLE GET (vMaxAngleBackup)
# changing the max angle value
FILE OPTIONS IMPORT PLANAR_GRID MAX_ANGLE (10)
# do something that is affected by this new value
#
# restoring the original value
FILE OPTIONS IMPORT PLANAR_GRID MAX_ANGLE ($vMaxAngleBackup)
```

Note that GET commands can also be entered in the command-line area of the Command History pane without an output argument variable. The output information appears in the echo of the command in the pane. This is true for all commands with output argument variables. This information may be useful in the case of a macro script under construction. For an example, see [Figure 6.2](#).

6.4.6 Array variables

An array is a variable that can contain several values. An index is used to access each value. An array can then be conceived of as several variables, all having the same name, but with a different index. Arrays are useful when automating tasks that have a number of elements that varies, unknown to you at the time you write the macro script.

For an example of using an array in a macro script, see [Section 9.4.2 Macro script with an array variable](#).

6.4.6.1 Assigning values to an array

A variable becomes an array when it is used as an array, for example when it is assigned a second value. There is no explicit statement to identify a variable as an array. All variables, including arrays, are declared the same way.

Like variables, array elements (same variable name but different index) do not have a type. An element contains the last value that was assigned to it, so an array can contain elements of different types. To assign values to an array, place them between brackets, separated by commas:

```
DECLARE vMyArray {1, 2.2, "3rd!"}
```

To add/access an array element, for example the fourth one, square brackets [] are used:

```
SET vMyArray[4] "Fourth"
# display the first four array elements (the $ character is needed)
MACRO ECHO (" $vMyArray[1] $vMyArray[2] $vMyArray[3] $vMyArray[4]")
# displays: 1 2.2 3rd! Fourth
```

Note the following in this example:

- The array vMyArray is declared in a standard manner.
- An integer, a double, and a string value are assigned to the first three elements of the array.
- When using the SET statement, the variable is not preceded by the \$ symbol. This is equally true for an array.
- The elements can be displayed using the MACRO ECHO statement. In this case, the \$ is required in order to replace the elements by their value in the text string to be displayed.

You can copy an entire array to another variable using the SET command:

```
# declare and assign values to three variables
DECLARE vMyFirstArray {11, 22, 33, 44, 55}
DECLARE vMySecondArray {1, 2, 3}
DECLARE vName "John"
SET vMySecondArray $vMyFirstArray
SET vName $vMyFirstArray
# now all three variables are identical arrays {11, 22, 33, 44, 55}
```

Note that you can also provide an array of values to a command without using a variable:

```
DECLARE vArrayMacros {"C:\macrodir\macro1","C:\macrodir\macro2", \
```

```
"C:\macrodir\macro3"}
# the two commands that follow are identical...they open three macros
MACRO FILE OPEN({"C:\macrodir\macro1","C:\macrodir\macro2", \
"C:\macrodir\macro3"})
MACRO FILE OPEN($vArrayMacros)
```

Note that if the same array of values are used elsewhere and could change, good programming practices suggest using an array variable instead of providing an array of values. You could then change the values assigned to the array variable and everywhere the variable is used its contents are automatically updated.

From version 6.0:

You can provide an empty array to a command using the { } syntax:

```
DECLARE count
# Counts the number of occurrences of "A" in an empty array (hard)
MACRO ARRAY ELEMENTS COUNT_OF ( { }, "A", count )
```

6.4.6.2 Using array indices

Arrays are indexed using positive one-based (≥ 1) integers. As for all variables, elements that have not been assigned a value have the default value integer 0. This is true even if the index used is invalid.

```
DECLARE vMyFirstArray
# display an element for which a value has not been explicitly assigned
MACRO ECHO ($vMyFirstArray[22])
# displays: 0
# use of an invalid index (indices are one based)
MACRO ECHO ($vMyFirstArray[0])
# displays: 0
```

The direct use of double, string, or negative integer values, or of an expression as an index, is invalid and is detected when the macro script is compiled.

```
DECLARE vMyFirstArray
# the following statements will not be compiled
MACRO ECHO ($vMyFirstArray[2.4])
MACRO ECHO ($vMyFirstArray["hello"])
MACRO ECHO ($vMyFirstArray[EXPR (4 + 2)])
```

However, if a variable is used as an index, and the variable is not an integer type, a conversion of the variable output will be performed (see [Section 6.9.3 Implicit data type conversions](#)). If the conversion does not result in a valid index, integer 0 is used.

```

DECLARE vMyFirstArray
DECLARE vIndex
SET vMyFirstArray[2] 28

# case where the index is converted from double to integer
SET vIndex 2.4
MACRO ECHO ($vMyFirstArray[$vIndex])
# displays: 28

# case where the index is converted from string to integer
SET vIndex "2"
MACRO ECHO ($vMyFirstArray[$vIndex])
# displays: 28

# case where the conversion does not result in a valid index - integer 0 is used
SET vIndex "hello"
MACRO ECHO ($vMyFirstArray[$vIndex])
# displays: 0

```

6.4.6.3 Using implicit indices

An array cannot be assigned to an array element. In this case, the implicit index 1 is used for the source array:

```

# declare and assign values to two arrays
DECLARE vMyFirstArray {11, 22, 33}
DECLARE vMySecondArray {1, 2, 3}

# an array cannot be assigned to an array element
SET vMyFirstArray[1] vMySecondArray
# in this case, an implicit index is used: vMySecondArray[1]
MACRO ECHO ($vMyFirstArray[1])
# displays: 1

```

6.4.6.4 Finding the size of an array

The SIZE keyword returns the number of elements in an array. It has the following syntax:

```

SIZE <variable_name>

```

Variables that have not been used as an array have a size of 1.

```
# display the size after declaration
DECLARE vMyFirstArray
MACRO ECHO (SIZE(vMyFirstArray))
# displays: 1

# display the size after assigning a value
SET vMyFirstArray 2
MACRO ECHO (SIZE(vMyFirstArray))
# displays: 1
```

The size of an array increases as elements are added using the SET statement.

```
DECLARE vMyFirstArray
# add new elements
SET vMyFirstArray {1, 2, 3}
MACRO ECHO (SIZE(vMyFirstArray))
# displays: 3
```

Array elements do not have to be added in a sequential manner. As a result, the size of an array is equal to the largest index used to assign a value to an element of an array.

```
DECLARE vMyFirstArray
# assign a value to the 100th element
SET vMyFirstArray[100] 1
MACRO ECHO (SIZE (vMyFirstArray))
# displays: 100
```

However, accessing an array element in read-only mode (using \$) does not increase the array size.

```
DECLARE vMyFirstArray
SET vMyFirstArray {22, 33}
MACRO ECHO (SIZE (vMyFirstArray))
# displays: 2

# read the 100th element
MACRO ECHO ($vMyFirstArray[100])
# displays: 0
MACRO ECHO (SIZE (vMyFirstArray))
# displays: 2
```

From version 6.0, the size of a variable that was left empty is zero.

6.4.7 Reserved variables

Reserved variables are reserved for use by the PolyWorks Metrology Suite. They do not have to be declared and are always available. Reserved variables are those that begin with the underscore character `_` (e.g., `_DOLLAR`). Users cannot declare variables that begin with an underscore.

Reserved variables were introduced in version 3 of the language. It is possible to use reserved variables in earlier versions, but they are considered to be user-defined variables and as such must be declared.

To see the list of reserved variables available for an application, access the LANGUAGE > RESERVED VARIABLES branch of the Command Reference (see [Section 2.4.2 Command Reference Guide](#)). The list is also displayed in the last column of the table that follows. One group of reserved variables, called *escaped characters*, are offered by all the applications. Their purpose is to overcome limitations with respect to the content of a string.

Characters that cannot be included in a string		Reserved variable containing the character	All reserved variables
Character	Why it cannot be included in a string		
\$	The dollar sign is used to precede variables to access their content.	<code>_DOLLAR</code>	RESERVED VARIABLES <code>_APP_NB_BIT</code> <code>_DOLLAR</code> <code>_INSTALL_PATH</code> <code>_NEWLINE</code> <code>_PI</code> <code>_PWK_FILES_PATH</code> <code>_QUOTES</code> <code>_TAB</code> <code>_TEMP_PATH</code> <code>_USERCONFIG_PATH</code>
" "	Quotes are used to delimit a string, so they cannot be used within a string.	<code>_QUOTES</code>	
new line character	An invisible character that adds a new line and that must be surrounded by quotes that must be on the same line.	<code>_NEWLINE</code>	

Note that while the TAB character is not excluded from being in a string, the `_TAB` reserved variable was added to the list because it is found in other popular script languages.

Here is an example that uses reserved variables:

```
MACRO ECHO \
("300 M${_DOLLAR}!${_NEWLINE}${_TAB}${_QUOTES}Wow!${_QUOTES}")
# displays: 300 M$!
#           "Wow!"
```


Note that braces are used in the example above to delimit variables names from the characters that follow. For more information, see [Section 6.4.4.1 Using braces to delimit variable names](#).

6.4.8 String variable modifiers

Modifiers are expressions that modify the result of a substitution operation (i.e., when a string value is substituted for the string variable name in a statement or a command). Modifiers must be specified immediately after the variable name. If the variable name is enclosed in braces, the modifiers must follow the variable name inside the braces. Modifiers do not change the value stored in the variable. The list of modifiers, by category, follows. Each category is followed by an example.

General string variable modifiers:

- `:l` Converts all string characters in English to lowercase; has no effect in other languages.
- `:u` Converts all string characters in English to uppercase; has no effect in other languages.

Examples of string variable modifiers:

```
DECLARE vPlane_name "Plane 4"
MACRO ECHO ("vPlane_name: $vPlane_name")
# displays: vPlane_name: Plane 4

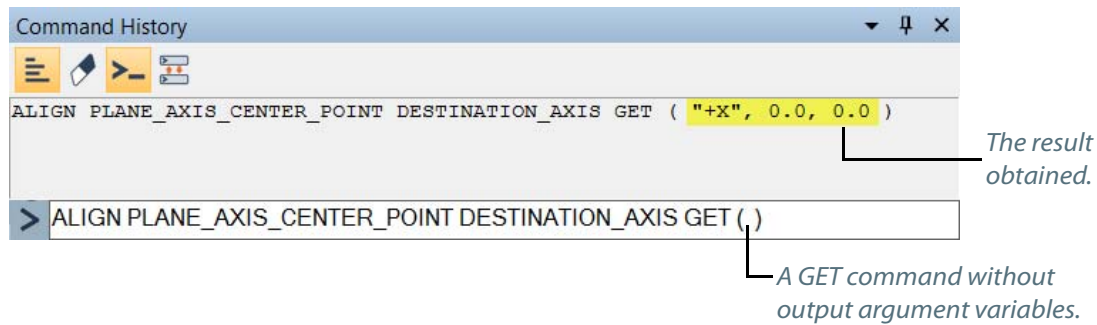
# the :l modifier
MACRO ECHO ("vPlane_name in lowercase: $vPlane_name:l")
# displays: vPlane_name in lowercase: plane 4

# the :u modifier
MACRO ECHO ("vPlane_name in uppercase: $vPlane_name:u")
# displays: vPlane_name in uppercase: PLANE 4
```

Specific string variable modifiers, for string values that are file names:

- `:e` Removes all but the suffix. Useful for extracting the extension from a file name.
- `:h` Removes a trailing path name component, leaving the head. Useful for extracting the parent folder from a path name.
- `:r` Removes a trailing suffix of the form `.xxx`, leaving the base name. Useful for removing the extension from a file name.
- `:t` Removes all leading path name components, leaving the tail. Useful for extracting the file name from a path name.

Figure 6.2 An example of giving a GET command in the Command History pane without an output variable. The information obtained is displayed in the echo of the command.



An example of each specific string variable modifier applied to a file name:

declaring the string variable

```
DECLARE vFilename "c:\temp\backup\readme.txt"
```

the :e modifier - extracts the file extension of the file name

```
MACRO ECHO ("Extension of the file name is: $vFilename:e")
```

displays: Extension of the file name is: txt

the :h modifier - extracts only the path of the parent folder of the file name

```
MACRO ECHO ("Folder of the file name is: $vFilename:h")
```

displays: Folder of the file name is: c:\temp\backup

the :r modifier - extracts all but the extension of the file name

```
MACRO ECHO ("file name with new extension: ${vFilename:r}.doc")
```

displays: file name with new extension: c:\temp\backup\readme.doc

the :t modifier - extracts only the file name and extension of the

complete file name

```
MACRO ECHO ("Tail of the file name is: ${vFilename:t}")
```

displays: Tail of the file name is: readme.txt

6.4.9 Incrementing and decrementing variable values

Two statements allow incrementing and decrementing the value stored in a variable:

```
++ <variable_name>
```

increments by 1 the value of <variable_name>.

```
-- <variable_name>
```

decrements by 1 the value of `<variable_name>`. The following examples illustrate these statements:

```
DECLARE vNumber 4
++ vNumber
MACRO ECHO ($vNumber)
# displays: 5.0
-- vNumber
MACRO ECHO ($vNumber)
# displays: 4.0
```

These two statements are especially useful for controlling the number of loops that will be executed using the WHILE structure, described in [Section 6.7.3 Repeating the execution of statements using the WHILE statement](#).

6.4.10 The scope of variables

Variables only exist and are available with a scope. In a simple script, the entire script may be the scope and defined variables are available at all times. However, if the script has logical structures (e.g., WHILE and IF), a variable that is defined within a logical structure only exists within the scope of that logical structure. This subject is presented in [Section 6.8 Controlling the scope of variables](#).

6.4.11 Special variables

In the previous section, the creation and use of variables was described. This section describes the following groups of special variables:

- Macro script variables are variables that are automatically declared and to which values are automatically assigned; you can access their content (read), but cannot edit it.
- Global variables are shared variables, offered by the scripting language and not the MSCL, that are useful in sharing information between module scripts.

6.4.11.1 Macro script variables

Macro script variables are a special category of variables managed by the MSCL. When a macro script is called by the MACRO EXEC command, macro script variables are automatically declared and are automatically assigned values that the user can access but cannot change.

The MACRO EXEC command and macro script variables are discussed in [Section 8.2 Modular programming](#).

6.4.11.2 Global variables

The scripting language, and not the MSCL, offers shared variables which are useful in sharing information between module scripts, such as a user or company name.

Shared variables are totally different from the notion of MSCL variables. They are created, accessed, and deleted using MACRO ... commands. For complete information, see [Section 7.3.6 Using shared variables](#).

6.5 Constructing mathematical expressions

The `EXPR` and `EXPR_I` keywords are used to construct mathematical expressions:

```
EXPR (<mathematical expression>)  
EXPR_I (<mathematical expression>)
```

The difference between these keywords is that `EXPR_I` returns only the integer component of the result of the mathematical expression, without any rounding.

This subsection first presents the available mathematical operators and mathematical functions and then explains how to use them within mathematical expressions.

6.5.1 Mathematical operators

The following mathematical operators can only be used within mathematical expressions:

+	(addition)	/	(division)
-	(subtraction)	%	(modulo)
*	(multiplication)		

Note that parentheses `()` allow changing the order of evaluation in a mathematical expression.

A simple mathematical expression consists of 2 operands and an operator:

```
<operand> <Mathematical operator> <operand>
```

Operands can be values or variables.

Here are some examples of mathematical expressions, which always require using the EXPR keyword:

```
DECLARE vVar1 EXPR (2 * 4)
DECLARE vVar2 EXPR (20 - 4)
DECLARE vMyResult EXPR ($vVar2 / $vVar1)
MACRO ECHO ($vMyResult)
# displays: 2.0 (i.e., 16/8)
```

The EXPR keyword is presented in [Section 6.5.3 Mathematical expressions](#) along with additional examples of mathematical expressions.

The standard order of evaluation of operators in a mathematical expression is respected:

Order of evaluation	Operators (from left to right)
1	Mathematical expressions in parentheses
2	Unary - (e.g., -2)
3	*, /, %
4	+, -

6.5.2 Mathematical functions

The mathematical functions described in the table that follows can only be used within mathematical expressions.

Mathematical function	Description
ABS	absolute value of x
ACOS	arccosine of x (returns a value in radians)
ASIN	arcsine of x (returns a value in radians)
ATAN	arctangent of x (returns a value in radians)
COS	cosine of x
LOG10	base-10 logarithm value of x
POW	value of x raised to the power of y
SIN	sine of x
SQRT	square root of x
TAN	tangent of x

Here is an example of using a mathematical function within a mathematical expression:

```
DECLARE vNumber1 2
DECLARE vNumber2 4
DECLARE vMyResult
Set vMyResult EXPR (POW($vNumber1,$vNumber2))
MACRO ECHO ($vMyResult)
# displays: 16.0
```

The EXPR keyword is presented in [Section 6.5.3 Mathematical expressions](#) along with additional examples of mathematical expressions.

6.5.3 Mathematical expressions

The EXPR and EXPR_I keywords are used to construct mathematical expressions:

```
EXPR (<mathematical expression>)
EXPR_I (<mathematical expression>)
```

The difference between these keywords is that EXPR_I returns only the integer component of the result of the mathematical expression, without any rounding.

The standard order of evaluation of operators in a mathematical expression is respected (for more information, see [Section 6.5.1 Mathematical operators](#)). Mathematical expressions can only contain values that are numerical or double or integer type variables.

The result type depends on the operands used in the mathematical expression. It is integer if all operands are integer, and double if there is a double operand. The division operation results in a double value, and the modulo operation, which requires integer

operands, always produces an integer result. The examples that follow illustrate the evaluation, and the result type, of mathematical expressions.

assigning the result of mathematical expressions to variables

```
DECLARE vMyInteger EXPR (4 + 8)
```

```
DECLARE vMyDouble EXPR (4.2 * $vMyInteger)
```

displaying the result type

```
MACRO ECHO ("The value of vMyInteger is $vMyInteger")
```

displays: The value of vMyInteger is 12

```
MACRO ECHO ("The value of vMyDouble is $vMyDouble")
```

displays: The value of vMyDouble is 50.4

parentheses can be used to change the order of evaluation

of a mathematical expression

```
SET vMyDouble EXPR (4.2 * (4 + 8))
```

```
MACRO ECHO ("The value of vMyDouble is $vMyDouble")
```

displays: The value of vMyDouble is 50.4

```
SET vMyDouble EXPR (4.2 * 4 + 8)
```

```
MACRO ECHO ("The value of vMyDouble is $vMyDouble")
```

displays: The value of vMyDouble is 24.8

Mathematical expressions can also be used as arguments. In the example that follows, the MACRO INPUT INTEGER command is used to display a window that displays the string *Enter a value*, read the value entered by the user at the keyboard, and assign it to the vGraylevel variable. However, the next command, EDIT OBJECT COLOR, cannot accept as arguments values larger than 255. In this case, the modulo operator provides the required control for all three arguments. Note the use of the command continuation character \.

```
DECLARE vGraylevel
```

```
MACRO INPUT INTEGER (vGraylevel, , "Enter a value")
```

```
EDIT OBJECT COLOR(EXPR($vGraylevel % 256), \
                    EXPR($vGraylevel % 256), \
                    EXPR($vGraylevel % 256))
```

The example that follows use the EXPR_I keyword to return only the integer component of a mathematical expression as an argument.

```
MACRO ECHO (EXPR_I (5.4))
```

displays: 5

```
MACRO ECHO (EXPR_I (-5.4))
```

displays: -5

The `EXPR` and `EXPR_I` keywords can also be included within a mathematical expression. This is particularly useful, as `EXPR_I` can provide the integer component of a value to an operator that requires integer operands, such as the modulo operator (`%`).

```
DECLARE vMyDouble 4.3
# Modulo "%" requires 2 integer operands
MACRO ECHO (EXPR(8 % EXPR_I ($vMyDouble)))
# displays: 0 (because the 8 % 4 operation produces no remainder)
```

6.6 Constructing logical expressions

A logical expression returns a boolean value (True or False) on evaluation. Logical and comparison operators are used to compose a logical expression.

This subsection first presents the comparison and the logical operators used in logical expressions and then presents logical expressions.

Logical expressions can be used as conditions within `IF` and `WHILE` structures; for more information, see [Section 6.7 Controlling the flow of execution](#).

Logical expressions can also be used to assign a resulting boolean value within `DECLARE` and `SET` statements; for more information, see [Section 6.4.3 Assigning values to existing variables](#).

6.6.1 Comparison operators

A comparison operation within a logical expression is composed of one operator and two operands. An operand can be a specific value or a variable.

<operand> <Comparison operator> <operand>

The following comparison operators can be used in logical expressions:

<code><</code>	smaller than
<code><=</code>	smaller than or equal to
<code>></code>	larger than
<code>>=</code>	larger than or equal to
<code>EQ</code>	equal to
<code>!EQ</code>	different from
<code>==</code>	equal to
<code>!=</code>	different from

The logical expressions in the following examples all return TRUE (i.e., numerical 1):

```

DECLARE vVar1 10
DECLARE vVar2 20
DECLARE vVar3 30
DECLARE vMyResult
# Example 1
SET vMyResult $vVar1 < $vVar2
MACRO ECHO ($vMyResult)
# displays: 1

# Example 2
SET vMyResult ($vVar3 >= $vVar2)
MACRO ECHO ($vMyResult)
# displays: 1

# Example 3
SET vMyResult $vVar1 != $vVar2
MACRO ECHO ($vMyResult)
# displays: 1

```

Comparison operators in a logical expression are evaluated from left to right; for more information, see [Section 6.6.4 Evaluating logical expressions](#).

6.6.2 Logical operators

A logical expression can have one or more logical operators, each having two operands. Operands of logical operators must be boolean values (i.e., True or False) or evaluate to a boolean value.

<operand> <Logical operator> <operand>

The logical operators that follow can be used in logical expressions:

Logical operator	Description
AND	Returns True if both statements are true. Example that returns True: (3 < 4) AND (6 > 5)
OR	Returns True if one of the statements is true. Example that returns True: (7 < 11) OR (10 < 2)

Logical operator	Description
NOT	Reverses the result. Returns False if the result is true or True if the result is false. Example that returns True: NOT (6 > 9)

The “AND” and “OR” logical operators allow constructing logical expressions with any number of comparison operators, while the “NOT” logical operator reverses the result of a logical expression.

Here are some simple examples of logical expressions using logical operators. In each case, the expression returns True (i.e., numerical 1).

```
DECLARE MyResult
```

```
# Example 1: Age is greater than 2 AND less than 10
```

```
DECLARE vAge 5
```

```
SET MyResult $vAge > 2 AND $vAge < 10
```

```
MACRO ECHO ($MyResult)
```

```
# Displays 1
```

```
# Example 2: Age is less than 60 OR greater than 64
```

```
DECLARE vAge1 55
```

```
SET MyResult $vAge1 < 60 OR $vAge1 >= 65
```

```
MACRO ECHO ($MyResult)
```

```
# Displays 1
```

```
# Example 3: Age is between 15 and 65 inclusively using the NOT operator
```

```
DECLARE vAge2 55
```

```
SET MyResult NOT ($vAge2 < 15 OR $vAge2 > 65)
```

```
# The parentheses are not required in the above logical expression but they make  
# it easier to read
```

```
MACRO ECHO ($MyResult)
```

```
# Displays 1
```

In complex logical expressions containing mathematical, comparison, and logical operators, the logical operators are evaluated last. For more information, see [Section 6.6.4 Evaluating logical expressions](#).

6.6.3 Logical expressions

A logical expression returns one of two values on evaluation: *True*, or *False*. False is considered as the numerical value 0, or as an empty string (""), and True is considered as any nonzero value or any nonempty string.

A logical expression is composed of logical and comparison operators. A simple logical expression has one operator and two operands. An operand can be a specific value, a boolean value (i.e., True or False), or a variable.

Here is an example of a simple logical expression containing just a comparison operator:

```
# Example of logical expression with a comparison operator: 6 > 4
# > returns True if the operand to its left is greater than the operand to its right
DECLARE VMyResult
SET VMyResult 6 > 4
MACRO ECHO ($VMyResult)
# Displays 1 (value for True)
```

Here is an example of a simple logical expression containing just a logical operator. In this case, an operand can be a variable containing a boolean value (i.e., True or False) or a comparison operation that evaluates to True or False.

```
# Example with a logical operator: AND (returns True if both operands are True)
DECLARE VMyResult
# Declaring 2 variables using logical expressions that evaluate to True values
DECLARE vResult1 6 > 4
DECLARE vResult2 2 > 1
SET VMyResult $vResult1 AND $vResult2
# The logical expression is evaluated to be True; True is assigned to vMyResult
MACRO ECHO ($VMyResult)
# Displays 1 (value for True)
```

Logical operators can be combined with comparison operators to allow for more than one comparison:

```
# A logical expression can have an infinite number of logical operators
DECLARE vPartNb "A2425"
DECLARE vNbHoles 5
DECLARE VMyResult (($vPartNb == "A2425") AND ($vNbHoles == 5))
# Parenthesis were added in the above example but are not required
# The logical expression is evaluated to be True; True is assigned to vMyResult
MACRO ECHO ($VMyResult)
# Displays 1 (value for True)
```

Note that a logical expression can have just one variable and no operator:

```
DECLARE VMyResult
DECLARE vVar1 6 > 4
# Logical expression with just one operand containing the boolean value True
SET VMyResult $vVar1
MACRO ECHO ($VMyResult)
# Displays 1 (value for True)
```

The comparison and the logical operators are presented in the subsections that follow, with additional examples of logical expressions.

6.6.4 Evaluating logical expressions

The order in which operators are evaluated in a logical expression is presented in the table that follows. The first line has the highest priority, and the priority diminishes from one line to the next. If two or more operators have the same priority, i.e. are on the same line, they are evaluated from left to right. Note that parentheses () allow changing the order of evaluation in a logical expression.

Order of evaluation	Operators (from left to right)
1	Expressions in parentheses
Mathematical operators	
2	Unary - (e.g., -2)
3	*, /, %
4	+, -
Comparison operators	
5	<, <=, >, >=, EQ, !EQ, ==, !=
Logical operators	
6	NOT
7	AND
8	OR

Here are an example of a complex logical expression:

```

DECLARE vOperatorName "Bob"
DECLARE vPartNumber 2400
DECLARE vCMMModel "A42R3"
DECLARE vResult
# Logical expression that returns True
# The use of parentheses changes the order of evaluation of the logical expression
SET vResult ($vOperatorName == "Bob" OR $vOperatorName == "John") AND \
$vPartNumber >= 2233 AND $vCMMModel != "Oldie"
MACRO ECHO ($vResult)
# Displays 1

```

6.7 Controlling the flow of execution

This subsection presents the IF-ENDIF and the WHILE-ENDWHILE structures that allow executing statements when a condition is True.

6.7.1 Constructing conditions

A condition is an expression that evaluates to a True or a False value:

- A False value may be a boolean value (i.e., False) or the numerical value 0.
- A True value may be a boolean value (i.e., True), a numerical value different than 0, or a string value.

As a result, a condition may be a logical expression or a variable. For more information on logical expressions, see [Section 6.6 Constructing logical expressions](#).

6.7.2 Conditionalizing the execution of statements using the IF statement

The IF statement permits the conditional execution of specific statements. It will also be referred to as the IF *structure*, as it begins with the IF keyword and ends with the ENDIF keyword.

6.7.2.1 One condition

A simple IF structure that processes one condition has the following syntax:

```
IF <condition>
    < Statement1 >
    < Statement2 >
ENDIF
```

If the <condition> is True, the statements are executed. If the <condition> is False, the execution of the macro script continues with the line after the ENDIF keyword. For more information on conditions, see [Section 6.7.1 Constructing conditions](#).

```
DECLARE vNbHoles 3
DECLARE vMinNbHoles 4
IF $vNbHoles < $vMinNbHoles
    # display a message to the effect that there are not enough holes
    MACRO ECHO ("Not enough holes.")
ENDIF
```

The IF structure also permits the conditional execution of one of two sets of statements. In this case, the IF structure has the following syntax:

```
IF <condition>
    < Statement1.1 >
    < Statement1.2 >
ELSE
    < Statement2.1 >
    < Statement2.2 >
ENDIF
```

If the <condition> is True, the first set of statements is executed. If the <condition> is False, the second set of statements is executed. In both cases, the execution of the macro script continues with the line after the ENDIF keyword.

The example that follows evaluates a mathematical expression that results in numeric 0, which is a False condition. Consequently, the second set of statements will be executed.

```
IF EXPR (8 % 2)
    # it would be awkward to read this
    MACRO ECHO ("8 is an odd number")
ELSE
    MACRO ECHO ("8 is an even number")
ENDIF
```

6.7.2.2 Multiple conditions

The IF structure allows specifying an unlimited number of conditions. Each condition is evaluated in sequential order until a True condition is found. When a condition is True, its statements are executed, and the execution of the macro script continues with the line after the ENDIF keyword. In this case, the IF structure has the following syntax:

```
IF <condition1>
    < Statement1.1 >
    < Statement1.2 >
ELSEIF <condition2>
    < Statement2.1 >
    < Statement2.2 >
ELSEIF <condition3>
    < Statement3.1 >
    < Statement3.2 >
ENDIF
```

If <condition1> is True, the first set of statements is executed. If <condition1> is False, <condition2> is evaluated. If it is True, the second set of statements is executed. If it is False, <condition3> is evaluated. If it is True, the third set of statements is executed. If no condition is True, no statements are executed. In all cases, the execution of the macro

script continues with the line after the ENDIF keyword. The example that follows has three conditions.

```
DECLARE vAge 25
IF $vAge < 10
    MACRO ECHO ("Age is less than 10")
ELSEIF $vAge <= 40
    MACRO ECHO ("Age is between 10 and 40 inclusively")
ELSEIF $vAge > 40
    MACRO ECHO ("Age is greater than 40")
ENDIF
```

Note that the comparison operations shown in the example above can be combined using logical operators. For example:

```
DECLARE vAge 25
IF $vAge =>10 AND $vAge <= 40
    MACRO ECHO ("Age is between 10 and 40 inclusively")
ENDIF
```

6.7.3 Repeating the execution of statements using the WHILE statement

The WHILE statement is used to repeat statements, depending on a run-time condition. It will also be referred to as the WHILE structure, because it begins with the WHILE keyword, and ends with the ENDWHILE keyword.

The WHILE structure has the following syntax:

```
WHILE <condition>
    <Statements>
ENDWHILE
```

As long as the <condition> is True, the set of statements is repeated. When the <condition> is False, the execution of the macro script continues with the line after the ENDWHILE keyword. For more information on conditions, see [Section 6.7.1 Constructing conditions](#).

```
DECLARE vLoopVar 4
WHILE $vLoopVar
    MACRO ECHO ($vLoopVar)
    # decrement vLoopVar within the WHILE structure to avoid infinite looping
    -- vLoopVar
ENDWHILE
# end of example
```

On execution of these instructions, the following would be displayed:

```
4
3
2
1
```

Two additional keywords can be used within the WHILE structure:

- **BREAK:** A keyword that can be used to exit the WHILE structure.
- **CONTINUE:** A keyword that returns immediately to the beginning of the WHILE structure, bypassing the remaining list of operations within the structure.

The next example uses the BREAK keyword to exit the WHILE structure when the value of the vLoopVar variable is less than 6.

```
DECLARE vLoopVar 8
# A WHILE structure that will continue repeating instructions until the value
# of the vLoopVar variable = 0
WHILE $vLoopVar
    MACRO ECHO ($vLoopVar)
    IF $vLoopVar < 6
        # exiting the structure
        BREAK
    ENDIF
    # Subtract one from the vLoopVar variable value
    -- vLoopVar
ENDWHILE
```

Execution displays:

```
8
7
6
5
```

The next example uses the CONTINUE keyword to avoid executing the MACRO ECHO (\$vLoopVar) statement, as long as the value of the variable vLoopVar is greater than 4.

```
DECLARE vLoopVar 8
WHILE $vLoopVar
    # Subtract one from the vLoopVar variable value
    -- vLoopVar
    IF $vLoopVar > 4
        # jump back to the WHILE statement
        CONTINUE
    ENDIF
```



```
MACRO ECHO ($vLoopVar)
ENDWHILE
```

On execution of these instructions, the following would be displayed on separate lines:

```
4
3
2
1
0
```

6.7.4 Other examples

The family of MACRO utility commands also offers ways of controlling script execution. They are presented later in this document, in these sections:

- [Section 7.3.3 Pausing macro scripts](#)
- [Section 8.2 Modular programming](#)
- [Section 8.3 Error handling](#)

6.8 Controlling the scope of variables

Scope refers to the availability, or visibility, of variables within a macro script. A variable is visible from the moment it is declared within a specific scope, and remains visible until the end of that scope. Variables do not exist outside of their scope.

Each logical structure (e.g., WHILE and IF) has its own scopes.

Example #1:

```
# beginning of File scope
IF <condition>           beginning of True scope
...
ENDIF                   end of True scope
# end of File scope
```

Example #2:

```
# beginning of File scope
IF <condition>           beginning of True scope
...
ELSE                     end of True scope/beginning of False scope
```

```

...
ENDIF          end of False scope
# end of File scope

```

Example #3:

```

# beginning of File scope
WHILE <condition>  beginning of Loop scope
...
ENDWHILE          end of Loop scope
# end of File scope

```

The largest scope is the File scope. It starts at the beginning of a macro script and ends at the end of the macro script. Scopes can be nested within other scopes. In the previous example, each scope is nested within the File scope. The following example illustrates nested scoping.

```

# beginning of File scope
WHILE <condition>  beginning of Loop scope
    IF <condition>  beginning of True scope1
        ...
        IF <condition>  beginning of True scope2
            ...
        ENDIF          end of True scope2
    ELSE              end of True scope1/beginning of False
scope
        ...
    ENDIF          end of False scope
ENDWHILE          end of Loop scope
# end of File scope

```

A variable is said to be declared *at File scope* when its declaration is made outside of any logical structure. In the following example, the variables vA and vE are declared at File scope.

```

# beginning of File scope
DECLARE vA
WHILE <condition>
    DECLARE vB
    IF <condition>
        IF <condition>
            DECLARE vC
        ENDIF
    ELSE
        DECLARE vD
    ENDIF
ENDWHILE
DECLARE vE
# end of File scope

```

Variables declared at File scope exist from the moment of their declaration to the end of the macro script. All other variables exist from the moment of their declaration to the end of their scope. In the example in [Figure 6.3](#), the scope, or life span, of each variable is indicated by way of comments, as well as lines.

The notion of nested scopes authorizes the use of variables having the same name but having different scopes. The variable that is the most nested is the one that is used.

```
# declaring the first variable vA, which we will refer to it as first vA
DECLARE vA 100
# display first vA
MACRO ECHO ($vA)
IF $vA > 50
    # display first vA
    MACRO ECHO ($vA)
    # declaring the second variable vA, which we will refer to as second vA
    DECLARE vA 200
    # display second vA, which has the most nested scope
    MACRO ECHO ($vA)
ENDIF
# display first vA (second vA does not exist any more as we are outside its scope)
MACRO ECHO ($vA)
```

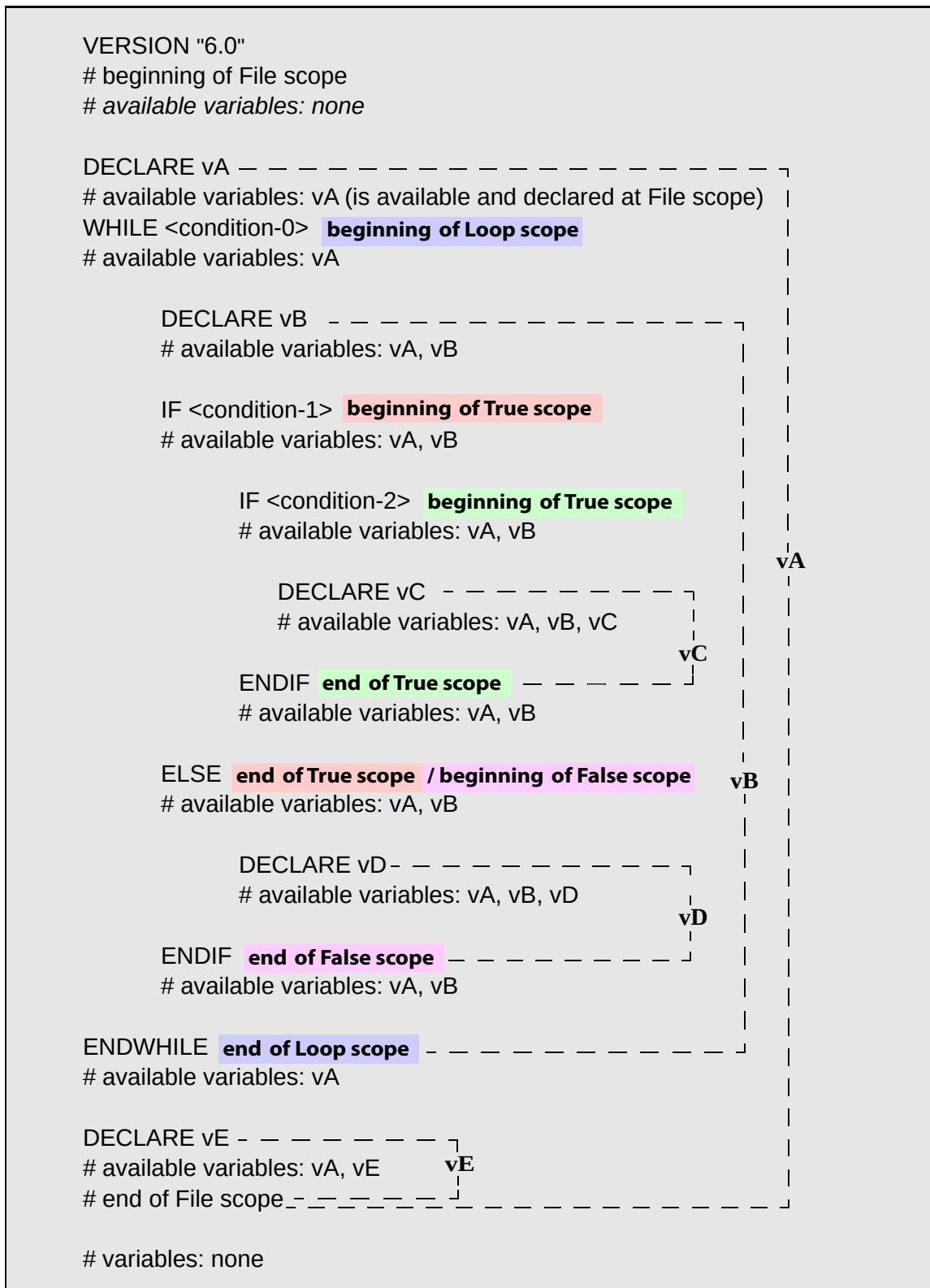
On execution of these instructions, the following would be displayed:

```
100
100
200
100
```

In the next example, similar to the previous one, a variable vA is used to initialize a second variable vA that is defined within a more nested scope.

```
# declaring the first variable vA
# let's refer to it as First vA
DECLARE vA 100
# display First vA
MACRO ECHO ($vA)
IF $vA > 50
    # display First vA
    MACRO ECHO ($vA)
    # declaring the second variable vA
    # let's refer to it as Second vA
    # First vA is used to initialize Second vA
    DECLARE vA EXPR (2 * $vA)
    # display Second vA (Second vA has the most nested scope)
```

Figure 6.3 An example of the scope of variables.



```

    MACRO ECHO ($vA)
ENDIF
# display First vA (Second vA doesn't exist anymore, as we are outside it's scope)
MACRO ECHO ($vA)

```

On execution of these instructions, the following would be displayed:

```

100
100
200.0
100

```

6.9 Converting data types

The Macro Script Control Language has three built-in data types: *integer*, *double*, and *string* (ASCII or Unicode). The first two are numeric data types, while the third is a character data type.

This section explains the data type conversions that can occur when using the Macro Script Control Language. A conversion occurs when data of type A is converted to data of type B with or without the loss of precision or information. When the conversion is requested by the user through an element of the control language, it is said to be *explicit*. Conversions that occur without being requested are referred to as *implicit*.

This section presents the standard conversion from one data type to another, and then presents operations that result in explicit data type conversion as well as situations that result in implicit data type conversion.

6.9.1 Standard data type conversion

Data types may be converted from one data type to another data type, either explicitly or implicitly. Certain conversions may result in a loss of precision. This section explains how standard data type conversions are performed.

Possible conversions for the *integer* data type are explained in the table that follows.

Table 1: Integer conversions

Convert to	Method	Loss of precision
double	Integers are considered to be doubles with a 0 after the decimal.	none
string	The number is rendered as a string of characters. Scientific notation is never used. Negative numbers are prefixed with the "-" symbol.	none

Possible conversions for the *double* data type are explained in the table that follows.

Table 2: Double conversions

Convert to	Method	Loss of precision
integer	The decimal part is truncated, without any rounding. For example, -4.8 becomes -4, and 4.8 becomes 4. Certain doubles may not be representable if they are outside the limits of the integer range defined by (INT_MIN, INT_MAX).	yes
string	The number is rendered as a string of characters having six significant numbers. Scientific notation is used if needed. Negative numbers are prefixed with the "-" character. The numbers after the decimal are only present if they are significant and nonzero. For example, 123456.1 becomes 123456, and 4.0 becomes 4.	yes

Possible conversions for the *string* data type are explained in the table that follows.

Table 3: String conversions

Convert to	Method	Loss of precision
numeric	The numeric data type refers to the situations where a number is required, but it is not specifically an integer or a double. In this case, a conversion to double is first attempted. If it fails, a conversion to integer is applied.	yes
integer	The string value must represent a valid integer and all of its characters must be used. If the conversion fails, integer 0 is obtained.	yes
double	The string value must represent a valid double and all of its characters must be used. If the conversion fails, integer 0 is obtained.	yes

6.9.2 Explicit data type conversions

Explicit conversions may be the result of a command, or the result of inserting a variable in a character string.

Conversions made by the `EXPR`, the `EXPR_I`, and the `STRING_DECIMAL` keywords are explained in the table that follows.

Table 4: Explicit command conversions

	Conversion done by		
expression	EXPR (expression)	EXPR_I (expression)	STRING_DECIMAL (expression)
integer	none	none	string value using a decimal notation
double	none	double to integer	string value using a decimal notation
string	string to numeric	string to numeric, and then from numeric to integer	invalid

Conversions made as a result of inserting a variable in a character string are explained in the table that follows.

Table 5: Explicit variable conversions

\$variable	Conversion done by "...\$variable..."
integer	integer to string
double	double to string
string	none

6.9.3 Implicit data type conversions

There is an implicit data type conversion in the following situations:

- Command arguments

Arguments sent to a command are converted to the type of their corresponding parameters.

Command arguments are presented in [Section 6.3.1 Command format](#).

- Conditions (`==`, `!=`, `<`, `<=`, `>`, `>=`)

Conditions with only one operand undergo no conversion. However, if there are two operands and a conditional operator, and the operands are not of the same type, then there can be an implicit conversion of the type of one of the operands.

Comparisons between two numeric types are done without any difficulty. In the case of numeric - string comparisons, the string operand is always converted to the numeric type.

Conditions are presented in [Section 6.6.3 Logical expressions](#).

- Mathematical expressions (+, -, *, /, %)

Mathematical operators require numeric operands. String types are first converted to numeric types. If a mathematical operation is carried out on an integer and a double, the result is a double type. However, there are two exceptions to this rule. A division always results in a double type, and a modulo operation requires integer types, and gives an integer type result.

Mathematical expressions are presented in [Section 6.5.3 Mathematical expressions](#).

- Incrementing and decrementing a variable (++ , --)

The ++var1 operation is the equivalent of the SET var1 EXPR (\$var1 + 1) statement, and the --var1 operation is the equivalent of the SET var1 EXPR (\$var1 - 1) statement. The content of the variable is converted to a numeric type, if necessary.

Incrementing and decrementing a variable is presented in [Section 6.4.9 Incrementing and decrementing variable values](#).

- Modifiers (:e, :h, :l, :t, :u)

Modifiers require string variables. If the variable is a numeric type, it is converted to string.

Modifiers are presented in [Section 6.4.8 String variable modifiers](#).

- Array index []

Arrays require integer indices. The value used as an index is converted to an integer. If the conversion fails, the default value of 0 is assigned, which is an invalid value for an index.

Array indices are presented in [Section 6.4.6 Array variables](#).

Getting Started

7

This chapter presents basic application and utility commands, and examples, that may be useful to users when writing macro scripts for the first time.

7.1 Introduction

This chapter presents application commands and utility commands to get started writing macro scripts. For each command, the content from the Command Help is provided containing the command syntax and an example is given.

7.2 Application commands

This section presents application commands and concepts, with examples, that are helpful when writing macro scripts for the first time. For example, commands are presented in the subsections that follow that allow reading/storing/setting/restoring values found in dialog boxes. Examples and syntax are provided for each command. The syntax is in the form of snapshots taken from the Command Reference documentation that is available from the Macro Script Editor's *Help* menu.

As the software evolves, so do the commands used for scripting. As new commands appear, certain commands may be made obsolete. Obsolete commands are detected in the Macro Script Editor and flagged appropriately to help the user identify them and replace them with the updated commands. A subsection explains how to update a macro script containing obsolete commands.

7.2.1 Specifying objects and object properties

A macro script typically executes actions on objects. You can specify which objects to use, and they must respect certain constraints (e.g., used, not locked, visible), similar to doing a task manually.

7.2.1.1 Specifying objects

There are several ways to specify objects to a command. Two ways are useful when the objects are known ahead of time:

- The name of the object. This lets you process specific objects.
- The index number of the object. This is useful for cycling through consecutive objects, such as all the objects under a specific branch. Note that index numbers for objects under a branch in the tree view always start at 1 and are consecutive.

Two ways are useful when the selection must be determined at run time:

- The current selection in the tree view is used. The selection can be made at run-time by the user, or be performed by a command in a script that selects objects.
- The default selection, when only one valid object exists in the tree view for the command that will be executed.

The selection must correspond to the requirements of the command to execute in the macro script. For example, certain commands only operate on specific objects, and other commands require that only one object be selected.

As selecting objects manually opens the door to human error, it is recommended to specify objects explicitly within a script when possible, by name or by index number.

Note that some commands do not require an explicit selection, but rather use all the objects of a certain type that have specific properties. The section that follows deals with specifying object properties.

7.2.1.2 Specifying object properties

When using the PolyWorks Metrology Suite, actions operate on objects that must have certain properties, such as being unlocked or visible. See the following commands in the Command Reference, available on choosing the [Help > Commands](#) item on the menu of each Macro Script Editor tool, to specify certain properties for objects:

Command	Property applied to objects
IMAlign, PolyWorks Modeler, and PolyWorks Inspector Macro Script Editor	
TREEVIEW OBJECT VIEW HIDE	hidden
TREEVIEW OBJECT VIEW RESTORE	visible
EDIT OBJECT USE	used
EDIT OBJECT IGNORE	ignored
IMAlign Macro Script Editor	
EDIT IMAGE LOCK	locked
EDIT IMAGE RELEASE	released
PolyWorks Inspector Macro Script Editor	
EDIT OBJECT LOCK	locked
EDIT OBJECT RELEASE	released

7.2.2 Getting information about objects in the tree view

Scripting offers many TREEVIEW commands. The first command that follows is an actual command. The second command is a general format – choose the category of objects (e.g., planes, points), and write as many commands as there are object categories.

- **TREEVIEW OBJECT SELECT**

TREEVIEW OBJECT SELECT (array of strings, string)			
Description : Selects or deselects the specified objects			
Arguments	Optional	In/Out	Description
array of strings	no	in	Object names
string	yes	in	Possible values: "On", "Off" Default value: "On"

This command selects or deselects an object in the tree view by its name.

Example:

Objective: Select circle 1.

Command: TREEVIEW OBJECT SELECT ("circle 1", "On")
 # or, using the default value
 TREEVIEW OBJECT SELECT ("circle 1")

- **TREEVIEW *object_category* COUNT GET**

TREEVIEW COMPARISON_POINT COUNT GET (integer)			
Description : Gets the number of comparison points			
Argument	Optional	In/Out	Description
integer	no	out	Number of objects found

Example:

Objective: Count the number of Comparison Point objects.

Command: DECLARE vNbr_CmpPtObjects
 TREEVIEW COMPARISON_POINT COUNT GET (vNbr_CmpPtObjects)

- **TREEVIEW FEATURE *feature_type* COUNT GET**

TREEVIEW FEATURE CIRCLE COUNT GET (integer, string, string)			
Description : Gets the number of circle features			
Arguments	Optional	In/Out	Description
integer	no	out	Number of objects found
string	yes	in	Consider features in patterns Possible values: "On", "Off" Default value: "Off"
string	yes	in	Consider features in cross-section feature groups Possible values: "On", "Off" Default value: "Off"

Example:

Objective: Count all circle features, including those contained within pattern features and cross-section feature groups.

Command: DECLARE vNbr_Circles
TREEVIEW FEATURE CIRCLE COUNT GET (vNbr_Circles, "On", "On")

7.2.3 Working with text boxes and check boxes

When running a macro, you may want to temporarily change values in a dialog box, execute commands using the temporary values, and then restore the original values at the end of the macro. This is possible for values in text boxes and check boxes.

To read and store the original values, output variables are used. To read the stored values, and reset them in the dialog box, input variables are used. For more information, see [Section 6.4.5 Using variables as input and output arguments for commands](#).

The macro script that follows will make use of two items, found in the PolyWorks|Inspector Options dialog box, and shown in [Figure 7.1](#):

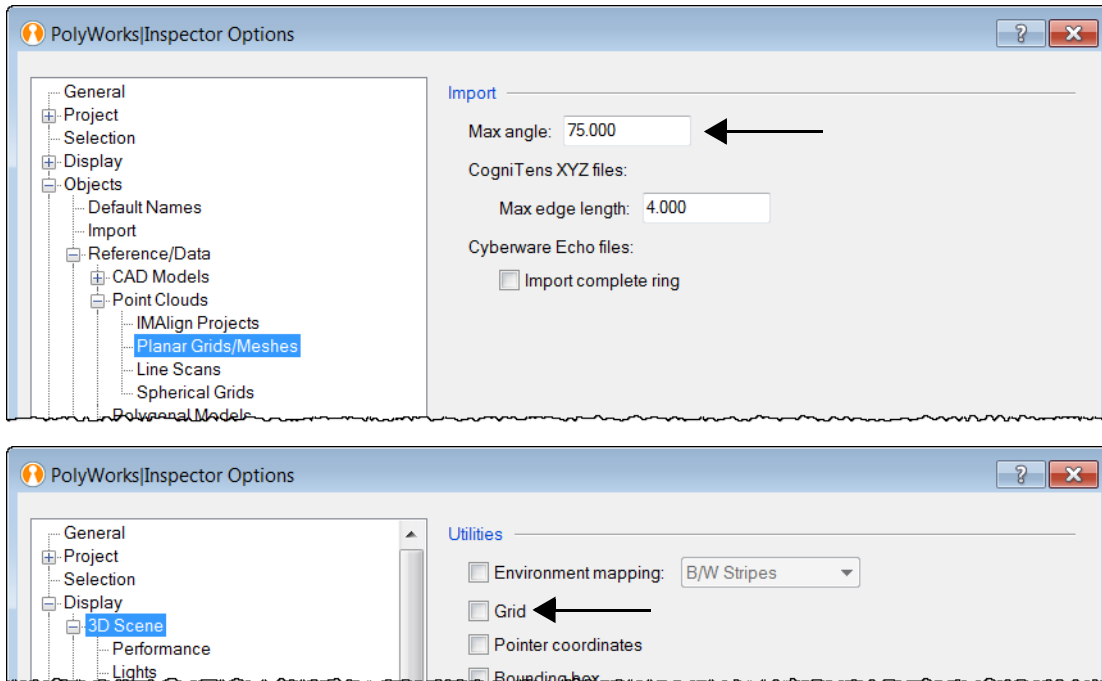
- Text box: **Max angle** (on the **Objects > Reference/Data > Point Clouds > Planar Grids/Meshes** page).
- Check box: **Grid** (on the **Display > 3D Scene** page).

Two variables are needed to store the actual values for each item:

```
DECLARE vInitial_max_angle  
DECLARE vInitial_view_grid
```

To read and store the value of each item, use the GET keyword, preceded by the appropriate keywords. Note that pressing the TAB key for command completion in the Macro Script Editor results in the display of the possible following keywords at any position in a command (see [Section 3.5.2.1 Command Completion feature](#) for more information on command completion). Note that the exact command to use can be easily found by changing the value for each option in the options dialog box and pressing the **Apply** button, and then consulting the Command History pane for the resulting commands.

Figure 7.1 Pages of the PolyWorks|Inspector Options dialog box. In each case, the arrow indicates the option that will be used in the example scripts.



The GET commands for the two options are:

```
FILE OPTIONS IMPORT PLANAR_GRID MAX_ANGLE GET (vInitial_max_angle)
VIEW GRID GET (vInitial_view_grid)
```

use MACRO ECHO commands to show the value stored in each variable:

```
MACRO ECHO ("The initial Max angle is $vInitial_max_angle")
```

displays: The initial Max angle is 75.0

```
MACRO ECHO ("Initial View Grid is $vInitial_view_grid")
```

displays: Initial View Grid is On

where "On" means that the check box is selected

Next, to set temporary values for the same items, the commands are identical, except for the GET keyword. In a sense, it is replaced by an implicit PUT keyword. Note that the values to set are sometimes specified directly in the command, or obtained as

values previously stored in variables, by an MACRO INPUT command for example. To set values for these items, two variables will be declared, and values assigned to them:

```
# declaring/assigning values to variables
DECLARE vTemp_max_angle 77
# to enable or disable a check box, set it to "On" or "Off" respectively
DECLARE vTemp_view_grid "Off"
```

Values can be set for the two options using input variables (with \$) or by specifying values directly; examples of both follow:

```
# specifying values using variables
FILE OPTIONS IMPORT PLANAR_GRID MAX_ANGLE ($vTemp_max_angle)
VIEW GRID ($vTemp_view_grid)

# specifying values directly
FILE OPTIONS IMPORT PLANAR_GRID MAX_ANGLE (77)
VIEW GRID ("Off")
```

Note also, in the case of the **Grid** check box, that three values are possible: "On", "Off", and "Toggle". If the command is given without parentheses or argument, the default action is to switch the current value of the check box to its other value (i.e., toggle).

At the end of the macro execution, reset the initial values, using input variables:

```
FILE OPTIONS IMPORT PLANAR_GRID MAX_ANGLE ($vInitial_max_angle)
VIEW GRID ($vInitial_view_grid)
```

7.2.4 Using QR codes

Two commands contained in a script run in the PolyWorks|Inspector or the IMAlign module allow Talisman to read QR Codes and generate specific text as QR Codes.

- **TALISMAN BARCODE READ**

TALISMAN BARCODE READ (array of strings)			
Description :			
Pops up the barcode reader page on the Talisman client in order to obtain barcode data			
Argument	Optional	In/Out	Description
array of strings	no	out	Barcode data

This command displays pages that guide the user as to how to scan a QR Code. This information can then be used in the script, to open a PolyWorks|Inspector or an IMAlign project of the scanned name for example.

- **TALISMAN BARCODE QR_CODE GENERATE**

TALISMAN BARCODE QR_CODE GENERATE (array of strings, string)			
Description : Generates a QR Code and writes the content to an image in the PNG format			
Arguments	Optional	In/Out	Description
array of strings	no	in	QR Code data
string	no	in	Image file name

This command allows the user to enter text that is encoded as QR Codes which can then be printed.

7.2.5 Replacing obsolete application commands

Certain application commands (e.g., commands that are the equivalent of menu items of the PolyWorks Metrology Suite modules) may become obsolete as the software evolves. In the editor, obsolete commands in a script are flagged with an orange background. Certain obsolete commands are still supported, while others are not and may hinder the execution of the script. On placing the pointer over an obsolete command, an information tooltip is displayed, stating what to do, as shown in [Figure 7.2 \(a\)](#). An obsolete command can be replaced by the following:

- A renamed command.

The command has been renamed but it still has the same parameters, so technically it can still be used. The new renamed command, and its parameters, are displayed. A warning sign ⚠ is displayed on the first line of the information tooltip. For an example, see [Figure 7.2 \(a\)](#). In this case, the correct command name should be used.

- A superseded command.

The command has been renamed and a change has been made to its parameters. A warning sign ⚠ is displayed on the first line of the information tooltip, and the command name that supersedes it, and its parameters, are displayed. In this case, the command should no longer be used – it must be rewritten using the new command name and parameters.

Note that certain obsolete commands simply can no longer be used (e.g., the subject of the command no longer exists). A danger sign ☠ is displayed on the first line of the information tooltip. Such commands must be removed from the script. If their intent is still important to the purpose of the script, other commands must be used to perform their functions.

7.2.5.1 Automatically upgrading scripts containing obsolete commands

The *Edit > Upgrade* command attempts to resolve issues related to obsolete commands, which produces the following results:

- The current script is backed up by adding the current date and time to the script name.
- The original script is modified as follows, without saving the changes:

- Renamed commands are automatically replaced.
- With respect to superseded commands, two lines in comment are added beneath the obsolete command:

The first line displays a message starting with the term TODO, which allows searching these cases easily in a long script.

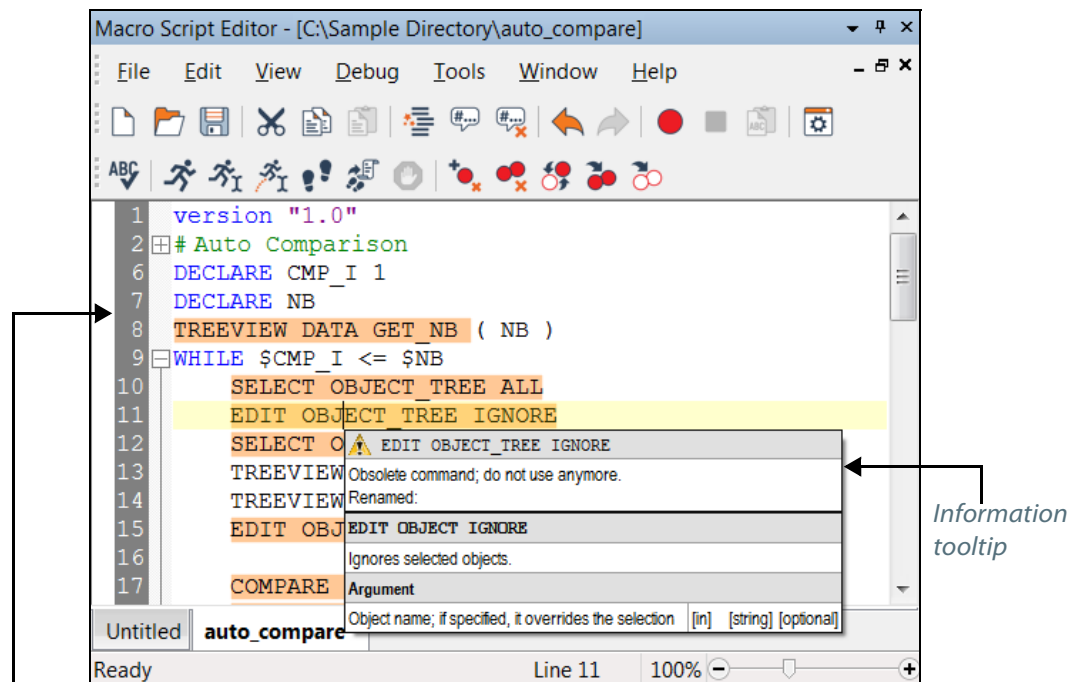
The second line includes the name of the command that supersedes the obsolete command, without any arguments.

- A window is displayed, indicating the number and type of changes made.

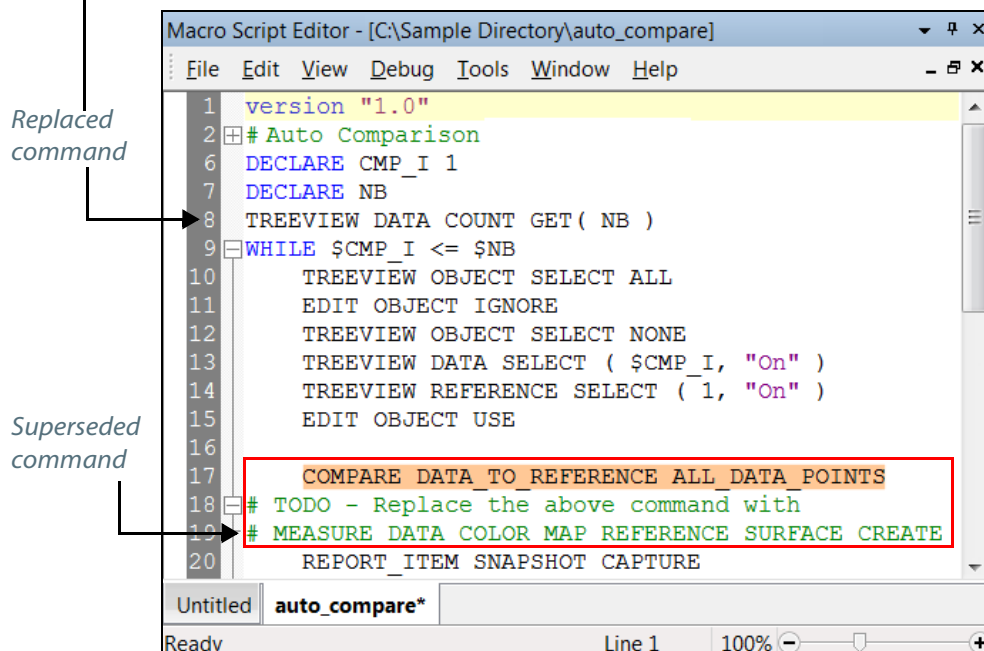
For an example of an updated script, see [Figure 7.2 \(b\)](#).

Figure 7.2 In (a), obsolete application commands are displayed with a colored background. On placing the cursor in a command, an information tooltip indicates that the command is obsolete, and is replaced by a renamed command, given underneath the message. In (b), the Edit > Upgrade operation has been performed. The first arrow points an obsolete command that has been replaced by its renamed command, and a second arrow points a command that is superseded by another command.

(a) The script before the Upgrade operation.



(b) The script after the Upgrade operation.



7.3 Utility commands

This section presents utility commands that should be of interest to users who are creating macro scripts for the first time. For example, commands to query the user for information, to read parameters stored in a text file, and to manipulate strings are presented. Each subject is presented in the subsections that follow.

7.3.1 Refreshing the screen and echoing commands

The utility commands that follow may also be useful in your macro scripts.

- **WINDOW REFRESH**

WINDOW REFRESH (string, string, string)			
Description : Enables or disables the refresh of the application's 3D Scene, status bar, and/or message bar			
Arguments	Optional	In/Out	Description
string	yes	in	Enables or disables the refresh of the application's 3D Scene Possible values: "On", "Off", "Toggle" Default value: "Toggle"
string	yes	in	Enables or disables the refresh of the application's status bar Possible values: "On", "Off" Default value: "On"
string	yes	in	Enables or disables the refresh of the application's message bar Possible values: "On", "Off" Default value: "On"

This command enables/disables the refresh of three elements of the application's graphical user interface, in this order: 3D scene, status bar, and message bar.

Disabling the screen refresh at the beginning of a long macro script, or when the project contains large Reference and Data objects, may speed up macro script execution. The screen refresh can then be re-enabled at the end of the macro script.

Note: When invoked without parentheses/arguments, toggling is performed.

- **MACRO COMMAND ECHO**

MACRO COMMAND ECHO (string)			
Description : Turns the echoing of commands ON/OFF			
Argument	Optional	In/Out	Description
string	yes	in	Possible values: "On", "Off", "Toggle" Default value: "Toggle"

This command turns on/off the echoing of commands in the Command History pane.

Note: When invoked without parentheses/arguments, toggling is performed.

7.3.2 Querying the user

The MACRO INPUT group of commands can be used to query information from a user at macro script run time. The information is stored in output variables, which must already be declared. If the command takes only one string argument, it represents the text to appear in the window title; if there is a second string argument, it represents text that will appear to prompt the user. For an example, see [Figure 7.3](#).

The dialog box that is displayed offers two buttons, **OK** and **Cancel**; the **Cancel** button generates an error code that can be detected using the MACRO GET_ERROR_STATUS command.

For the purposes of the commands that follow, the following variables need to be declared:

```
# integer variables
DECLARE vButtonpressed
DECLARE vNbrDigits
# a double variable
DECLARE vMaxAngle
# string variables
DECLARE vFolder_name
DECLARE vRef_file_name
# a string variable
DECLARE vUsername
```

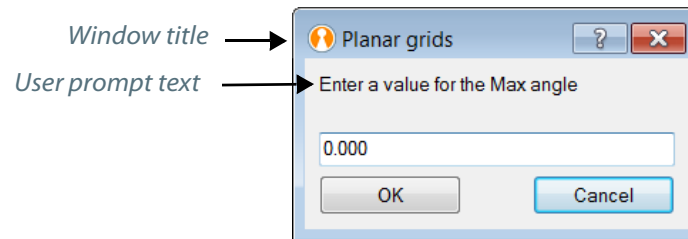
- **MACRO INPUT QUESTION**

MACRO INPUT QUESTION (integer, array of strings)			
Description :			
Pops up a question window with Yes and No buttons			
Note :			
The variable passed will contain the value 1 if the user presses the Yes button, or 0 if the user presses the No button			
Arguments	Optional	In/Out	Description
integer	no	out	Variable that will contain the value
array of strings	no	in	Window prompt

This command displays a window with the prompt message. The user presses the **Yes** or the **No** button, and the corresponding integer value, 1 or 0 respectively, is assigned to an output variable.

Example:

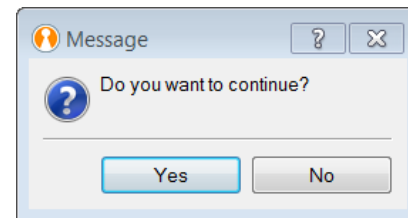
Figure 7.3 An example of the two string arguments of a MACRO INPUT ... command.



Objective: Display a window showing the message “Do you want to continue?” and store the user-pressed button in the vButtonpressed output variable.

Command: MACRO INPUT QUESTION (vButtonpressed, "Do you want to continue?")

A message window is displayed with the message (shown to the right). The **Yes** and the **No** buttons are displayed. By default the **Yes** button is selected. Press the desired button – the result, 1 for Yes, 0 for No, is stored in the output variable. The result could be used to end the current macro script:



Normally the script would first execute a list of commands...

MACRO INPUT QUESTION (vButtonpressed, "Do you want to continue?")

IF \$vButtonpressed == 0

MACRO END

ENDIF

Continue with other commands if the script was not ended

- **MACRO INPUT FOLDER_PATH**

MACRO INPUT FOLDER_PATH (string, string, string)			
Description :			
Pops up an interface to query the user for a folder name			
Arguments	Optional	In/Out	Description
string	no	out	Folder name
string	yes	in	Window title
string	yes	in	Startup folder

Example:

Objective: Display a folder browser window titled “Workspace path” and store the user-selected value in the vFolder_name output variable.

Command: `MACRO INPUT FOLDER_PATH (vFolder_name, "Workspace path")`

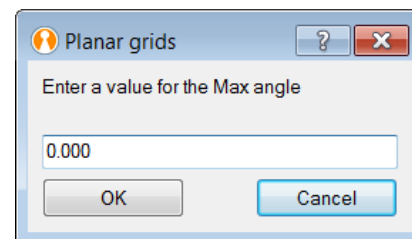
No prompt is necessary as a folder browser is displayed, letting you choose the appropriate folder. On pressing the browser's **Select Folder** button, the browsed folder is stored in the output variable.

- **MACRO INPUT DOUBLE**

MACRO INPUT DOUBLE (double, string, array of strings, double)			
Description :			
Pops up an interface to query the user for a double value			
Arguments	Optional	In/Out	Description
double	no	out	Double value
string	yes	in	Window title
array of strings	yes	in	Window prompt
double	yes	in	Suggested value

Example:

Objective: Display a window titled "Planar grids" with the prompt "Enter a value for the Max angle", and store the user-entered value in the vMaxAngle output variable.



Command: `MACRO INPUT DOUBLE (vMaxAngle, "Planar grids", "Enter a value for the Max angle")`

- **MACRO INPUT FILE_PATH**

MACRO INPUT FILE_PATH (array of strings, string, string, string)			
Description :			
Pops up an interface to query the user for file paths			
Arguments	Optional	In/Out	Description
array of strings	no	out	File paths
string	yes	in	Window title
string	yes	in	Startup folder
string	yes	in	Multiple file selection Possible values: "On", "Off" Default value: "Off"

Example:

Objective: Display a file browser titled "Reference object path" and store the user-selected value in the vRef_file_name output variable.

Command: `MACRO INPUT FILE_PATH (vRef_file_name, "Reference object path")`

No prompt is necessary as a file browser is displayed, letting you choose the appropriate file. On pressing the browser's **Open** button, the file chosen is stored in the variable.

- **MACRO INPUT INTEGER**

MACRO INPUT INTEGER (integer, string, array of strings, integer)			
Description : Pops up an interface to query the user for an integer value			
Arguments	Optional	In/Out	Description
integer	no	out	Integer value
string	yes	in	Window title
array of strings	yes	in	Window prompt
integer	yes	in	Suggested value

Example:

Objective: Display a window titled "Config info" with the prompt "Enter no. digits after point", and store the user-entered value in the vNbrDigits output variable.

Command: `MACRO INPUT INTEGER (vNbrDigits, "Config info", "Enter no. digits after point")`

- **MACRO INPUT PWK_OBJECTS**

MACRO INPUT PWK_OBJECTS (array of strings, string, string, string, array of strings)			
Description : Pops up an interface to query the user for workspace object names			
Arguments	Optional	In/Out	Description
array of strings	no	out	Workspace object names
string	yes	in	Workspace file name; if omitted, the current workspace is used
string	no	in	Object type Possible values: "IMAlign Project", "Polygonal Model", "PolyWorks Modeler Project", "PolyWorks Inspector Project", "IMTexture Input File", "3D Digitized Dataset"
string	yes	in	Window title
array of strings	yes	in	- obsolete -

- **MACRO INPUT STRING**

MACRO INPUT STRING (string, string, array of strings, string)			
Description : Pops up an interface to query the user for a string value			
Arguments	Optional	In/Out	Description
string	no	out	String value
string	yes	in	Window title
array of strings	yes	in	Window prompt
string	yes	in	Suggested value

Example:

Display a window titled "User info" with the prompt "Enter your name", and store the user-entered value in the vUsername output variable.

MACRO INPUT STRING (vUsername, "User info", "Enter your name")

- **MACRO INPUT DIALOG_BOX**

The MACRO INPUT DIALOG_BOX commands allow creating dialog boxes with multiple parameters. These commands are described in the tables below.

To use these commands, proceed as follows:

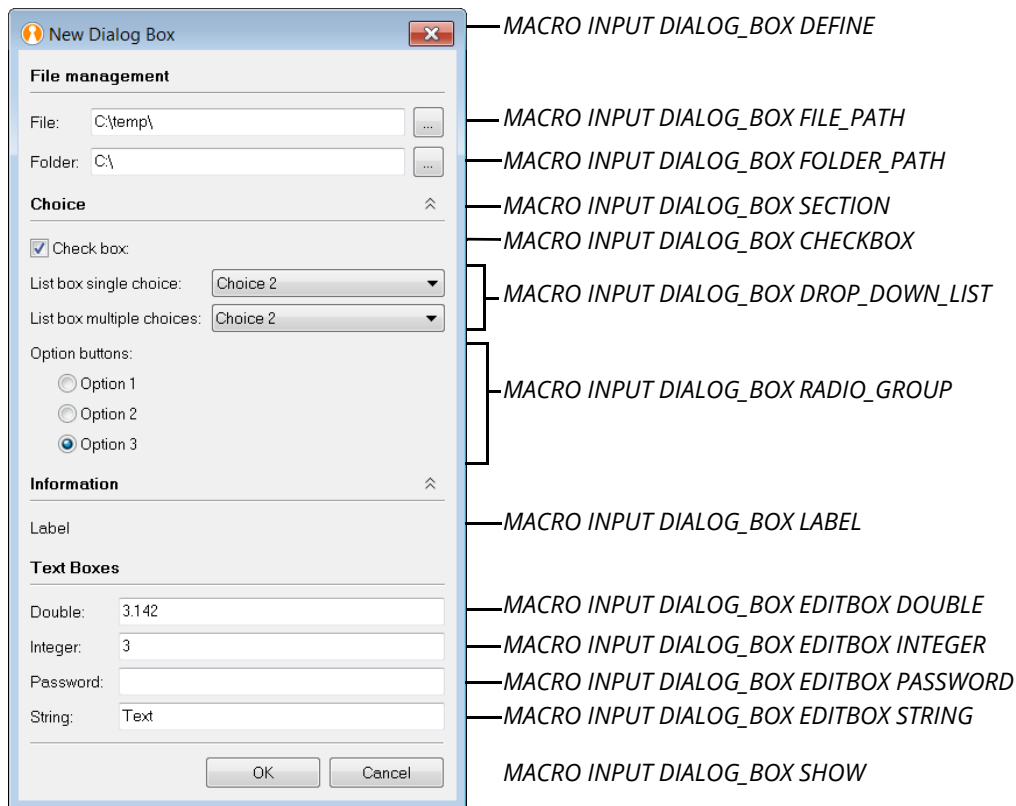
1. Define the dialog box and give it its title using the MACRO INPUT DIALOG_BOX DEFINE command.
2. Add the desired parameters to the dialog box using the corresponding MACRO INPUT DIALOG_BOX commands. Note that they will be displayed in the same order as they are entered.
3. Display the dialog box using the MACRO INPUT DIALOG_BOX SHOW command and collect the results from the user.

An example of such a dialog box with each type of parameter is shown in [Figure 7.4](#).

MACRO INPUT DIALOG_BOX DEFINE

MACRO INPUT DIALOG_BOX DEFINE (string, string)			
Description : Defines a dialog box			
Arguments	Optional	In/Out	Description
string	no	out	Window unique ID
string	no	in	Window title

Figure 7.4 Example of a dialog box with multiple parameters.



MACRO INPUT DIALOG_BOX CHECKBOX

MACRO INPUT DIALOG_BOX CHECKBOX (string, string, string)			
Description : Adds a check box to the dialog box			
Arguments	Optional	In/Out	Description
string	no	in	Window unique ID
string	no	in	Label of the check box
string	yes	in	Default value; if omitted, the default value will be "Off" Possible values: "On", "Off" Default value: "Off"

MACRO INPUT DIALOG_BOX DROP_DOWN_LIST

MACRO INPUT DIALOG_BOX DROP_DOWN_LIST (string, string, string, array of strings, array of strings)

Description :

Adds a drop-down list box to the dialog box

Arguments	Optional	In/Out	Description
string	no	in	Window unique ID
string	yes	in	Label of the drop-down list box
string	no	in	Multiple selection Possible values: "On", "Off"
array of strings	no	in	Values
array of strings	yes	in	Default values; if omitted and multi-selection is off, the first value will be selected

Note that both single-choice and multiple choice list boxes can be created.

MACRO INPUT DIALOG_BOX EDITBOX DOUBLE

MACRO INPUT DIALOG_BOX EDITBOX DOUBLE (string, string, double)

Description :

Adds a text box to the dialog box

Arguments	Optional	In/Out	Description
string	no	in	Window unique ID
string	yes	in	Label of the text box
double	yes	in	Default value; if omitted, the text box will be empty

Adds a text box to the dialog box for which valid values are double values.

MACRO INPUT DIALOG_BOX EDITBOX INTEGER

MACRO INPUT DIALOG_BOX EDITBOX INTEGER (string, string, integer)

Description :

Adds a text box to the dialog box

Arguments	Optional	In/Out	Description
string	no	in	Window unique ID
string	yes	in	Label of the text box
integer	yes	in	Default value; if omitted, the text box will be empty

Adds a text box to the dialog box for which valid values are integer values.

MACRO INPUT DIALOG_BOX EDITBOX PASSWORD

MACRO INPUT DIALOG_BOX EDITBOX PASSWORD (string, string)

Description :

Adds a password text box to the dialog box

Arguments	Optional	In/Out	Description
string	no	in	Window unique ID
string	yes	in	Label of the text box

MACRO INPUT DIALOG_BOX EDITBOX STRING

MACRO INPUT DIALOG_BOX EDITBOX STRING (string, string, string)			
Description : Adds a text box to the dialog box			
Arguments	Optional	In/Out	Description
string	no	in	Window unique ID
string	yes	in	Label of the text box
string	yes	in	Default value; if omitted, the text box will be empty

Adds a text box to the dialog box for which valid values are string values.

MACRO INPUT DIALOG_BOX FILE_PATH

MACRO INPUT DIALOG_BOX FILE_PATH (string, string, string, string, string, array of strings)			
Description : Adds a file path text box and browse button to the dialog box			
Arguments	Optional	In/Out	Description
string	no	in	Window unique ID
string	yes	in	Label of the text box
string	yes	in	Browser startup folder; if omitted, the default Windows folder
string	yes	in	Specifies whether the selection will be multiple Possible values: "On", "Off" Default value: "Off"
string	yes	in	Specifies which types of files will be visible in the browser; if omitted, all files Syntax: <Name of the filter>:<extensions>, where <extensions> is a semicolon-separated list of file extensions. Example: My Image Files: *.jpeg;*.jpg;*.tiff;*.gif
array of strings	yes	in	Default file(s); if omitted, the text box will be empty if multi-selection is off, only the first item will be displayed Example: {"c:\project\file1.txt", "c:\project\file2.txt"}

Note that both single file paths and multiple file paths can be created.

MACRO INPUT DIALOG_BOX FOLDER_PATH

MACRO INPUT DIALOG_BOX FOLDER_PATH (string, string, string, string)			
Description : Adds a folder path text box and browse button to the dialog box			
Arguments	Optional	In/Out	Description
string	no	in	Window unique ID
string	yes	in	Label of the text box
string	yes	in	Browser startup folder; if omitted, the default Windows folder
string	yes	in	Default folder; if omitted, the text box will be empty

MACRO INPUT DIALOG_BOX LABEL

MACRO INPUT DIALOG_BOX LABEL (string, string)			
Description : Adds a label to the dialog box			
Arguments	Optional	In/Out	Description
string	no	in	Window unique ID
string	no	in	Label text

MACRO INPUT DIALOG_BOX RADIO_GROUP

MACRO INPUT DIALOG_BOX RADIO_GROUP (string, string, array of strings, string)			
Description : Adds a radio button group to the dialog box			
Arguments	Optional	In/Out	Description
string	no	in	Window unique ID
string	yes	in	Label of the group
array of strings	no	in	Values
string	yes	in	Default value; if omitted, the first value will be selected

MACRO INPUT DIALOG_BOX SECTION

MACRO INPUT DIALOG_BOX SECTION (string, string, string, string)			
Description : Adds a section to the dialog box			
Arguments	Optional	In/Out	Description
string	no	in	Window unique ID
string	yes	in	Section title
string	yes	in	Specifies whether the section will be collapsible Possible values: "On", "Off" Default value: "Off"
string	yes	in	Specifies whether to align the controls with labels within the section Possible values: "On", "Off" Default value: "Off"

Note that the section can be expanded or collapsed.

MACRO INPUT DIALOG_BOX SHOW

MACRO INPUT DIALOG_BOX SHOW (string, string, string, integer, any type, ...)			
Description : Displays a dialog box to the user			
Arguments	Optional	In/Out	Description
string	no	in	Window unique ID
string	no	in	Label of the OK button
string	yes	in	Label of the Cancel button; if omitted, only the OK button will be visible
integer	no	out	Result, 1 if the OK button was used to close the dialog box, 0 otherwise
any type	yes	out	The number of output arguments must match the number of controls and the order in which they were added to the dialog box
...	yes	in	The previous argument may be repeated any number of times

- MACRO INPUT MULTIPLE_PARAMETERS**

MACRO INPUT MULTIPLE_PARAMETERS (string, array of strings, array of strings, any type, ...)			
Description : Pops up an interface to query the user for multiple parameters			
Arguments	Optional	In/Out	Description
string	yes	in	Window title
array of strings	yes	in	Window prompt
array of strings	no	in	Parameter descriptions Each parameter must have 3 consecutive entries in the array specifying: - Type ("double", "integer", or "string") - Label - Suggested value (" " if none)
any type	no	out	First parameter value
...	yes	in	The previous argument needs to be repeated to match the number of parameters

This command queries the user by means of an input window for one or more values of any type and assigns them to as many output variables. However, to do so, it is now recommended to use the MACRO INPUT DIALOG BOX command. For more information, refer to the MACRO INPUT DIALOG BOX command explained in the previous pages.

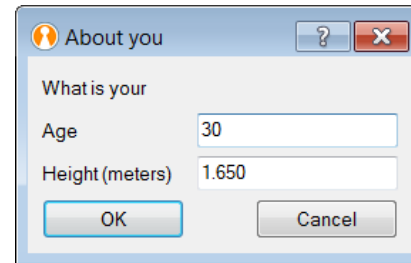
Example:

Objective: Query the user for his/her age and height (meters).

Command: DECLARE vq1 {"What is your"}
 # create an array of two parameters with three elements / parameter:
 # 1-type, 2-label, 3-default value
 DECLARE vA1 {"INTEGER", "Age", 30, "DOUBLE", "Height (meters)", 1.65}
 # declare as many output variables as there are parameters
 DECLARE vAge
 DECLARE vHeight

Query the user; the first argument is the optional window title
 MACRO INPUT MULTIPLE_PARAMETERS ("About you", \$vq1, \$vA1, vAge, vHeight)

On execution, these commands create and display the dialog box shown to the right. When values are entered and the **OK** button pressed, vAge will be assigned the first value and vHeight the second value.



7.3.3 Pausing macro scripts

The utility commands that follow allow you to pause or suspend the execution of the macro script.

- **MACRO PAUSE**

MACRO PAUSE (string, array of strings, ...)			
Description : Pops up a message window with an OK button			
Note : Popping up the window pauses the script, and pressing the OK button to dismiss the window resumes execution of the script			
Arguments	Optional	In/Out	Description
string	yes	in	Window title
array of strings	yes	in	Window prompt
...	yes	in	The previous argument may be repeated any number of times

Example:

Objective: Pause the macro script execution and display a window with the title *Pause window*, and two messages, *Consider the color map* and *Press the OK button to continue*.

Command: MACRO PAUSE ("Pause window", "Consider the color map", "Press the OK button to continue")

- **MACRO SLEEP**

MACRO SLEEP (integer)			
Description : Suspends the execution of a macro script for a specified period			
Argument	Optional	In/Out	Description
integer	no	in	Pause duration (milliseconds) Possible values: 0 or greater

Example:

Objective: Make the macro sleep for one second (i.e., 1000 milliseconds).

Command: MACRO SLEEP (1000)

7.3.4 Reading parameters from a text file

It may be convenient to store parameters that need to be passed to a macro in a text file. For example, if you need to process 20 datasets with different sets of parameters, an efficient approach would be to store the file names and parameters in a file, and make a loop in a macro to extract this information. Commands that read information from a text file are referred to as file-parsing commands.

Let's assume a file *c:\inspect_para.txt*, with three lines of six fields per line, which each contains three dataset names and three numeric values in the following format:

```
"dataset1" 11 "dataset2" 22 "dataset3" 33
"dataset4" 44 "dataset5" 55 "dataset6" 66
"dataset7" 77 "dataset8" 88 "dataset9" 99
```

Note that the default field delimiters are the *space* or the *TAB* character. The delimiter may also be user defined, but it then has to be specified in certain file-parsing commands. In addition, empty fields are supported in certain commands. See the second command that follows for examples.

The file-parsing commands that follow are located under the DATA_FILE branch of the macro command tree of the Command Reference. The variables must already have been declared.

```
# declaring variables
DECLARE vNbr_lines
DECLARE vNbr_fields
DECLARE vLine2
DECLARE vL2_F3
```

- **DATA_FILE PROPERTIES NB_LINES GET**

DATA_FILE PROPERTIES NB_LINES GET (string, integer)			
Description : Gets the number of lines in a file			
Arguments	Optional	In/Out	Description
string	no	in	File name
integer	no	out	Number of lines

Example:

Objective: Read the number of lines in the file c:\inspect.txt.

Command: DATA_FILE PROPERTIES NB_LINES GET ("c:\inspect.txt", vNbr_lines)

- **DATA_FILE PROPERTIES NB_FIELDS_IN_LINE GET**

DATA_FILE PROPERTIES NB_FIELDS_IN_LINE GET (string, integer, integer, string, string)			
Description : Gets the number of fields in a file line			
Arguments	Optional	In/Out	Description
string	no	in	File name
integer	no	in	Line number (must be greater than 0)
integer	no	out	Number of fields in the line
string	yes	in	Delimiters (space, tab (" \${_TAB}"), and new line (" \${_NEWLINE}") by default)
string	yes	in	Allow empty fields Possible values: "On", "Off" Default value: "Off"

Example 1:

Objective: Read the number of fields in the first line in the file c:\inspect_Ex1.txt. A default delimiter is used (i.e., the space or the TAB character), and empty fields are not allowed. The first line contains: Me You Us. The value returned to the output variable should be 3.

Command: DATA_FILE PROPERTIES NB_FIELDS_IN_LINE GET \ ("c:\inspect_Ex1.txt", 1, vNbr_fields)

Example 2:

Objective: Read the number of fields in the first line in the file c:\inspect_Ex2.txt. The delimiter symbol in the file is @. The first line contains: Me@You@Us. The value returned to the output variable should be 3.

Command: DATA_FILE PROPERTIES NB_FIELDS_IN_LINE GET \ ("c:\inspect_Ex2.txt", 1, vNbr_fields, "@")

Example 3:

Objective: Read the number of fields in the first line in the file c:\inspect_Ex3.txt. The delimiter symbol in the file is @. The first line contains: Me@@Us. If empty fields are allowed (i.e., the last argument is set to "On"), the value returned to the output variable is 3, otherwise it is 2.

Command: DATA_FILE PROPERTIES NB_FIELDS_IN_LINE GET \
("c:\inspect_Ex3.txt", 1, vNbr_fields, "@", "On")

Note the following cases when allowing empty fields:

Line in file	Value returned – Number of fields
"abc,"	3
"abc" and "abc,"	both return 2
"" ,	2
""	error (same result when allowing empty fields is Off)

- **DATA_FILE READ LINE**

DATA_FILE READ LINE (string, integer, string)			
Description : Gets a line from a file			
Arguments	Optional	In/Out	Description
string	no	in	File name
integer	no	in	Line number (must be greater than 0)
string	no	out	Line

Example:

Objective: Read the second line in the file c:\inspect_para.txt.

Command: DATA_FILE READ LINE ("c:\inspect_para.txt", 2, vLine2)

- **DATA_FILE READ LINE_FIELD**

DATA_FILE READ LINE_FIELD (string, integer, integer, any type, string, string)			
Description : Gets a field from a file line			
Arguments	Optional	In/Out	Description
string	no	in	File name
integer	no	in	Line number (must be greater than 0)
integer	no	in	Field number (must be greater than 0)
any type	no	out	Field
string	yes	in	Delimiters (space, tab (" \${_TAB}"), and new line (" \${_NEWLINE}") by default)
string	yes	in	Allow empty fields Possible values: "On", "Off" Default value: "Off"

Example:

Objective: In the file c:\inspect_para.txt, read the third field on the second line.

Command: DATA_FILE READ LINE_FIELD ("c:\inspect_para.txt", 2, 3, vL2_F3)

7.3.5 Working with string values

The MACRO STRING group of commands provides standard string processing operations, such as replacing a series of characters in the string with a different series of characters. Each command is presented below. An example using each command is provided in [Figure 7.5](#).

- **MACRO STRING FIND_FIRST**

MACRO STRING FIND_FIRST (string, string, integer, integer)			
Description : Returns the index of the first occurrence of a given sequence in a string			
Arguments	Optional	In/Out	Description
string	no	in	Input string
string	no	in	String to search
integer	yes	in	Start index Possible values: 1 or greater Default value: 1
integer	no	out	Index of the first occurrence, or 0 if not found

- **MACRO STRING FIND_LAST**

MACRO STRING FIND_LAST (string, string, integer, integer)			
Description : Returns the index of the last occurrence of a given sequence in a string			
Arguments	Optional	In/Out	Description
string	no	in	Input string
string	no	in	String to search
integer	yes	in	Maximum index Possible values: 1 or greater
integer	no	out	Index of the last occurrence, or 0 if not found

- **MACRO STRING FORMAT**

MACRO STRING FORMAT (string, string, any type, ...)			
Description : Converts the value of variables to characters according to the specified format and returns the result as a new string			
Arguments	Optional	In/Out	Description
string	no	out	Formatted string
string	no	in	Format specification
any type	yes	in	The number of input arguments must match the number of format items and the order in which they were added to the format string
...	yes		The previous argument may be repeated any number of times

Using %s for strings

Example:

Objective: Specify the minimum width of a string with space padding on the left (for example, **%10s**) or on the right (for example, **%-10s**). The minimum width of the string is specified by the number before the "s".

Command: DECLARE resultStr
 DECLARE name "Joe"
 MACRO STRING FORMAT (resultStr , "[%10s]\${_NEWLINE}[%-10s]",
 \$name, \$name)
 # Result:
 # [Joe]
 # [Joe]

For the above example, both results contain ten characters, including the word "Joe". In the first result, the space padding is on the left, while the space padding is on the right for the second result.

Using %d for an integer number

Example 1:

Objective: Specify the minimum width of a number with space padding (for example, **%5d**). The minimum width is specified by the number between the "%" and the "d" letter.

Command: DECLARE resultStr
DECLARE angle 180
MACRO STRING FORMAT (resultStr,"Angle:%5d", \$angle)
Result: "Angle: 180"

For the above example, the result contains five characters, including the number "180". There are two spaces before the number.

Example 2:

Objective: Specify the minimum width of a number with zero padding (for example, **%05d**). The minimum width is specified by the number between the "%0" and the "d" letter.

Command: DECLARE resultStr
DECLARE angle 180
MACRO STRING FORMAT (resultStr,"Angle:%05d", \$angle)
Result: "Angle:00180"

For the above example, the result contains five characters, including the number " 180 ". There are two zeros before the number.

Using %f for float numbers

Example 1:

Objective: Specify the minimum width of a number with space padding (for example, **%9.4f**). The minimum width is specified by the number between the "%" and the ".", while the number between the "." and the "f" truncates the decimal part up to the given number.

Command: DECLARE resultStr
DECLARE distance 246.1214151627
MACRO STRING FORMAT (resultStr, "Distance:%9.4f", \$distance)
Result: "Distance: 246.1214"

For the above example, the decimal is truncated to four numbers. To get a result that contains nine numbers, a space is added at the beginning.

Example 2:

Objective: Specify the minimum width of a number with zero padding (for example, **%019.4f**). The minimum width is specified by the number between the

"%0" and the ".", while the number between the "." and the "f" truncates the decimal part up to the given number.

Command: `MACRO STRING FORMAT ( resultStr, "Distance:%019.4f", $distance )`
 Result: "Distance:00000000000246.1214"

For the above example, the decimal is truncated to four numbers. To get a result that contains nineteen numbers, a zero is added at the beginning.

Example with multiple formats %s, %d, %f

Example:

Command: `DECLARE resultStr`
`DECLARE name "Joe"`
`DECLARE angle 180`
`DECLARE distance 246.1214151627`
`MACRO STRING FORMAT ( resultStr, "Name:%s, Distance:%f,`
`Angle:%d", $name, $distance, $angle )`
 # Result: "Name:Joe, Distance:246.1214, Angle:180"

- **MACRO STRING LENGTH**

MACRO STRING LENGTH GET (string, integer)			
Description :			
Returns the number of characters in a string			
Arguments	Optional	In/Out	Description
string	no	in	Input string
integer	no	out	Number of characters Possible values: 0 or greater

- **MACRO STRING REPLACE**

MACRO STRING REPLACE (string, string, string, integer, integer, string)			
Description :			
Replaces a specified part of a string with another string			
Arguments	Optional	In/Out	Description
string	no	in	Input string
string	no	in	String to search
string	no	in	Replacement string
integer	yes	in	Start index Possible values: 1 or greater Default value: 1
integer	yes	in	Maximum number of replacements; if omitted, all occurrences are replaced Possible values: 1 or greater
string	no	out	Output string

- **MACRO STRING SPLIT**

MACRO STRING SPLIT (string, string, array of strings)			
Description : Splits a string into multiple substrings using delimiters			
Arguments	Optional	In/Out	Description
string	no	in	Input string
string	no	in	Delimiter characters
array of strings	no	out	Output strings

- **MACRO STRING SUBSTRING**

MACRO STRING SUBSTRING (string, integer, integer, string)			
Description : Returns a substring with the specified number of characters			
Arguments	Optional	In/Out	Description
string	no	in	Input string
integer	yes	in	Start index Possible values: 1 or greater Default value: 1
integer	yes	in	Number of characters; if omitted, all the characters from start index are returned Possible values: 0 or greater
string	no	out	Output substring

- **MACRO STRING TRIM_SPACES**

MACRO STRING TRIM_SPACES (string, string)			
Description : Removes the spaces on the left and the right side of a string			
Arguments	Optional	In/Out	Description
string	no	in	Input string
string	no	out	Output string

7.3.6 Using shared variables

The scripting language, and not the Macro Script Control Language, offers shared variables. A shared variable can be created, read, modified, and deleted in any macro script (e.g., PolyWorks|Inspector, PolyWorks|Modeler, IMAlign, Workspace Manager). If a shared variable is not deleted explicitly, it is available until the Workspace Manager is closed.

Figure 7.5 An example of a macro script that uses the MACRO STRING group of commands. The string being processed contains repeated terms, and it begins and ends with a space. The macro script execution is shown in the Command History pane. The distinctive part of each command has been highlighted.

The screenshot shows the Macro Script Editor interface. The top pane displays a macro script for 'StringTests.pwmacro'. The script defines a variable 'vMyString' with the value ' big big white cat ' and then uses several MACRO STRING commands to process it. The bottom pane, titled 'Command History', shows the execution of these commands, with the resulting output of each command displayed below the command text.

```

Macro Script Editor - [StringTests.pwmacro (PolyWorks 2017.pwk)]
File Edit View Debug Tools Window Help

1 version "5.0"
2 #-----
8 Declare vMyString " big big white cat "
9 Declare vOutput
10 MACRO STRING LENGTH GET ( $vMyString, vOutput )
11 MACRO ECHO ( $vOutput )
12 MACRO STRING SUBSTRING ( $vMyString,1,4,vOutput )
13 MACRO ECHO ( $vOutput )
14 MACRO STRING TRIM SPACES ( $vMyString,vOutput )
15 MACRO ECHO ( $vOutput )
16 MACRO STRING REPLACE ( $vMyString, "big", "bold", , ,vOutput )
17 MACRO ECHO ( $vOutput )
18 MACRO STRING FIND_FIRST ( $vMyString,"big",,vOutput)
19 MACRO ECHO ( $vOutput )
20 MACRO STRING FIND_LAST ( $vMyString,"big",,vOutput)
21 MACRO ECHO ( $vOutput )
22 MACRO STRING SPLIT ( $vMyString," ",vOutput)
23 MACRO ECHO ( $vOutput )

Ready Line 2 100%

Command History
#-----
# Running macro script: StringTests.pwmacro
MACRO STRING LENGTH GET ( " big big white cat ", 19 )
19
MACRO STRING SUBSTRING ( " big big white cat ", 1, 4, " big" )
big
MACRO STRING TRIM SPACES ( " big big white cat ", "big big white cat" )
big big white cat
MACRO STRING REPLACE ( " big big white cat ", "big", "bold", 1, , " bold bold
white cat " )
bold bold white cat
MACRO STRING FIND_FIRST ( " big big white cat ", "big", 1, 2 )
2
MACRO STRING FIND_LAST ( " big big white cat ", "big", , 6 )
6
MACRO STRING SPLIT ( " big big white cat ", " ", { "big", "big", "white", "cat" } )
big
big
white
cat
# End of macro script: StringTests.pwmacro
#-----

```

Such variables are useful for sharing information between different scripts, such as the company name. Note that shared variables are useful and dangerous at the same time, as they are also available to any other script that may change its value.

The table that follows lists the related commands; for their syntax, see the Command Reference of any Macro Script Editor tool.

MACRO_SHARED_VARIABLE...	Description
CREATE	Creates a shared variable.
DELETE	Deletes a shared variable.
GET_VALUE	Gets the value of a shared variable and stores it in an output variable.
SET_VALUE	Assigns a new value to a shared variable.

Here are some good practices:

- Avoid simple variable names, such as *name* or *number*, that may be used by other scripts. You can personalize your shared variables by adding a prefix to your shared variable names (ex. *ab_name*, where *ab* is the project in question or even a number).
- Check, at the time of creating a shared variable, if the command was successful – the command fails if a shared variable of the same name already exists.
- Delete shared variables when you are finished with them.

A shared variable is used in the macro script sample provided in [Section 9.5 A multiapplication script](#).

Basic Programming Concepts

A decorative graphic on the right side of the page. It features a large, dark blue circle containing the white number '8'. Below this circle, a white line resembling a circuit board trace extends downwards, then turns left and continues horizontally across the page. Several small white circles, also resembling circuit components, are placed along this trace. The background is a light gray with a subtle gradient.

8

This chapter presents basic concepts that help users to write more robust macro scripts more efficiently.

8.1 Introduction

This chapter presents basic programming practices that will make debugging easier, save you time, and lead you to making smaller, specialized scripts that become tools that are available from any other script.

Each subject is presented in the subsections that follow.

8.2 Modular programming

As you become more experienced with macro scripts, you will identify groups of commands that are needed on a regular basis. It is more efficient then to create short macro scripts and to execute them from larger, task-oriented macro scripts. Any changes made to such small macro scripts are immediately available to all the macro scripts that make use of them. The analogy would be to create many tools that are ready for use to solve many types of problems. The MACRO EXEC command, inserted in a macro script, permits the execution of another macro script.

MACRO EXEC (string, any type, ...)			
Description : Executes the specified macro script			
Note : Use MACRO OUTPUT_ARGUMENT in the specified macro script to set the value of an output argument passed to the macro script			
Arguments	Optional	In/Out	Description
string	no	in	Macro script file name
any type	yes	in	Argument passed to the macro script
...	yes	in	The previous argument may be repeated any number of times

8.2.1 An example of modular programming

The example that follows illustrates this divide and conquer approach, common to programming in general. A first macro script is provided – within a project, it aligns a Data and a Reference object, makes a comparison, and generates an inspection report. Parts of the macro script are then used to make separate, short macro scripts. Finally, a second macro script is proposed that makes calls to four smaller macro scripts, using the MACRO EXEC command.

The first macro script contains all the commands.

```

DECLARE vProject
DECLARE vReport
MACRO INPUT DIRECTORY_PATH (vProject, "Select a project to open")
MACRO INPUT DIRECTORY_PATH (vReport, "Select a report export folder")
# open the project
FILE OPEN_PROJECT_IN_PWK ( , $vProject)
# perform a rough alignment
TREEVIEW SELECT NONE
TREEVIEW DATA SELECT (1, "On")
ALIGN POINT_PAIRS
# perform a Best-Fit alignment
ALIGN BEST_FIT DATA_TO_REFERENCE CREATE \
    FIT_TO_REFERENCE_OBJECT_SURFACES
# perform a comparison
TREEVIEW REFERENCE SELECT (1, "On")
MEASURE DATA_COLOR_MAP REFERENCE_SURFACE CREATE
# export the report
FILE EXPORT_REPORT HTML_FILE ($vReport)

```

A second, modular version of the first macro script follows. Its uses MACRO EXEC calls to short, mono-task, macro scripts. As explained previously, the first argument of the MACRO EXEC command is the name of the macro script to execute; arguments required by the named macro script follow, separated by commas.

```

DECLARE vProject
DECLARE vReport
MACRO INPUT DIRECTORY_PATH (vProject, , "Select a project to open")
MACRO INPUT DIRECTORY_PATH (vReport, , "Select a report export folder")
MACRO EXEC ("OpenProject", $vProject)
MACRO EXEC ("RoughAlignment")
MACRO EXEC ("BestFitAlignment")
MACRO EXEC ("CompareDataToRef")
MACRO EXEC ("ExportReport", $vReport)
# end of example

```

Here are the macro scripts called by MACRO EXEC:

```

# OpenProject
FILE OPEN_PROJECT_IN_PWK ( , $1)

# RoughAlignment
TREEVIEW SELECT NONE
TREEVIEW DATA SELECT (1, "On")
ALIGN POINT_PAIRS

# BestFitAlignment
ALIGN BEST_FIT DATA_TO_REFERENCE CREATE \

```

```
FIT_TO_REFERENCE_OBJECT_SURFACES
```

```
# CompareDataToRef
```

```
TREEVIEW REFERENCE SELECT (1, "On")
```

```
MEASURE DATA_COLOR_MAP REFERENCE_SURFACE CREATE
```

```
# ExportReport
```

```
FILE EXPORT_REPORT HTML_FILE ($1)
```

8.2.2 Introducing macro script variables

When a macro script is called by the MACRO EXEC command, macro script variables are automatically declared and are automatically assigned values that the user can access, but cannot change (except for output arguments). The table that follows describes the macro script variables.

Variable	Type	Description
\$#	integer	The number of arguments sent to a macro script by way of the MACRO EXEC command.
\$0	string	The name of the macro script. This is the <u>first</u> argument of the MACRO EXEC command. \$0 is the <i>only required argument</i> .
\$1 to \$n	same type as the argument	The arguments that follow the name of the macro script in the MACRO EXEC command. \$1 corresponds to the <u>second</u> argument, \$2 corresponds to the <u>third</u> argument, and so on. Note that output arguments can be assigned a value in the script called by the MACRO EXEC command; for more information, see Section 8.2.3 Using output arguments.

The examples that follow provide a complete review of using macro script variables.

The macro script Macro_1 contains a MACRO EXEC command that calls Macro_2. Macro_2 contains two MACRO ECHO commands that display the content of the variables \$0 and \$#, which are declared and initialized when the MACRO EXEC command is executed. \$0 is assigned the name of the macro script, Macro_2, and \$# is assigned the number of arguments passed to the MACRO EXEC command, 1.

```
# Macro_1
```

```
MACRO EXEC ("Macro_2")
```

```
# a reminder:      $0
```

```
# Macro_2
```

```
MACRO ECHO (" $0 is running right now.")
```

```
MACRO ECHO (" $0 has received $# arguments.")
```

The execution of Macro_1 will display two strings:

```
Macro_2 is running right now.
Macro_2 has received 1 arguments.
```

A string and an integer argument are added to the example as additional arguments of the MACRO EXEC command.

```
# Macro_1
MACRO EXEC ("Macro_2", "Bobby", 33)
# a reminder:      $0      $1      $2

# Macro_2
MACRO ECHO ("$0 is running right now.")
MACRO ECHO ("$0 has received $# arguments.")
MACRO ECHO ("$1 is $2 years old.")
```

The execution of Macro_1 will display three strings:

```
Macro_2 is running right now.
Macro_2 has received 3 arguments.
Bobby is 33 years old.
```

The next example illustrates that the value integer 0 is assigned to a macro script variable when the user does not enter a specific value as an argument, as is the case for the variable \$1. However, the user must always specify the name of the macro script to be executed, hence the value of \$0.

```
# Macro_1
# the only required argument is the name of the macro script
# to be executed.
MACRO EXEC ("Macro_2", , 33)
# a reminder:      $0      $1      $2

# Macro_2
MACRO ECHO ("$0 is running right now.")
MACRO ECHO ("$0 has received $# arguments.")
MACRO ECHO ("$1 is $2 years old.")
```

The execution of Macro_1 will result in the display of:

```
Macro_2 is running right now.
Macro_2 has received 3 arguments.
0 is 33 years old.
```

Note that the last line displayed contains a 0 (*0 is 33 years old*). The \$1 macro script variable has been assigned the default value, integer 0.

A last example illustrates how the execution of the instructions in a macro script, in this case the MACRO ECHO commands in Macro_2, can be preempted by testing if the right number of arguments have been passed to the calling MACRO EXEC command, in Macro_1, or not.

```
# Macro_1
MACRO ECHO ("First call:")
MACRO EXEC ("Macro_2")
# a reminder:      $0
MACRO ECHO ("Second call:")
MACRO EXEC ("Macro_2", "Bobby", 33)
# a reminder      $0      $1      $2

# Macro_2
IF $# < 3
    MACRO ECHO ("Not enough arguments.")
    MACRO END
ENDIF
MACRO ECHO ("$0 is running right now.")
MACRO ECHO ("$0 has received $# arguments.")
MACRO ECHO ("$1 is $2 years old.")
```

The execution of Macro_1 will result in the display of:

```
First call:
Not enough arguments.
Second call:
Macro_2 is running right now.
Macro_2 has received 3 arguments.
Bobby is 33 years old.
```

Finally, it is possible to use an array as an argument of the MACRO EXEC command.

```
DECLARE vPointsArray {34.5, 100.2, 20.8}
MACRO EXEC ("MyFavoriteMacroScript", $vPointsArray)
# the array vPointsArray has been assigned to the macro script variable $1,
# which is now an array identical to vPointsArray
```

8.2.3 Using output arguments

Output arguments, presented in [Section 6.4.5 Using variables as input and output arguments for commands](#), can be used when calling a macro script using the MACRO EXEC command.

In this case, the MACRO OUTPUT_ARGUMENT command can be used in the macro script that is called to assign a value to the output argument.

Here is a simple example. The following commands are contained in a first macro script:

```
# Macro1.pwmacro
# The MACRO EXEC command contains an output argument
DECLARE vAnyName
MACRO EXEC ("Macro2.pwmacro", vAnyName)
MACRO ECHO ($vAnyName)
```

The following command is contained in the script being called:

```
# Macro2.pwmacro
# A value is assigned to the output argument, identified using its argument index
MACRO OUTPUT_ARGUMENT (1,"How about Ringo?")
```

The execution of Macro1.pwmacro will result in the display of:

How about Ringo?

8.3 Error handling

The execution of a macro script can be controlled with a simple error-handling mechanism. It may be useful to react to the failure of a command or to end the execution of a macro script.

The execution of a command can succeed or fail. This result is called the *error status*. It can be obtained with the `MACRO GET_ERROR_STATUS` command. Once the error status of a command is known, a decision can be made as to the actions to take in case of failure. The exception to this rule is the `MACRO EXEC` command. The error status of this command is that of the last command to be executed in the macro script.

Three commands can be used to do error handling.

- **MACRO GET_ERROR_STATUS** (string variable)

This command gets the error status (i.e., *No Error* or *Error*) of the last statement that was executed and assigns it to the specified out string variable. In earlier versions of the scripting language, version 3 and below, this command returned the status of the last executed command. As a result, a script written using an earlier version and containing this command is not fully forward compatible with the version 4 or later of the scripting language.

Statements include:

- Assignations (`DECLARE`, `SET`, `++`, `AND --`)
- Commands (application commands, other commands)
- Conditions (`IF` and `WHILE`)
- Loop modifiers (`BREAK` and `CONTINUE`)

Note that the `++` and `--` statements cannot fail. If the variable to increment or decrement is not a number, an implicit conversion is carried out. If the conversion fails, integer 0 is obtained and then incremented or decremented.

- **MACRO END** (string)

Stops the execution of the current macro script. It can be placed in a main macro script, in which case the macro script execution terminates. If it is placed in a nested macro script called from a main macro script by way of the `MACRO EXEC` command, the nested macro script would terminate and the control would return to the main macro script from which it was called.

The argument *string* is optional. You can specify "No Error" or "Error". The default value is "No Error". The command `MACRO END` ends the execution of a macro script and the argument specifies the error status of the macro script at that moment.

- **MACRO END_ON_ERROR** (string)

This command is used to activate/deactivate the automatic error-handling mode. Two values can be specified for the optional argument: "On" or "Off". "On" is the default value.

When the *end on error* mode is activated, the execution of a macro script is immediately terminated when the error status of the most recently executed statement is *Error*. It is as if the following commands were executed after each statement in the macro script (let's assume that the variable `vStatus` has already been declared):

```
MACRO GET_ERROR_STATUS (vStatus)
  IF $vStatus== "Error"
    MACRO END ("Error")
  ENDIF
```

The *end on error* mode is specific to each macro script. If you activate this mode in one macro script, it has no influence on the mode in a parent or a child macro script.

Examples:

The following macro script uses the `MACRO GET_ERROR_STATUS` command to open a valid PolyWorks|Inspector project in the active workspace. If an error occurs, the open operation is repeated:

```
# set vStatus to "Error" to run the WHILE loop body at least once
DECLARE vStatus "Error"
DECLARE vProject
# as long as FILE OPEN_PROJECT_IN_PWK fails, retry the command
WHILE $vStatus == "Error"
  MACRO INPUT DIRECTORY_PATH (vProject, "Specify the desired folder",)
  FILE OPEN_PROJECT_IN_PWK ( , $vProject)
  MACRO GET_ERROR_STATUS (vStatus)
ENDWHILE
#...proceed with the valid PolyWorks|Inspector project
```

The following example uses the `MACRO END` command to end a macro script that has not received all the necessary arguments:

```
IF $# <= 2
  MACRO PAUSE ("Error", "Too few arguments for this macro script.", \
    "Expected 2 arguments; got $#.", "leaving macro script.")
  MACRO END ("Error")
ENDIF
```

The following example uses the MACRO END_ON_ERROR command to avoid executing commands in a macro script when one of its statements has already failed (error status is *Error*):

```
# exit as soon as an error occurs
MACRO END_ON_ERROR ("On")
# no test for Macro Script arguments $1 and $2 since MACRO END_ON_ERROR
# is activated; if an error occurs, the execution of the macro script is terminated
FILE OPEN_PROJECT_IN_PWK ($1, $2)
# transfer execution to Process, a useful macro script that could fail
MACRO EXEC ("Process")
# if Process exits with an "Error" error status, the next macro script will not be
# executed, since execution is ended as soon as an error occurs; this command
# could prevent executing commands unnecessarily
MACRO EXEC ("ExtraLongProcess")
```

Sample Macro Scripts

9

This chapter presents a macro script for each main module that accomplishes a specific task as well as a multiapplication macro script.

9.1 Introduction

This chapter presents a macro script that performs a described task for each of the IMAlign, the PolyWorks|Modeler, and the PolyWorks|Inspector modules.

9.2 IMAlign macro script

This simple IMAlign macro script can be used immediately after a Point Pairs alignment. Usually, you have to follow this alignment with a best-fit alignment, and then lock the just-aligned 3D Image in the tree view. The following macro script carries out those two steps for you.

```
DECLARE vConv
# read the Convergence factor in the alignment dialog box
ALIGN OPTIONS CONVERGENCE GET (vConv)
# start the best-fit alignment using the Convergence factor
ALIGN START ($vConv,-1)
# lock the selected 3D images in the tree view
EDIT IMAGE LOCK ( )
```

9.3 PolyWorks|Modeler macro script

In models built from 3D digitizer data, it is not uncommon to find several isolated islands of triangles. These islands are usually undesirable and difficult to find and delete. The purpose of this macro is to delete all the connected sets of triangles that contain less than a user-specified number of triangles.

The first IF-ENDIF determines the minimum number of triangles for a connected set of triangles. This number of triangles may or may not be specified by the user. The IF \$1 command is true only if an argument was passed to the macro. In this case, the first argument's value is copied to vNbTriMin. Otherwise, vNbTriMin keeps its value of 20 assigned at declaration.

```
DECLARE vNbLevels
DECLARE vNbCurLevel
DECLARE vNbTriForLevel
DECLARE vNbTriMin 20
IF $1
    SET vNbTriMin $1
ENDIF
#
# Find all connected sets of triangles and select the triangles that belong to these
# objects. Each connected set, or object, occupies one selection stack level.
```

```
SELECT TRIANGLES ALL_OBJECTS
#
# Copy the number of selection stack levels to the variable vNbLevels. This number
# also represents the number of objects.
SELECT ELEMENTS LEVEL COUNT GET (vNbLevels)
SET vNbCurLevel $vNbLevels
#
# Unmark all selection stack levels.
SELECT ELEMENTS LEVEL MARK NONE
#
# Outputs the number of objects to the screen.
MACRO ECHO ("{$vNbLevels} objects have been found...")
# Process each selection stack level (loop). If the number of triangles in the object
# for a particular level is smaller than vNbTriMin, the level is preserved.
# Otherwise, the level is marked using the SELECT LEVEL MARK command.
WHILE $vNbCurLevel > 0
    SELECT ELEMENTS LEVEL PROPERTIES NB_ELEMENTS \
        GET ({vNbCurLevel}, vNbTriForLevel)
    IF $vNbTriForLevel >= $vNbTriMin
        SELECT ELEMENTS LEVEL MARK ($vNbCurLevel)
    ENDIF
    --vNbCurLevel
# Display a message every time 1000 objects have been processed.
    DECLARE nbProcessedObjects EXPR_I ($vNbLevels - $vNbCurLevel)
    DECLARE modulo EXPR_I ($nbProcessedObjects % 1000)
    IF $modulo == 0
        MACRO ECHO ("{$nbProcessedObjects} objects processed")
    ENDIF
ENDWHILE

# Deselect all marked selection stack levels.
SELECT ELEMENTS LEVEL DESELECT_MARKED

# Following this operation, the only objects that are still selected are those that
# contain less than vNbTriMin triangles. Now delete all selected triangles.
SELECT ELEMENTS LEVEL COUNT GET (vNbLevels)
IF $vNbLevels > 0
    EDIT OBJECT DELETE_SELECTED_ELEMENTS
ENDIF
MACRO ECHO ("===== Completed =====")
# End of script
```

9.4 PolyWorks|Inspector macro scripts

Two PolyWorks|Inspector macro scripts are presented, each one demonstrating different functions.

9.4.1 Macro script with an alignment and a data color map

The following example automates the alignment of three point clouds, and then generates a data color map using the Reference object in the project. The first point cloud is called *pointcloud1*. Each point cloud must be imported in a different alignment group.

This macro executes an automatic alignment of three different point clouds. The original position of the three point clouds is approximately the same. The rough alignment of the first point cloud (scan) is done by a manual operation, then the two other scans are aligned automatically, using the first point cloud's global alignment matrix. Note that the three point clouds initially have the "ignored" status in the project. One by one, the objects are assigned the "used" status, are aligned, and then are reassigned the "ignored" status.

```

DECLARE vDatanb 1
WHILE $vDatanb <= 3
    # rough alignment of the first point cloud
    TREEVIEW OBJECT SELECT NONE
    TREEVIEW DATA SELECT ($vDatanb, "On")
    EDIT OBJECT USE
    IF $vDatanb == 1
        # rough alignment of the first scan
        ALIGN POINT_PAIRS
        ALIGN BEST_FIT DATA_TO_REFERENCE CREATE \
            FIT_TO_REFERENCE_OBJECT_SURFACES
    ELSE
        # rough alignment of the second and third point cloud
        TREEVIEW OBJECT SELECT NONE
        TREEVIEW DATA SELECT ($vDatanb, "On")
        ALIGN TRANSFORM_USING_MATRIX FROM_OBJECT \
            ("pointcloud1", "Global", "Off", )
        ALIGN BEST_FIT DATA_TO_REFERENCE CREATE \
            FIT_TO_REFERENCE_OBJECT_SURFACES
    ENDIF
    # generation of a color map of the deviations
    VIEW POSE CENTER
    VIEW POSE Z_POS
    MEASURE DATA_COLOR_MAP REFERENCE_SURFACE CREATE
    REPORT_ITEM SNAPSHOT CAPTURE
    # set the vStatus of the selected point cloud to ignored
    TREEVIEW OBJECT SELECT NONE
    TREEVIEW DATA SELECT ($vDatanb, "On")

```

```

EDIT OBJECT IGNORE
++ vDatanb
ENDWHILE

```

9.4.2 Macro script with an array variable

The following macro script displays the center of mass coordinate derived from the measured point primitives in the open PolyWorks|Inspector project. A point named *CM* will be created in the project to visualize more easily the center of mass coordinates.

```

DECLARE vNbpts
# check that there are points in the project; if not, end the macro
# get the number of points to process
TREEVIEW FEATURE POINT COUNT GET(vNbpts)
If $vNbpts == 0
    MACRO ECHO ("No points found...leaving.")
    MACRO END ("No Error")
ENDIF
TREEVIEW OBJECT SELECT NONE
# select all measured point primitives and make them visible
TREEVIEW PRIMITIVE SELECT MEASURED
TREEVIEW OBJECT VIEW KEEP
# define variables:
# vNbptsmeasured: holds the number of point features with a measured primitive
# vStatus: holds status of last point feature measured primitive selection
# vNbpts: holds the number of point features in the project
# vPtArray: is an array that will hold the current point's X, Y, and Z values
# vCmArray: is an array that will hold the center of mass coordinates
# vNb: is a counter in a loop
DECLARE vNbptsmeasured 0
DECLARE vStatus
DECLARE vPtArray
DECLARE vCmArray
DECLARE vNb 1
# accumulates in vCmArray[1], vCmArray[2] and vCmArray[3] the sum
# of the X, Y, and Z coordinates
WHILE $vNb <= $vNbpts
    # get the X, Y, and Z values of the point
    TREEVIEW OBJECT SELECT NONE
    # select measured primitive by feature index
    TREEVIEW FEATURE POINT MEASURED SELECT($vNb, "On")
    # validate selection - the feature may have no measured primitive
    MACRO GET_ERROR_STATUS (vStatus)
    IF $vStatus == "No Error"
        # get the X, Y, Z coordinates of the selected point
        TREEVIEW PRIMITIVE POINT PROPERTIES POINT \
            GET(vPtArray[1], vPtArray[2], vPtArray[3])
    
```

```
# X coordinate sum in first array element
SET vCmArray[1] EXPR ($vCmArray[1] + $vPtArray[1])
# Y coordinate sum in second array element
SET vCmArray[2] EXPR ($vCmArray[2] + $vPtArray[2])
# Z coordinate sum in third array element
SET vCmArray[3] EXPR ($vCmArray[3] + $vPtArray[3])
++ vNbptsmeasured
ENDIF
++ vNb
ENDWHILE
# presently, the array vCmArray contains three sums, one for each component
# divide each sum by vNbptsmeasured to obtain the center of mass
SET vCmArray {EXPR ($vCmArray[1] / $vNbptsmeasured), \
  EXPR ($vCmArray[2] / $vNbptsmeasured), \
  EXPR ($vCmArray[3] / $vNbptsmeasured)}
# display the center of mass
MACRO ECHO ("The center of mass is ($vCmArray[1], $vCmArray[2], \
  $vCmArray[3])")
# create a measured point primitive at the center of mass having the name CM
FEATURE PRIMITIVE POINT CREATE($vCmArray[1], $vCmArray[2], \
  $vCmArray[3], "Measured", "CM")
# end of script
```


9.5 A multiapplication script

A task may require performing operations in more than one module. The Macro Script Editor tool of the Workspace Manager has specialized commands that enable you to create scripts that perform the execution of tasks in a required set of modules.

Basic commands are available that let you perform tasks in the IMAlign, the PolyWorks|Modeler, and the PolyWorks|Inspector modules:

- Open a module. Each instance of a module is identified by a unique ID number.
- Execute remote commands in those modules.
- Execute scripts written for those modules (does not apply to PolyWorks|Reviewer).
- Save the open projects and close the modules.

Concerning the IMMerge module, a command is available that runs the module using the specified parameters.

Considered together, these commands allow you to script a complete PolyWorks process.

The subsections that follow describe an imaginary reverse-engineering task to perform and provide the scripts that would realize the task.

9.5.1 The tasks to accomplish in each module

The example that follows respects a possible workflow in the reverse-engineering domain: align scans in IMAlign, merge the project in IMMerge, edit the output mesh in PolyWorks|Modeler, and export the edited polygonal model.

The named workspace is: C:\Multi-applicationScript.pwk.

It contains two projects: *IMBlock_Aligned* and *Empty_Modeler_Project*.

As the example demonstrates how to process in one module and then use the output in a downstream module, the actual tasks executed are purposefully simple. A more complex task could be executed using a similar structure.

The user wants to perform the following tasks:

Step A. To do in IMAlign:

1. Open an instance of the IMAlign module.
2. Set a configuration parameter.
3. Run a script in IMAlign (*IMAlign_script*):
 - 3.1 Load the IMAlign project *IMBlock_Aligned*, in the active workspace.
 - 3.2 Perform a best-fit alignment.
 - 3.3 Get the number of scans in the project.
 - 3.4 Save the project.
4. Close the instance of the IMAlign module.
5. Specify notes for the IMAlign project.

Step B. To do in IMMerge:

1. Open the IMMerge module.
2. Specify the IMAlign project (*IMBlock_Aligned*, in the active workspace) and the desired parameters for IMMerge.
3. Start the merging task. The output polygonal model, *Model.pol*, is written to the associated workspace.
4. Close IMMerge.

Step C. To do in PolyWorks|Modeler:

1. Open an instance of the PolyWorks|Modeler module.
2. Set a configuration parameter.
3. Load the project *Empty_Modeler_Project*, in the active workspace.
4. Run a PolyWorks|Modeler script (*Modeler_script*):
 - 4.1 Import a polygonal model: *Model.pol*.
 - 4.2 Perform a hole- and gap-filling operation.
 - 4.3 Export the edited polygonal model to the associated workspace as *Clean_model.pol*.
 - 4.4 Delete the polygonal model and save the project.
5. Close the instance of the PolyWorks|Modeler module.

9.5.2 The macro scripts to create

Carrying out the desired tasks requires creating three scripts:

- *WM_script* – The main script that drives the entire process, written in the Macro Script Editor tool of the Workspace Manager.
- *IMAlign_script* – A script written in the Macro Script Editor tool of the IMAlign module.
- *Modeler_script* – A script written in the Macro Script Editor tool of the PolyWorks|Modeler module.

Note that when an instance of a module (i.e., IMAlign, PolyWorks|Modeler, PolyWorks|Inspector) or PolyWorks|Reviewer is executed, it is given a unique ID number. This number will be stored in variables and used in certain commands to identify a specific instance of a module. Note that the scripts also make use of a shared variable to store the number of scans in an IMAlign project and to edit the notes of an IMAlign project by way of the Workspace Manager. For help on a particular command syntax, or to see similar or related commands, see the Command Reference.

Here are the three scripts in order. The WM_script content is as follows:

```
# Beginning of macro: WM_script
# declare variables
DECLARE vIMAlign_unique_ID
DECLARE vModeler_unique_ID
DECLARE vWS_path "C:\Multi-applicationScript.pwk"
DECLARE vNotes_IMAlign
MACRO SHARED_VARIABLE DELETE ("svNb_images")
MACRO SHARED_VARIABLE CREATE ("svNb_images", 0)

# >>>> STEP A - DO THE WORK IN IMALIGN <<<<<<<<<<
MODULE IMALIGN START ("vWS_path",vIMAlign_unique_ID,"On")
MACRO EXEC REMOTE_COMMAND \
("CONFIG MODIFY LOAD_APPLICATION_SETTINGS_FROM_PROJECT", \
    $vIMAlign_unique_ID,"Yes")

# Execute the IMAlign macro script
MACRO EXEC REMOTE_SCRIPT ("IMAlign_script", $vIMAlign_unique_ID)
MODULE CLOSE ($vIMAlign_unique_ID, "On")

# Set the notes of the IMAlign project
TREEVIEW OBJECT SELECT NONE
TREEVIEW IMALIGN_PROJECT SELECT (1, "On")
MACRO SHARED_VARIABLE GET_VALUE ("svNb_images", vNotes_IMAlign)
TREEVIEW OBJECT PROPERTIES NOTES \
    ("vWS_path",,"This project contains ${vNotes_IMAlign} scans.")

# >>>>>> STEP B - DO THE WORK IN IMMERGE <<<<<<<<<<<
MODULE IMMERGE MERGE_IMALIGN_PROJECT \
    ($vWS_path,"IMBlock_Aligned",,, "Model.pol")

# >>>>>> STEP C - DO THE WORK IN POLYWORKS|MODELER <<<<<<<<<<
MODULE POLYWORKS_MODELER START ("vWS_path",
vModeler_unique_ID, "On")
MACRO EXEC REMOTE_COMMAND \
("CONFIG MODIFY LOAD_APPLICATION_SETTINGS_FROM_PROJECT", \
    $vModeler_unique_ID, "Yes")
MACRO EXEC REMOTE_COMMAND ("FILE OPEN_PROJECT_IN_PWK", \
    $vModeler_unique_ID, "vWS_path", "Empty_Modeler_Project")
# Execute the Modeler macro script
MACRO EXEC REMOTE_SCRIPT ("Modeler_script", $vModeler_unique_ID)
MODULE CLOSE ($vModeler_unique_ID, "On")

# HOUSEKEEPING
MACRO SHARED_VARIABLE DELETE ("svNb_images")

# End of WM_SCRIPT
```

The IMAAlign_script content:

```
# Beginning of macro: IMAAlign_script
DECLARE vName
DECLARE vNumber_images_in_project
# Open an IMAAlign project in the active workspace
FILE OPEN_PROJECT_IN_PWK (,"IMBlock_Aligned")
VIEW POSE CENTER
# Perform a Best-Fit - Evaluation Only
TREEVIEW OBJECT SELECT ALL
ALIGN OPTIONS SUBSAMPLING ("1/1")
ALIGN OPTIONS MAX_ITERATION (0)
ALIGN START (0.000001, 0)
# Get the number of 3D images in the project
TREEVIEW IMAGE COUNT GET (vNumber_images_in_project)
# Use of shared variables
MACRO SHARED_VARIABLE SET_VALUE ("svNb_images", \
    $vNumber_images_in_project)
FILE SAVE_PROJECT ( , )
# End of IMAAlign_script
```

The Modeler_script content:

```
# Beginning of macro: Modeler_script
DECLARE vWS_path "C:\Multi-applicationScript.pwk"
# Open a PolyWorks|Modeler project in the active workspace
FILE OPEN_PROJECT_IN_PWK (,"Empty_Modeler_Project")
VIEW AXES LOWER_LEFT_CORNER
FILE IMPORT_MODEL POLYGONAL_FILE_IN_PWK ($vWS_path,"Model.pol")
TREEVIEW MODEL SELECT (1, "On")
EDIT TRIANGLES FILL_IN_HOLES_AND_GAPS AUTOMATIC (5.0, 0.0, 0.0, 10)
TREEVIEW MODEL SELECT (1, "On")
FILE EXPORT_OBJECT POLYGONAL_MODEL_IN_PWK \
    ($vWS_path, "Clean_model.pol",)
TREEVIEW MODEL SELECT (1, "On")
EDIT OBJECT DELETE
FILE SAVE_PROJECT ( , )
# End of Modeler_script
```

9.5.3 Running the script

Load the main macro script, WM_script, in the Macro Script Editor tool of the Workspace Manager and execute it. It starts the other modules and executes scripts written in their own Macro Script Editor tools. On completion, a simple multiapplication task has been performed.

Glossary

#	Symbol used in dialog boxes to represent the term “Number (of)”. For example, # displayed points should be read as “Number of displayed points”.
MSCL	An abbreviation that stands for the PolyWorks Macro Script Control Language.
NA	An abbreviation that stands for Not Applicable. Used in tables where no value can be attributed for a given item.
Statement	One or more commands and language elements on the same line that can be executed by the Macro Script Editor. For this reason, a statement is also said to be the basic unit of a macro script.
Workspace	A workspace is composed of a descriptive XML file and a folder containing files and projects. The container holds certain files input to PolyWorks Metrology Suite modules (e.g., polygonal models and point clouds) as well as files output as part of the PolyWorks process (e.g., projects, polygonal models). The XML file keeps track of which files/projects are used by which project (notion of dependency). This information is used in copy/paste, delete, and file open operations performed in the Workspace Manager.
Workspace Manager	The Workspace Manager allows managing your data by way of workspaces, in addition to launching PolyWorks Metrology Suite applications. It also allows configuring PolyWorks options and entering license keys.

Index

A

- abs() 85
- acos() 85
- application commands
 - help for 41
- arguments, input and output 74
- arrays
 - finding the size of 78
 - implicit indexes 78
 - in a sample macro script 151
 - indexes 77
- asin() 85
- assigning values to variables 71
- atan() 85

B

- BREAK 96

C

- calling other macro scripts 138
- Command History pane
 - Command Line area 53
- Command reference documentation 41
- comments in a macro script 63, 66
- CONTINUE 96
- converting data types 101
- cos() 85
- creating variables 70

D

- DATA_FILE 127
- DECLARE 70
- declaring variables 70

E

- exiting a macro 144
- EXPR 84, 86
- EXPR_I 84, 86

F

- false, definition of 90
- file-parsing commands 127

G

- GET 109

I

- IF-ELSE-ENDIF structure 93
- INPUT 116
- input arguments 74

J-L

- log10() 85
- logical expressions 90
- looping in a macro script 95

M

- MACRO END 144
- MACRO EXEC 138
- MACRO GET_ERROR_STATUS 144
- MACRO PAUSE 126
- Macro Script Control Language. See MSCL
- Macro Script Editor
 - editing macro scripts 43
 - interface 23

- menus 24
 - recording macro scripts 58
 - running macro scripts 61
- macro scripts
 - accessing check and text boxes 109
 - adding comments to 67
 - and programming 64 to 65
 - creating 59
 - data conversion in 101
 - editing 61
 - general rules that apply to 62, 66
 - running 61
 - testing 61
 - writing commands over more than one line 67
- MACRO SLEEP 127
- MACRO STRING 130
- mathematical expressions 84, 86
- mathematical functions 85
- modular programming 138
- MSCL 67
 - a programming language 65
 - DECLARE 70
 - EXPR 84, 86
 - EXPR_I 84, 86
 - general rules 62, 66
 - IF-ENDIF structure 93
 - input and output arguments 73
 - keywords 68
 - logical expressions 90
 - mathematical expressions 84, 86
 - SET 71
 - SIZE 78
 - statement format 67
 - variables 69 to 75
 - VERSION 63, 66
 - WHILE-ENDWHILE structure 95

O

- object tree, commands that work with the 107
- obtaining information from the user 116
- output arguments 74

P

- pow() 85

Q

- querying the user 116

S

- SET 71
- sin() 85
- SIZE 78
- sqrt() 85
- storing and requerying parameters in a text file 127

T

- tan() 85
- TREEVIEW 107
- true, definition of 90

V-Z

- variables
 - assigning values to 71
 - declaring 70
 - decrementing 82
 - incrementing 82
 - MSCL 69
 - reserved 80
 - scope of 97
 - types 69, 71
 - using value stored in 72
- VERSION 63, 66
- WHILE-ENDWHILE structure 95
- WINDOW REFRESH 115